ARM CORTEX-M EMBEDDED SYSTEMS TECHNICAL TRAINING

STM32 Complete Professional Reference

The Definitive Technical Textbook & Practical Training Manual

A rigorous, hardware-level and HAL-level consolidation of STM32 microcontrollers, development workflows, direct memory access (DMA), vectored interrupts, multi-channel timers, serial communication protocols (UART/SPI/I^2C), analog-to-digital conversion, reusable modular driver architecture, digital signal processing, and FreeRTOS multitasking.

Institutional Batch	Assiut Robotics Embedded Systems Training Program
Baseline Toolchain	STM32CubeIDE 2.2.0 • STM32CubeMX 6.18.1 • GNU ARM Toolchain
Primary Target Silicon	STM32F103C8T6 (Cortex-M3, 72 MHz) with STM32F4/G4/H7 extensions
Document Edition	Version 5.0 (Consolidated Master Edition)
Status	Production Technical Reference — Verified Against Official Datasheets & RMs

STM32 Complete Professional Reference

Definitive Technical Textbook and Practical Training Manual

From STM32CubeIDE 2.2.0 and STM32CubeMX 6.18.1 configuration to deterministic real-time data acquisition: HAL, interrupts, DMA, timers, UART/SPI/I²C, ADC, reusable sensor drivers, digital filtering and FreeRTOS.

Document Information

Table 0.1 — Document information and baseline

Attribute	Specification	Scope and notes
Title	STM32 Complete Professional Reference	Consolidated university-level textbook and field manual
Version	5.0 (consolidated master edition)	Supersedes the seven source documents listed in Appendix A
Toolchain baseline	STM32CubeIDE 2.2.0, STM32CubeMX 6.18.1, GNU Arm Embedded toolchain (arm-none-eabi-gcc)	Menu names may differ slightly between tool releases; see Appendix B
Primary hardware	STM32F103C8T6 (Cortex-M3, 72 MHz max)	All worked numbers use this part unless stated otherwise
Secondary hardware	STM32F4 (Cortex-M4F), notes for G0/G4/L4/H7/U5	Differences are called out explicitly
Audience	Embedded and robotics students, instructors, junior firmware engineers	Self-study or a 90-minute, 2.5-hour or 6-hour workshop
Method	Problem → concept → decision → CubeMX → code → verify → debug	Every topic follows this loop (Section 1.3)

> **KEY CONCEPT:** Every listing is a pattern, not a drop-in. STM32 is a family of families. Pin mappings, clock trees, DMA request routing, ADC trigger sources, HAL function availability and register names differ between parts. Before wiring hardware or trusting a calculation, open the datasheet and reference manual for your exact part number. Items that must be checked on your device are marked Verify and collected in Appendix B.

Table of Contents

- 1. Introduction, Learning Objectives and the System View
  - 1.1 What an STM32 Really Is
  - 1.2 Learning Objectives
  - 1.3 How to Use This Reference
    - 1.3.1 The seven-step engineering loop
    - 1.3.2 Suggested delivery schedules
  - 1.4 The System as a Ten-Layer Chain
  - 1.5 Conventions Used in This Reference
- 2. Part 1 — STM32 Development Workflow
  - 2.1 STM32 Internal Architecture and the Bus Matrix
  - 2.2 The Toolchain Ecosystem
  - 2.3 Installation and Setup
  - 2.4 Creating a Project
  - 2.5 The `.ioc` File and the CubeMX Workspace
  - 2.6 GPIO Architecture, Alternate Functions and Electrical Behavior
  - 2.7 Clock Tree, PLL and the APB Timer ×2 Rule
  - 2.8 Code Generation, Project Structure and USER CODE Preservation
  - 2.9 HAL, LL and Direct Register Programming
    - 2.9.1 What each layer is
    - 2.9.2 One operation, three layers
    - 2.9.3 Comparison and selection
  - 2.10 Debugging Workflow in CubeIDE
- 3. Part 2 — Interrupts, DMA and Timers
  - 3.1 Polling
  - 3.2 Interrupts
    - 3.2.1 Definition and purpose
    - 3.2.2 Architecture on Cortex-M
    - 3.2.3 From IRQ to callback in HAL
    - 3.2.4 Priorities, preemption and nesting
    - 3.2.5 Strict ISR rules
  - 3.3 DMA — Direct Memory Access

        - 3.3.1 Definition and purpose
        - 3.3.2 What actually happens during a transfer
        - 3.3.3 Controller structure and configuration parameters
        - 3.3.4 Modes and buffer ownership
        - 3.3.5 DMA with the common peripherals
        - 3.3.6 Common DMA mistakes
    - 3.4 Polling vs. Interrupts vs. DMA
    - 3.5 When Should DMA Be Used?
    - 3.6 Timers
        - 3.6.1 Fundamentals
        - 3.6.2 Timer mathematics
        - 3.6.3 PWM generation
        - 3.6.4 Input capture (measuring frequency and pulse width)
        - 3.6.5 Output compare, one-pulse and encoder modes
        - 3.6.6 Timer trigger output (TRGO), interrupts and DMA
        - 3.6.7 Timer pitfalls
        - 3.6.8 Choosing a timer feature
        - 3.6.9 Watchdogs, RTC and SysTick compared with timers
    - 3.7 Integrated Example: Timer-Triggered ADC with Circular DMA
        - 3.7.1 Design decisions before touching CubeMX
        - 3.7.2 Configuration (STM32F103: ADC1 + TIM3 TRGO + DMA1 channel 1)
        - 3.7.3 The half-buffer deadline
        - 3.7.4 Verification
- 4. Part 3 — Serial Communication Protocols: UART, SPI, and I^2C
    - 4.1 UART / USART (Universal Asynchronous / Synchronous Receiver Transmitter)
        - 4.1.1 Operating Principle and Physics
        - 4.1.2 Baud Rate Generation and Clock Tolerances
        - 4.1.3 HAL Implementation: Polling, Interrupt, and DMA Modes
        - 4.1.4 The UART Stream Problem: Framing and Packet Delineation
        - 4.1.5 UART Hardware Errors and Failure Modes
        - 4.1.6 Retargeting `printf` via UART
    - 4.2 SPI (Serial Peripheral Interface)

        - 4.2.1 Operating Principle and Architecture
        - 4.2.2 The Shift Register Circular Ring
        - 4.2.3 Clock Polarity (CPOL) and Clock Phase (CPHA): The Four SPI Modes
        - 4.2.4 Hardware NSS vs. Software Chip Select (CS)
        - 4.2.5 SPI Timing Calculations and Maximum Baud Rates
    - 4.3 I^2C (Inter-Integrated Circuit)
        - 4.3.1 Operating Principle: The Open-Drain Wired-AND Bus
        - 4.3.2 Packet Structure: START, STOP, and ACK/NACK
        - 4.3.3 The 7-Bit Address Left-Shift Requirement in STM32 HAL
        - 4.3.4 Physical Layer Mathematics: Sizing Pull-Up Resistors (R_p)
        - 4.3.5 HAL Sensor Communication: `HAL_I2C_Mem_Read` vs. `HAL_I2C_Master_Receive`
        - 4.3.6 The Famous I^2C Bus Lockup and 9-Clock Hardware Recovery
    - 4.4 Comprehensive Protocol Comparison and Selection Matrix
        - 4.4.1 Industrial Protocols Overview: CAN, RS-485, USB, and Ethernet
        - 4.4.2 Practical Engineering Selection Scenarios
        - 4.4.3 Architectural Decision Flowchart
- 5. Part 4 — Sensors, ADC, and Real-Time Data Acquisition
    - 5.1 The Sensor Taxonomy
    - 5.2 Analog-to-Digital Converter (ADC) Fundamentals
        - 5.2.1 Resolution, Reference Voltage (V_{REF}), and Quantization
        - 5.2.2 The Sampling Switch, R_{AIN}, and Settling Dynamics
        - 5.2.3 Conversion Time and ADC Clock Limits
    - 5.3 Internal ADC Channels: V_{REFINT}, Temperature Sensor, and V_{BAT}
        - 5.3.1 Internal Voltage Reference (V_{REFINT})
        - 5.3.2 Internal Temperature Sensor
    - 5.4 ADC Acquisition Modes in STM32 HAL
        - 5.4.1 Mandatory Calibration Step
    - 5.5 Digital Sensor Interrogation: Registers, Burst Reads, and Conversions
        - 5.5.1 The Sensor Initialization Lifecycle
        - 5.5.2 Two's Complement Decoding and Unit Conversion
    - 5.6 Real-Time Acquisition Dynamics: Latency, Jitter, and Throughput

        - 5.6.1 Throughput and Bandwidth Mathematics
        - 5.6.2 Memory Buffering Strategies
    - 5.7 Digital Signal Processing: Anti-Aliasing and Filtering
        - 5.7.1 The Nyquist-Shannon Sampling Theorem and Anti-Aliasing
        - 5.7.2 Moving Average vs. Exponential Moving Average (EMA)
    - 5.8 Cortex-M7 and STM32H7 Data-Cache (D-Cache) Coherency
        - The Two Incoherency Hazards
        - The Three Architectural Solutions
- 6. Part 5 — Sensor Drivers and Reusable Embedded Software
    - 6.1 What is a Sensor Driver and Why Decouple It?
        - The Dangers of Tightly Coupled Code
    - 6.2 The Four-Layer Driver Architecture
    - 6.3 The 10 Golden Rules of Embedded Driver Design
    - 6.4 Professional Driver Implementation: InvenSense MPU6050
        - 6.4.1 Public Header File: `mpu6050.h`
        - 6.4.2 Driver Implementation: `mpu6050.c`
        - 6.4.3 Application Usage and HAL Binding
    - 6.5 Automated Unit Testing and Desktop Mocking
- 7. Part 6 — Real-Time Operating Systems (RTOS) and FreeRTOS
    - 7.1 Bare-Metal Superloops vs. RTOS: When is an RTOS Justified?
        - 7.1.1 The Anatomy of a Bare-Metal Superloop
        - 7.1.2 The RTOS Justification Matrix
    - 7.2 Core RTOS Architecture and Mechanics
        - 7.2.1 Tasks, Stacks, and the Task Control Block (TCB)
        - 7.2.2 Preemptive Scheduling and Rate Monotonic Intuition
        - 7.2.3 Context Switching on ARM Cortex-M: PendSV and SysTick
    - 7.3 Inter-Task Communication and Synchronization Primitives
        - 7.3.1 Priority Inversion and Priority Inheritance in Mutexes
    - 7.4 Interrupts and FreeRTOS: The Critical Priority Rule
        - 7.4.1 Strict ISR API Rules
        - 7.4.2 The `configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY` Rule
    - 7.5 Production Example: Thread-Safe Multi-Task Architecture

- 7.6 Critical CubeMX Setup: The HAL Timebase Source Conflict
  - The Required CubeMX Configuration
- 8. Capstone Architecture, Debugging Playbook, and Bench Practice
  - 8.1 The Multi-Sensor Robotic Capstone Architecture
    - Architectural Guarantees
  - 8.2 HardFault Forensics on ARM Cortex-M
    - 8.2.1 The Exception Stack Frame
    - 8.2.2 Decoding the Fault Status Registers
    - 8.2.3 Production HardFault Handler Implementation
  - 8.3 The 10-Layer Root-Cause Debugging Playbook
  - 8.4 Per-Subsystem Failure Diagnostic Checklists
    - 8.4.1 GPIO and Smoke-Test Checklist
    - 8.4.2 UART / USART Checklist
    - 8.4.3 I^2C Bus Checklist
    - 8.4.4 SPI Bus Checklist
    - 8.4.5 ADC Analog Front-End Checklist
    - 8.4.6 Timers and PWM Checklist
  - 8.5 Seven Practical Hands-On Labs and Capstone
    - Lab 1 — GPIO Heartbeat and Oscilloscope Clock Verification
    - Lab 2 — Interactive UART Telemetry Console with IDLE Line DMA
    - Lab 3 — Precision Timer Calculations and PWM Oscilloscope Verification
    - Lab 4 — ADC Polling and Dynamic Supply Compensation
    - Lab 5 — Deterministic Timer-Triggered ADC with Circular DMA
    - Lab 6 — Reusable InvenSense MPU6050 Sensor Driver Integration
    - Lab 7 — FreeRTOS Dual-Task Producer-Consumer Telemetry Pipeline
  - 8.6 Master Engineering Formula Sheet
  - 8.7 20-Question Technical Knowledge Check with Comprehensive Answers
- 9. Master Reference and Comparison Tables
  - 9.1 STM32 Microcontroller Families At a Glance
  - 9.2 Comprehensive Communication Protocol Matrix
  - 9.3 Execution and Data Transfer Paradigms
- 10. Conclusion and Recommended Learning Path

    - The Engineering Mindset: Observe and Measure
- 11. Comprehensive Technical Glossary
- 12. References and Official Documentation
    - 12.1 STMicroelectronics Official Documentation
    - 12.2 ARM Architecture Documentation
    - 12.3 FreeRTOS Documentation
    - 12.4 Industry Standards and Sensor Specifications
- Appendix A — Source Coverage Checklist and Reconciliation Log
    - A.1 Source File Coverage Checklist
    - A.2 Reconciled Contradictions and Technical Corrections
- Appendix B — Hardware-Specific Verification Items

List of Figures, Tables, and Code Listings

Figures

- **Figure 1.1**: (A) Ten-layer physical-to-application chain
- **Figure 2.1**: (A) `
- **Figure 3.1**: (A) Interrupt execution flow from peripheral event to exception return
- **Figure 3.2**: Execution paradigms: (1) blocking polling
- **Figure 3.3**: (A) Timer core: kernel clock → PSC → CNT with ARR and CCR
- **Figure 4.1**: Serial communication architectures: (A) Asynchronous UART with framing and start/stop bits
- **Figure 5.1**: Analog data acquisition: (A) Complete front-end signal chain from physical transducer to DMA buffer
- **Figure 6.1**: Software architecture: (A) Four-layer reusable driver hierarchy showing complete decoupling of hardware bus from application logic

Tables

- **Table 0.1**: Document information and baseline
- **Table 1.1**: STM32 families and cores (selected)
- **Table 1.2**: Six-hour evening workshop plan
- **Table 1.3**: Condensed schedules
- **Table 1.4**: The ten layers, their questions and typical failures
- **Table 2.1**: Buses on an STM32F1/F4-class device
- **Table 2.2**: Tool and artifact responsibilities versus misconceptions
- **Table 2.3**: CubeMX tabs and what they control

- **Table 2.4**: GPIO operating modes
- **Table 2.5**: Clock sources and nodes
- **Table 2.6**: HAL vs. LL vs. direct registers
- **Table 3.1**: DMA architecture by family
- **Table 3.2**: Per-channel configuration fields (CubeMX DMA settings)
- **Table 3.3**: Trade-offs
- **Table 3.4**: Starting point by requirement
- **Table 3.5**: Timer categories
- **Table 3.6**: Timer calculations
- **Table 4.1**: UART message framing strategies
- **Table 4.2**: The four standard SPI modes
- **Table 4.3**: I^2C debugging and symptom diagnostic matrix
- **Table 4.4**: Comprehensive technical comparison of serial protocols
- **Table 5.1**: Embedded sensor interface taxonomy
- **Table 5.2**: Comparison of ADC acquisition modes
- **Table 5.3**: Embedded data buffering architectures
- **Table 6.1**: The 10 golden rules of embedded driver architecture
- **Table 7.1**: Superloop vs. RTOS selection criteria
- **Table 7.2**: Good vs. poor reasons for adopting an RTOS
- **Table 7.3**: FreeRTOS synchronization and communication primitives
- **Table 8.1**: Cortex-M fault status registers and bit definitions
- **Table 8.2**: Master engineering formula reference sheet
- **Table 9.1**: Major STM32 microcontroller series comparison
- **Table 9.2**: Master comparison of serial, industrial, and networking protocols
- **Table 9.3**: Trade-offs between Polling, Interrupts, DMA, and RTOS

Code Listings

- **Listing 2.1**: Generated clock setup for F103 at 72 MHz (HSE 8 MHz, PLL ×9)
- **Listing 2.2**: main.c skeleton showing initialization order and USER CODE regions
- **Listing 2.3**: Setting PC13 low with HAL, LL and registers
- **Listing 2.4**: Bare-register PC13 setup on F1 (for understanding only)
- **Listing 3.1**: Polled ADC read with timeout
- **Listing 3.2**: Debounced button using EXTI

- **Listing 3.3**: Memory-to-memory DMA copy (names depend on the channel CubeMX generated)
- **Listing 3.4**: Periodic 1 ms timer interrupt
- **Listing 3.5**: Servo (50 Hz, 1–2 ms pulse) and 20 kHz motor PWM
- **Listing 3.6**: Frequency measurement with input capture (1 µs tick, 16-bit counter)
- **Listing 3.7**: 1 kHz square wave from output compare (toggle), 1 µs tick
- **Listing 3.8**: Timer-triggered ADC with circular DMA, ping-pong processing and overrun counting
- **Listing 4.1**: Robust variable-length packet reception via DMA and IDLE detection
- **Listing 4.2**: Comprehensive UART error handler callback
- **Listing 4.3**: Retargeting standard I/O to USART2 with GCC / Newlib-nano
- **Listing 4.4**: Reusable SPI register read/write transaction with Software CS
- **Listing 4.5**: Efficient sensor register read and write operations
- **Listing 4.6**: I^2C GPIO unjam / bus recovery procedure
- **Listing 5.1**: High-efficiency fixed-point integer EMA filter (Zero floating-point overhead)
- **Listing 6.1**: Production header file mpu6050.h
- **Listing 6.2**: Production source file mpu6050.c
- **Listing 6.3**: Application integration in main.c
- **Listing 6.4**: Desktop PC unit test mock runner
- **Listing 7.1**: Production FreeRTOS dual-task telemetry architecture
- **Listing 8.1**: Assembly stub and C diagnostic handler for HardFault forensics

1. Introduction, Learning Objectives and the System View

1.1 What an STM32 Really Is

An STM32 is a **system on chip (SoC)**, not just a CPU. Inside one package you find:

- an ARM Cortex-M processor core (with NVIC interrupt controller and SysTick timer),
- on-chip Flash (program and constants) and SRAM (variables, stacks, buffers),
- a **bus matrix** that connects bus masters (CPU, DMA) to memories and peripherals,
- a clock system (oscillators, PLL, prescalers) that feeds every block,
- peripherals: GPIO, timers, ADC/DAC, UART/USART, SPI, I^2C, CAN, USB and more,
- a debug port (SWD/JTAG) used by ST-LINK or J-Link.

Firmware is the code that configures these blocks and moves data between them. Almost every embedded problem is one of three problems: **configuring hardware correctly**, **moving data without losing it**, or **meeting a deadline**.

Table 1.1 — STM32 families and cores (selected)

Core	Example families	Characteristics
Cortex-M0 / M0+	STM32C0, G0, L0, F0	Low cost, low power, small footprint
Cortex-M3	STM32F1, F2, L1	Deterministic, no FPU; F103 is the classic 72 MHz part
Cortex-M4F	STM32F3, F4, G4, L4	Hardware FPU and DSP instructions
Cortex-M7	STM32F7, H7	L1 caches, dual-issue pipeline, up to hundreds of MHz
Cortex-M33	STM32U5, H5, L5	TrustZone security, FPU/DSP

Reading a part number (example STM32F103C8T6): F1 = family and core generation, 03 = line/feature set, C = 48 pins, 8 = 64 KB Flash, T = LQFP package, 6 = −40 to 85 °C. Always confirm the exact variant on the chip marking and in the datasheet; Flash and SRAM sizes drive linker scripts.

1.2 Learning Objectives

After studying this reference you should be able to:

1. **Use the toolchain:** create, configure, generate, build, flash and debug an STM32 project in CubeMX/CubeIDE, and keep your own code safe across regenerations.
2. **Explain HAL, LL and register access** and choose the right level for a task.
3. **Choose between polling, interrupts and DMA** using CPU load, latency and data-rate arguments, and implement each.

4. **Configure timers** (periodic events, PWM, input capture, output compare, ADC triggering) and compute PSC/ARR/CCR values.
5. **Implement and select UART, SPI and I²C**, including framing, modes, addressing, pull-up sizing and recovery.
6. **Acquire analog and digital sensor data in real time**: ADC equations, sampling time, buffers, filtering and data integrity.
7. **Write reusable sensor drivers** with clean APIs, error handling and hardware abstraction.
8. **Decide when an RTOS is justified** and use FreeRTOS tasks, queues, semaphores and mutexes correctly alongside interrupts.
9. **Debug systematically** from power and clocks up to application policy, including HardFault analysis.

1.3 How to Use This Reference

1.3.1 The seven-step engineering loop

The central teaching idea of the session is that learners need **technical judgment**, not a vocabulary list. A representative student remark (Arabic, from the session overview) captures it: the goal is not to explain DMA and ADC from scratch, but to know *when to think about DMA and when plain ADC reads are enough*. Every topic therefore follows the same loop:

1. **Define the hardware problem**: what physical signal exists, what must firmware produce, how often, and what happens on failure.
2. **Understand the mechanism**: why the hardware block exists and which bottleneck it removes.
3. **Apply decision criteria**: polling vs. interrupt vs. DMA, UART vs. SPI vs. I²C, superloop vs. RTOS.
4. **Consult the datasheet and reference manual**, then configure in CubeMX.
5. **Implement** modular C that checks every status, respects buffer ownership and never blocks critical paths.
6. **Verify with instruments**: oscilloscope, logic analyzer, multimeter or a diagnostic GPIO. A calculation without a measurement is a hypothesis.
7. **Anticipate failure modes**: configuration traps, electrical faults, race conditions, bus lockups.

1.3.2 Suggested delivery schedules

Table 1.2 — Six-hour evening workshop plan

Time	Block	Minutes	Key topics
18:00–18:30	Arrival and toolchain check	30	CubeIDE, CubeMX, MCU packages, ST-LINK drivers, hardware
18:30–19:10	Part 1: workflow and architecture	40	.ioc, clock tree, GPIO limits, code generation, HAL vs. LL vs. registers

Time	Block	Minutes	Key topics
19:10–20:00	Part 2: interrupts, DMA, timers	50	Polling vs. IRQ vs. DMA, NVIC, timer math, TRGO → ADC → DMA
20:00–20:30	Prayer and rest break	30	Adjust to local prayer times; informal Q&A
20:30–21:10	Part 3: UART, SPI, I^2C	40	Framing, baud math, SPI modes, I^2C pull-ups, protocol selection
21:10–21:45	Part 4: sensors, ADC, DSP	35	ADC equations, source impedance, MPU6050, throughput, filters
21:45–22:20	Part 5: reusable drivers	35	Layered architecture, dependency injection, status codes
22:20–22:45	Part 6: FreeRTOS	25	Superloop vs. RTOS, tasks, queues, semaphores, mutexes, ISR rules
22:45–23:15	Part 7: integrated capstone	30	Multi-sensor monitoring system
23:15–00:00	Debugging, labs, buffer	45	Live troubleshooting, logic analyzer, HardFault, Q&A

Table 1.3 — Condensed schedules

Module	2.5-hour session	90-minute session
CubeIDE, CubeMX, .ioc	15 min (workflow, USER CODE preservation)	10 min (essential .ioc, clocks, codegen)
Timers, interrupts, DMA	30 min (selection criteria, timer + ADC + DMA)	25 min (polling vs. IRQ vs. DMA, timer math)
Communication protocols	20 min (trade-offs, scenarios)	15 min (decision matrix, pitfalls)
Sensors and acquisition	25 min (ADC math, sampling, buffering)	15 min (scaling, source impedance, ping-pong)
Reusable drivers	25 min (architecture, C pattern)	15 min (header/source API)
DSP and FreeRTOS	25 min (10 min filters + 15 min RTOS)	10 min (overview)
Integrated system and Q&A	10 min	Included in wrap-up

> **ENGINEERING NOTE:** Instructor rule. Do not spend the session explaining every feature of every peripheral. Spend it explaining how to choose the right feature for a real engineering problem. End by giving the class a realistic requirement and asking them to design the STM32 architecture and justify each choice.

1.4 The System as a Ten-Layer Chain

Firmware does not "talk to a sensor" in one step. It configures analog and digital silicon so that a physical quantity becomes a conditioned electrical signal, then a code in a peripheral register, then a word in SRAM, then a validated value in SI units, and finally an actuation command or telemetry packet. **If you cannot sketch this chain on paper, you cannot debug it systematically.**

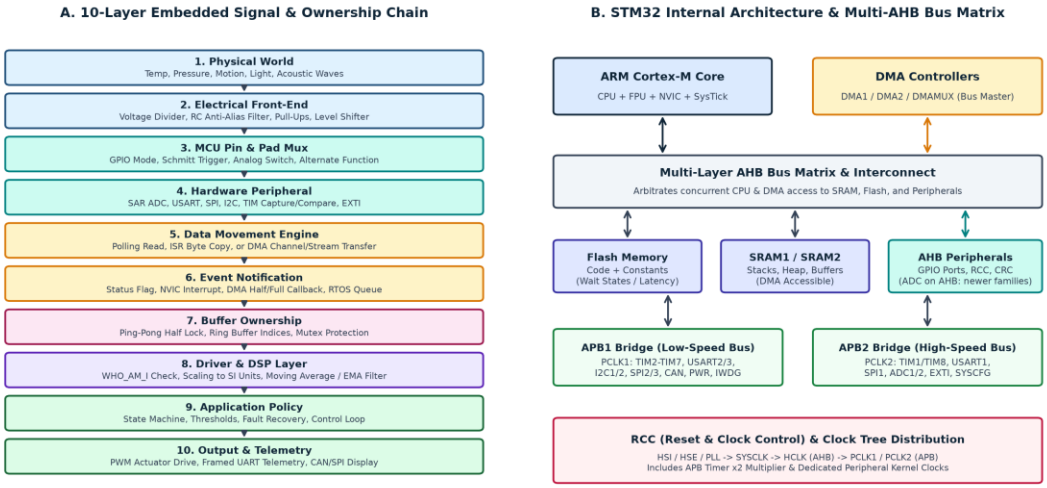


Figure 1.1 — Ten-layer signal and ownership chain, and STM32 internal bus matrix

Figure 1.1 — (A) Ten-layer physical-to-application chain; (B) STM32 internal multi-layer AHB bus matrix and clock distribution.

Table 1.4 — The ten layers, their questions and typical failures

Layer	Core engineering question	Failure if ignored	Concrete STM32 example
1. Physical world	What quantity, range and bandwidth?	Sensor saturates; vibration aliases	NTC over 0–100 °C; 1 kHz motor-current ripple
2. Electrical network	What impedance, voltage range, filtering, grounding?	ADC sample capacitor never settles; 5 V damages a 3.3 V pin	100 kΩ divider without buffer; missing I^2C pull-ups
3. Pin and pad mux	Which pin, mode and alternate function?	Schmitt trigger loads an analog signal; peripheral not connected	ADC pin left in digital input mode instead of GPIO_MODE_ANALOG
4. Peripheral engine	Which block converts or shifts the signal, at what clock?	Wrong baud, SPI phase or ADC clock	ADC channel 5 configured, sensor wired to channel 4

Layer	Core engineering question	Failure if ignored	Concrete STM32 example
5. Data movement	Polling, ISR copy or DMA?	CPU misses deadlines; DMA writes the wrong width	32-bit DMA words into a 16-bit array corrupt adjacent RAM
6. Event notification	When is data valid and how is software told?	Callback never fires because the IRQ is disabled	DMA half-transfer (HT) and transfer-complete (TC) interrupts
7. Buffer ownership	Which context owns the region until when?	Torn reads; circular DMA overwrites live data	Main loop reads adc_buf[] while DMA wraps
8. Driver and DSP	How are raw codes validated, scaled, filtered?	Stale data after a NACK; raw 2048 treated as 20.48 °C	WHO_AM_I check, two's-complement decode, EMA filter
9. Application policy	What states, thresholds and recovery?	System hangs when a sensor is unplugged	Hysteresis alarm with bounded retries
10. Output / telemetry	How is the result delivered; can the link keep up?	Blocking UART print in a 10 kHz ISR destroys timing	Framed CSV (seq,t_ms,temp_c,flags) via non-blocking DMA

WARNING: Data ownership is a first-class design item. Ownership defines which execution context (main loop, ISR, DMA controller, RTOS task) may read or write a buffer at any instant. A volatile ready flag is not a lock: it neither freezes a circular DMA controller nor makes multi-byte reads atomic. Use ping-pong buffers, single-writer/single-reader ring buffers, or RTOS queues and mutexes.

1.5 Conventions Used in This Reference

- **Reference part:** STM32F103C8T6 at 72 MHz (HSE 8 MHz crystal, PLL ×9). Numbers for other families are labeled.
- **Code style:** C99, STM32 HAL naming, static for file-local state, volatile only for ISR-shared flags, explicit return-status checks. Generated CubeMX names (hadc1, htim3, huart2, hi2c1, hspi1) are assumed; adapt them to your project.
- **Callouts:** *Key concept*, *Engineering note*, *Practical tip*, *Warning*, *Common mistake / danger*.
- **Units:** frequencies in Hz, times in seconds (µs, ms, ns), voltages in volts, data rates in bytes per second unless stated.
- **Verification policy:** statements about silicon that differ across families are flagged, never silently generalized.

2. Part 1 — STM32 Development Workflow

2.1 STM32 Internal Architecture and the Bus Matrix

A modern STM32 does not route every access through one bottlenecked bus. Bus **masters** (the Cortex-M core through its I-Code, D-Code and System buses, and the DMA controllers) are connected through a **multi-layer AHB bus matrix** to bus **slaves** (Flash, SRAM banks, AHB peripherals and the APB bridges). Because the matrix is a crossbar, the CPU can fetch instructions from Flash while a DMA controller moves ADC samples into SRAM, without the two stalling each other, unless both target the same slave in the same cycle.

Table 2.1 — Buses on an STM32F1/F4-class device

Bus	Hosts (examples)	Typical limit	Notes
AHB (HCLK)	Core, DMA, SRAM, Flash interface, GPIO on F4, RCC, CRC	F103: 72 MHz	Fast bus; newer families also place ADCs and GPIO here
APB1 (PCLK1, low speed)	TIM2–TIM7, USART2/3/UART4/5, I2C1/2, SPI2/3, CAN, DAC, PWR, IWDG, WWDG	F103: max 36 MHz	On F103 also the bus whose prescaler triggers the timer ×2 rule
APB2 (PCLK2, high speed)	TIM1/TIM8, USART1, SPI1, ADC1/2/3, EXTI/AFIO or SYSCFG, GPIO on F1	F103: max 72 MHz	Faster peripherals live here

> **ENGINEERING NOTE:** Why this matters to firmware. The same peripheral type can live on a different bus in a different family (for example GPIO is on APB2 in F1 but on AHB1 in F4). Always read the clock-enable macro that CubeMX generates (__HAL_RCC_GPIOA_CLK_ENABLE()) rather than assuming a bus.

2.2 The Toolchain Ecosystem

Professional STM32 development uses a set of cooperating tools, each with a strict responsibility:

- **STM32CubeMX 6.18.1** is a graphical configurator and initialization-code generator. It handles MCU or board selection, pinout and conflict resolution, the clock tree, peripheral parameters, NVIC and DMA routing and middleware (FreeRTOS, USB, FatFS, LwIP). It stores its state in the human-readable **.ioc** file.
- **STM32CubeIDE 2.2.0** is the Eclipse/CDT-based IDE that embeds the CubeMX configurator and adds the C/C++ editor, build manager and SWD/JTAG debugger front end (ST-LINK, J-Link) with register (SFR) views, live expressions, SWV/ITM trace and a fault analyzer.
- **GNU Arm toolchain** (arm-none-eabi-gcc, as, ld, objcopy, size) turns C and assembly into .elf, .bin and .hex images placed according to the linker script.
- **STM32CubeProgrammer** (and the ST-LINK GDB server inside CubeIDE) programs Flash and talks to the debug port.
- **Firmware packages** (STM32Cube_FW_xx) supply CMSIS headers and the HAL/LL drivers for each family.

Table 2.2 — Tool and artifact responsibilities versus misconceptions

Tool / artifact	Actual responsibility	Misconception (what it does *not* do)
CubeMX	Pinout, alternate functions, clock tree, peripheral init, middleware, code generation	Does not write your application state machine, frame packets or handle faults
CubeIDE	Editing, GCC build, link, SWD flashing, debugging	A warning-free build proves nothing about wiring, clocks or timing
.ioc file	Text blueprint of MCU, pins, clocks, settings	It is not firmware and is never flashed; commit it to Git with the sources
CMSIS-Core / device header	ARM-standard core registers (SCB, NVIC, SysTick) and device register structs	CMSIS is the register foundation, not the ST HAL
HAL / LL drivers	ST C driver abstractions for a specific family	APIs and capabilities are not identical across F1, F4, G4, H7
Startup file (startup_*.s)	Reset handler: set stack pointer, copy .data, zero .bss, call SystemInit() then main()	If the vector table or SystemInit() faults, execution never reaches main()
Linker script (*_FLASH.ld)	Flash/SRAM addresses, sizes and section placement	Linking for 512 KB but flashing a 128 KB part causes faults
SWD probe	Halts the CPU, programs Flash, reads/writes memory and peripherals	A breakpoint freezes the CPU while sensors and timers keep moving

2.3 Installation and Setup

The following sequence gives a reproducible environment. Menu names are those of the 1.x/2.x CubeIDE generation; confirm against the release notes of your exact version (Appendix B).

1. **Prepare the PC.** Use a path without spaces or non-ASCII characters for the workspace (for example C:\stm32_ws). Keep the path short; very long generated paths can break the build on Windows.
2. **Install STM32CubeIDE 2.2.0.** The installer bundles the GNU Arm toolchain, the ST-LINK GDB server and CubeMX integration. Accept the ST-LINK driver installation.
3. **Install STM32CubeMX 6.18.1 (optional standalone).** CubeIDE already contains the configurator; a standalone copy is useful for reviewing .ioc files without opening an IDE workspace.
4. **Download the family firmware package.** In CubeMX: *Help → Manage embedded software packages*, then install the package for your family (for example STM32CubeF1). CubeIDE offers to do this the first time you create a project.
5. **Update ST-LINK firmware** (CubeProgrammer or the ST-LINK upgrade tool) if the debugger reports a version mismatch.

6. **Verify the whole chain** with a blink project (Lab 1 in Chapter 8): create → generate → build → flash → LED toggles after power-cycling *without* the debugger attached.

> **PRACTICAL BENCH TIP:** Keep a known-good project. Save a working "blink + UART hello" project before experimenting with clocks or multiple peripherals. When something stops working, diff against it.

2.4 Creating a Project

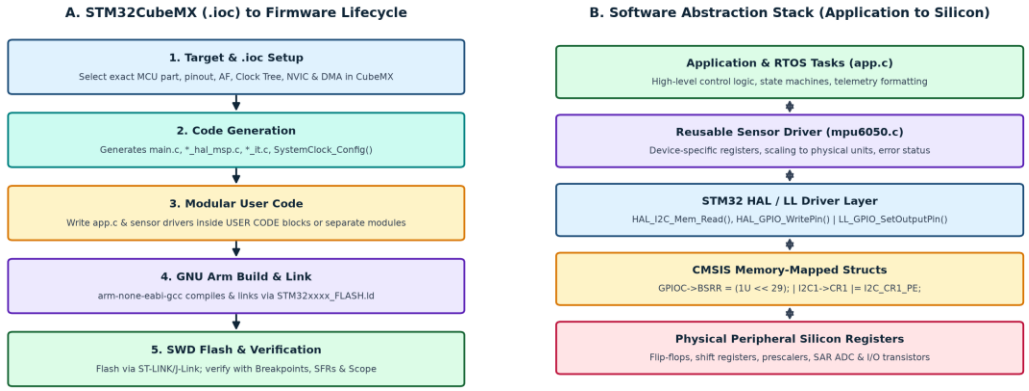


Figure 2.1 — CubeMX project lifecycle and software abstraction stack

Figure 2.1 — (A) .ioc-to-firmware lifecycle; (B) software abstraction stack from application to silicon.

Follow this disciplined sequence in CubeIDE 2.2.0 / CubeMX 6.18.1:

1. **Verify the silicon and schematic.** Read the chip marking (STM32F103C8T6 vs. STM32F401RCT6), identify the package (LQFP48/64, UFQFPN32) and check whether an external crystal (HSE) is fitted and at which frequency. Smaller packages do not bond out every pad.
2. **Select the target.** File → New → STM32 Project; choose the exact commercial part number (or the board, which preloads LEDs, buttons and clocks).
3. **Set SYS first.** System Core → SYS → Debug = Serial Wire. Leaving *No Debug* disables PA13/PA14 (SWDIO/SWCLK) after reset and can lock you out of the chip.
4. **Choose the HAL timebase.** With bare-metal code SysTick is fine. If you will use FreeRTOS, change *Timebase Source* to a spare timer (TIM6/TIM7 or a basic timer) so the RTOS owns SysTick while HAL_Delay() and HAL timeouts keep working.
5. **Configure RCC.** Enable HSE as *Crystal/Ceramic Resonator* (or *BYPASS Clock Source* for an external oscillator) if the board has one.
6. **Assign pins and labels.** Give pins meaningful User Labels (STATUS_LED, IMU_CS, FAULT_IN) so generated macros read like intent.
7. **Enable peripherals one at a time.** Set parameters, NVIC interrupts/priorities and DMA requests, and resolve conflicts deliberately.
8. **Set Project Manager options.** Under *Code Generator* enable *Generate peripheral initialization as a pair of .c/.h files per peripheral* and *Keep User Code when re-generating*; set a toolchain/IDE of *STM32CubeIDE*.

9. **Generate, inspect and commit.** Save (Ctrl+S) to generate. Review the Git diff: regeneration may change more than the setting you touched.

2.5 The .ioc File and the CubeMX Workspace

The .ioc file records everything CubeMX needs to regenerate the initialization code: target MCU/board, pin assignments, clock settings, enabled peripheral instances and parameters, interrupt and DMA configuration, middleware and code-generation options. **It is the configuration source, not the firmware.**

Table 2.3 — CubeMX tabs and what they control

Tab / section	Purpose	Typical decisions
Pinout & Configuration → Categories	Enable peripherals and set parameters	System Core (GPIO, DMA, NVIC, RCC, SYS, IWDG/WWDG), Analog (ADC, DAC), Timers, Connectivity (UART, SPI, I2C, CAN, USB), Middleware (FreeRTOS, FatFS)
Pinout view	Assign pins, alternate functions, labels	Resolve conflicts; confirm the pin exists on your package
Clock Configuration	Source selection, PLL, prescalers, bus and peripheral clocks	HSE vs. HSI, PLL multiplier, HCLK/PCLK limits, timer clock box
Project Manager → Project	Name, location, toolchain, stack/heap size	CubeIDE toolchain; minimum heap/stack sizes
Project Manager → Code Generator	File layout, library copying, user-code retention	One .c/.h per peripheral; keep user code
Project Manager → Advanced Settings	Driver choice per peripheral (HAL or LL), init order	Select HAL by default; LL where needed

Typical workflow: create project → pins → clocks → peripherals → parameters → NVIC/DMA/middleware → save .ioc → generate → review warnings → add application logic in user-code regions or separate files → build, flash, test → commit .ioc and sources together.

Code-generation cautions

- Keep application logic outside generated regions that are not protected by USER CODE markers.
- Prefer separate application and driver files for anything non-trivial.
- After changing the .ioc, read the diff rather than trusting regeneration.
- Resolve pin conflicts intentionally instead of selecting random pins.
- If a generated function is missing, check that the peripheral is enabled, generation finished, and the right configuration is active.

2.6 GPIO Architecture, Alternate Functions and Electrical Behavior

Table 2.4 — GPIO operating modes

Mode	Internal configuration	Electrical behavior	Typical use
Input floating	Output driver off; Schmitt trigger on; pulls off	High impedance; floats if undriven	Externally driven digital lines
Input pull-up / pull-down	Output off; Schmitt on; ~40 kΩ pull on	Weak bias to VDD or GND	Push-buttons, fault lines
Output push-pull	PMOS and NMOS active	Actively drives 1 and 0	LEDs, SPI chip select, enable pins
Output open-drain	PMOS off, NMOS only	Pulls low or releases (needs pull-up)	Wired-AND lines, level shifting, bit-banged buses
Alternate function push-pull	Peripheral controls push-pull driver	Peripheral drives both edges	UART TX, SPI SCK/MOSI, PWM
Alternate function open-drain	Peripheral controls open-drain driver	Peripheral pulls low or releases	I^2C SDA/SCL, 1-Wire
Analog	Output off; Schmitt off; pulls off; analog switch on	No digital switching noise	ADC/DAC/comparator pins

The **output speed** setting is a slew-rate control, not a clock. HIGH/VERY_HIGH increases edge rates and therefore ringing, overshoot and EMI; use the slowest setting that meets your rise-time budget.

COMMON MISTAKE / CAUTION: Electrical danger: 5 V tolerance and current limits.

1. Never assume a pin is 5 V tolerant. Look for the **FT** marking in the datasheet pinout table. Even on FT pins, tolerance applies only to digital input mode with VDD powered. **Never** apply more than VDD to an analog (ADC) pin: injection current corrupts other channels or latches up the silicon.
2. A GPIO pin is rated for roughly 8–25 mA depending on family and mode, with strict total package current limits. Never drive motors, solenoids or relay coils directly. Use a MOSFET/BJT driver with a flyback diode.
3. Check board schematics: an on-board LED may be active-low (Blue Pill PC13 lights when the pin is **low**) or shared with another function.

2.7 Clock Tree, PLL and the APB Timer ×2 Rule

A peripheral can be configured perfectly in its own panel and still misbehave if the clock tree is wrong. Clock settings affect CPU speed, bus frequencies, UART baud accuracy, timer periods and PWM frequencies, ADC clock and conversion time, USB (which needs an accurate 48 MHz) and Flash wait states.

Table 2.5 — Clock sources and nodes

Name	Role	Notes
HSI	Internal high-speed RC	Always present; accuracy of a few percent unless trimmed

Name	Role	Notes
HSE	External crystal, resonator or bypass	ppm-level accuracy with the right load capacitors; verify the board really has one
LSI	Internal low-speed RC	Watchdog, sometimes RTC; very inaccurate
LSE	32.768 kHz crystal	RTC timekeeping
PLL	Multiplies/divides HSI or HSE	Keep the VCO within the reference-manual range
SYSCLK → HCLK	System and AHB clock	Feeds core, DMA, memories
PCLK1 / PCLK2	APB1 / APB2 clocks	Peripheral clocks and timer-clock source
Kernel clocks	Per-peripheral clock mux (newer families)	A UART may run from a different PLL output than the CPU

Worked example — STM32F103C8T6 at 72 MHz. HSE = 8 MHz, PLL source HSE, PLLMUL = ×9 → SYSCLK = 72 MHz. AHB prescaler /1 → HCLK = 72 MHz. APB1 prescaler /2 → PCLK1 = 36 MHz (the maximum for APB1). APB2 prescaler /1 → PCLK2 = 72 MHz. ADC prescaler /6 → ADCCLK = 12 MHz (limit 14 MHz). Flash latency = 2 wait states for 48–72 MHz.

> **WARNING:** The APB timer ×2 trap. On classic F1/F2/F3/F4/F7/L4 trees the timer kernel clock is derived as follows: if the APB prescaler is 1, f_TIM = PCLK; if the prescaler is greater than 1, f_TIM = 2 × PCLK. With APB1 /2 on an F103 at 72 MHz, PCLK1 = 36 MHz but TIM2/3/4 are clocked at 72 MHz. Always read the "To APB1/APB2 timer clocks" box in CubeMX. Newer families (G0, G4, H7, U5) may use different timer-clock schemes, so do not apply F4 folklore blindly. A period that is off by exactly a factor of two is the signature of this rule; check the clock tree before changing values at random.

Clock-configuration workflow. Open the .ioc, go to Clock Configuration, identify the board's *real* clock source and frequency (do not assume a crystal exists because the MCU supports HSE), choose the system clock source, configure the PLL, check every bus and peripheral clock against its limit, check Flash latency, resolve CubeMX warnings, save/regenerate, read SystemClock_Config(), then verify with a known test (UART text, a toggling pin measured with a scope). Never copy clock settings from a tutorial written for a different board.

2.8 Code Generation, Project Structure and USER CODE Preservation

A typical generated HAL project contains:

```
Project/
├── Core/
│   ├── Inc/      main.h, stm32f1xx_it.h, stm32f1xx_hal_conf.h, <peripheral>.h
│   └── Src/      main.c, stm32f1xx_it.c, stm32f1xx_hal_msp.c, <peripheral>.c, syscalls.c
├── Drivers/
│   ├── CMSIS/                    core + device headers
│   └── STM32F1xx_HAL_Driver/     HAL/LL sources
├── Middlewares/                  only if enabled (FreeRTOS, USB, FatFS)
├── startup_stm32f103xb.s         vector table + reset handler
```

```
├── STM32F103C8TX_FLASH.ld        linker script
└── Project.ioc
```

Generated functions and their jobs

- HAL_Init() configures Flash prefetch/caches (family dependent), sets the NVIC priority grouping, starts the 1 ms time base (SysTick by default) and calls HAL_MspInit().
- SystemClock_Config() programs oscillators, PLL, prescalers and Flash latency from the Clock Configuration tab.
- MX_GPIO_Init(), MX_USARTx_UART_Init(), MX_ADCx_Init(), MX_TIMx_Init() and so on fill the peripheral handle structures and call HAL_xxx_Init().
- HAL_xxx_MspInit() (in *_hal_msp.c) does the board-level part: peripheral clock enable, GPIO alternate function setup, NVIC enable and DMA linking.
- stm32f1xx_it.c holds the interrupt handlers, each of which calls the matching HAL_xxx_IRQHandler().

Listing 2.1 — Generated clock setup for F103 at 72 MHz (HSE 8 MHz, PLL ×9)

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  RCC_OscInitStruct.OscillatorType      = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState            = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue      = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState            = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState        = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource       = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL          = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) { Error_Handler(); }

  RCC_ClkInitStruct.ClockType      = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK |
                                     RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource   = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider  = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;   /* PCLK1 = 36 MHz */
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;   /* PCLK2 = 72 MHz */
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK) { Error_Handler(); }
}
```

Listing 2.2 — main.c skeleton showing initialization order and USER CODE regions

```
/* USER CODE BEGIN Includes */
#include "app.h"
#include "mpu6050.h"
/* USER CODE END Includes */

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_DMA_Init();            /* MUST come before peripherals that use DMA */
  MX_USART2_UART_Init();
  MX_I2C1_Init();
  MX_ADC1_Init();
  MX_TIM3_Init();

  /* USER CODE BEGIN 2 */
```

```
  App_Init();
  /* USER CODE END 2 */

  while (1)
  {
    /* USER CODE BEGIN WHILE */
    App_Update();
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    /* USER CODE END 3 */
  }
}
```

COMMON MISTAKE / CAUTION: Common mistake: editing outside USER CODE. Anything outside a USER CODE BEGIN … END pair is regenerated and erased. CubeMX offers protected regions for includes, private types/defines/macros/variables, function prototypes, main() stages (0–4, WHILE), the clock/MSP functions and each interrupt file. Put your real logic in app.c, sensor.c and similar files and call them from a short USER CODE block. Also place MX_DMA_Init() before DMA-using peripherals; if it is after, the DMA handle link breaks silently (this ordering is a CubeMX-generated default you should not reorder by hand).

2.9 HAL, LL and Direct Register Programming

2.9.1 What each layer is

- **Register-level (CMSIS) programming** reads and writes the memory-mapped peripheral registers through the structures in the device header (GPIOC->BSRR, USART2->DR). It is the truth: every other layer ends here.
- **LL (Low-Layer)** is a set of thin static inline helpers named after the register fields (LL_GPIO_ResetOutputPin()), with no state tracking, timeouts or locks.
- **HAL (Hardware Abstraction Layer)** adds handle structures (UART_HandleTypeDef), a state machine (HAL_UART_STATE_*), locking, timeouts, status return codes and callbacks. It is what CubeMX targets by default.

2.9.2 One operation, three layers

The same operation, driving PC13 low, written at each level (F1; PC13 on a Blue Pill LED is active-low):

Listing 2.3 — Setting PC13 low with HAL, LL and registers

```
/* 1. HAL */
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);

/* 2. LL */
LL_GPIO_ResetOutputPin(GPIOC, LL_GPIO_PIN_13);

/* 3. Register: BSRR upper 16 bits reset the pin, lower 16 bits set it */
GPIOC->BSRR = (uint32_t)GPIO_PIN_13 << 16U;
```

HAL_GPIO_WritePin() is essentially the third line plus an argument assertion. That is the general pattern: **HAL function → (checks, state, timeouts) → register writes → silicon.** For complex peripherals the HAL value is far larger, because it sequences start/stop/ACK handling (I^2C), interrupt and DMA bookkeeping, and error flags.

Listing 2.4 — Bare-register PC13 setup on F1 (for understanding only)

```
RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;          /* clock to GPIOC (APB2 on F1)          */
GPIOC->CRH &= ~(0xFUL << 20);               /* clear CNF13[1:0] and MODE13[1:0]     */
GPIOC->CRH |=  (0x2UL << 20);               /* MODE13 = 10 (2 MHz), CNF13 = 00 PP   */
GPIOC->BSRR = (uint32_t)GPIO_PIN_13 << 16U; /* drive low (LED on)                   */
```

This assumes F1 register layout (CRL/CRH). On F4 the same configuration uses RCC->AHB1ENR and GPIOC->MODER. This is exactly why register code is non-portable.

2.9.3 Comparison and selection

Table 2.6 — HAL vs. LL vs. direct registers

Dimension	HAL	LL	Direct registers
Abstraction	High: handles, state machines, callbacks, timeouts	Medium-low: inline helpers with ST field names	Lowest: raw memory-mapped structs
Development speed	Fastest; CubeMX generates init	Moderate; you manage state	Slowest; read the reference manual
Portability	High across families (minor changes)	Moderate; tied to the peripheral IP version	Low; names and semantics change (SR vs. ISR, DR vs. TDR/RDR)
Code size / overhead	Larger; tens of cycles per call is typical	Near zero	Minimal
Safety	Timeouts (HAL_TIMEOUT), state (HAL_BUSY)	No automatic timeouts or locks	High risk: reserved bits, write-1-to-clear traps, read-modify-write races
Best use	Most application firmware; I^2C, USB, CAN, DMA state machines	Fast ISRs (for example a 50 kHz control loop)	Deep study, cycle-critical loops, features HAL does not expose

Recommendation. Start with CubeMX and HAL. Read the HAL source of the functions you call. Learn the registers from the reference manual. Measure. Optimize only after you have identified a real bottleneck. Do not teach that register code is automatically faster or that HAL is automatically slow: choose the lowest level that solves the problem safely and maintainably.

2.10 Debugging Workflow in CubeIDE

- **Breakpoints and stepping** on the ST-LINK GDB server; **Live Expressions** to watch variables while the program runs; the **SFR view** to read and write peripheral registers (invaluable for confirming what HAL really configured).
- **SWV/ITM** can stream printf output and data traces over the SWO pin without using a UART, where the board and probe wire SWO.
- **Halting freezes only the CPU.** Timers, ADC and external sensors keep running, so after a breakpoint the data stream is stale. Many parts provide DBGMCU freeze bits to stop timers and watchdogs while halted; a watchdog that keeps running will reset the MCU under the debugger.
- Print the **reset cause** and the HAL return codes of every call you care about; do not rely on silence.
- When execution lands in HardFault_Handler, use the analysis method in Section 8.2.

ENGINEERING NOTE: Key takeaway (Part 1). CubeMX configures hardware and generates initialization; CubeIDE edits, builds, flashes and debugs; the .ioc is the configuration source and belongs in version control; regeneration only preserves code inside USER CODE regions; and clocks are the first thing to verify when any peripheral misbehaves.

3. Part 2 — Interrupts, DMA and Timers

This part answers three connected questions: *how does the CPU learn that something happened* (polling, interrupts), *how does data get into memory without the CPU copying it* (DMA), and *how does the hardware keep time and generate or measure signals* (timers). Section 3.7 combines all three into the backbone of real-time data acquisition.

3.1 Polling

Definition. In polling the CPU repeatedly reads a peripheral status flag (or calls a function that does so) until the event it is waiting for has occurred.

How it works on STM32. A HAL polling call such as HAL_UART_Receive() or HAL_ADC_PollForConversion() loops on a status bit (RXNE, EOC) inside the HAL, compares HAL_GetTick() against a timeout and returns HAL_OK, HAL_TIMEOUT or HAL_ERROR. While it waits, nothing else in that thread runs.

Listing 3.1 — Polled ADC read with timeout

```
uint32_t raw = 0;
if (HAL_ADC_Start(&hadc1) == HAL_OK)
{
    if (HAL_ADC_PollForConversion(&hadc1, 10U) == HAL_OK)
    {
        raw = HAL_ADC_GetValue(&hadc1);   /* 0..4095 for 12-bit */
    }
    (void)HAL_ADC_Stop(&hadc1);
}
```

Strengths: trivial to write and reason about, no interrupt or DMA configuration, deterministic when the awaited event is fast and bounded. *Weaknesses:* burns CPU cycles, blocks other work, timing depends on how much other work the loop does, and a flag that never sets locks the system unless a timeout exists.

Use polling for: initialization checks (WHO_AM_I), infrequent sensor reads (1 Hz temperature), short bounded operations, labs and prototypes. **Avoid it for:** continuous high-rate streams or anything sharing a thread with a hard-deadline loop.

3.2 Interrupts

3.2.1 Definition and purpose

An **interrupt** is a hardware-signaled request that makes the CPU suspend the current code, run a short **interrupt service routine (ISR)**, and then resume exactly where it left off. It exists so that the CPU can do useful work (or sleep) and still react within microseconds to events: a button edge, a timer update, a received byte, a conversion or DMA completion, a fault.

3.2.2 Architecture on Cortex-M

- **Exceptions and IRQs.** The core defines system exceptions (Reset, NMI, HardFault, SVCall, PendSV, SysTick) and the device adds **external interrupts (IRQn)** from peripherals, managed by the **NVIC** (Nested Vectored Interrupt Controller).
- **Vector table.** A table in Flash (symbol g_pfnVectors in startup_*.s) holds the initial stack pointer followed by the address of every handler. Entry for IRQ number *n* is at word index *16 + n*.

- **NVIC registers.** Per interrupt: enable (ISER/ICER), pending (ISPR/ICPR), active status and an 8-bit priority field of which the top 4 bits are implemented on most STM32 devices (16 levels).
- **Peripheral side.** Each peripheral has an interrupt-enable bit (for example TIM_DIER.UIE, USART_CR1.RXNEIE) and a status flag. The line to the NVIC is asserted only when both the enable and the flag are set; EXTI lines additionally filter GPIO edges.

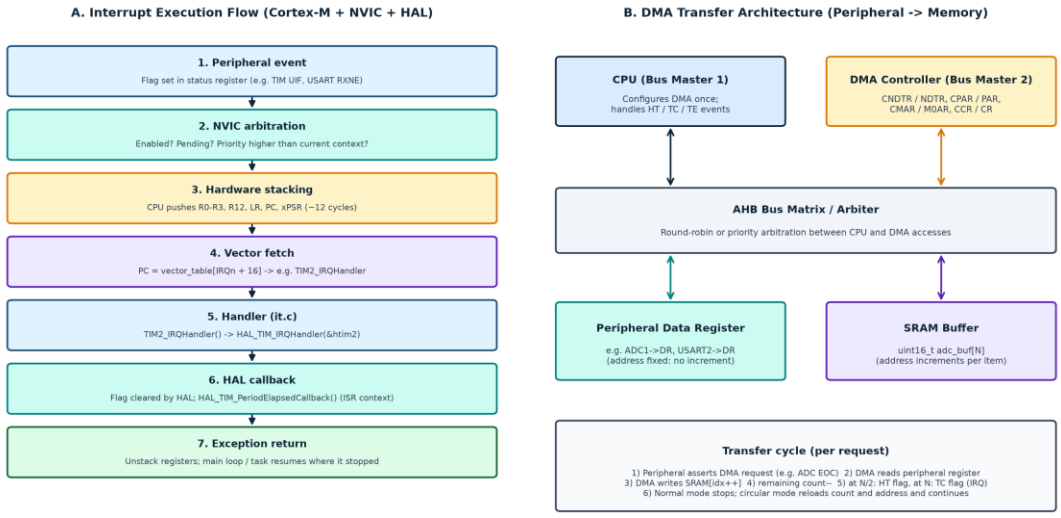


Figure 3.1 — Interrupt execution flow and DMA transfer architecture

Figure 3.1 — (A) Interrupt execution flow from peripheral event to exception return; (B) DMA controller as a second bus master moving data between a peripheral register and SRAM.

Entry sequence. When an enabled, pending interrupt has a higher priority than the running context, the core automatically pushes R0–R3, R12, LR, PC and xPSR (8 words) onto the active stack, fetches the vector, and starts the handler, about 12 cycles on Cortex-M3/M4 with zero-wait-state memory. If another interrupt is already pending at exit, **tail-chaining** skips the unstack/restack (about 6 cycles). Latency becomes much worse if Flash wait states, a higher-priority ISR or a long critical section intervene.

3.2.3 From IRQ to callback in HAL

CubeMX generates a one-line handler in stm32f1xx_it.c, and the HAL does the rest:

```
void TIM3_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&htim3);         /* reads flags, clears them, dispatches */
}
```

HAL_TIM_IRQHandler() checks which flag fired, clears it and calls a **callback** such as HAL_TIM_PeriodElapsedCallback(). HAL callbacks are declared __weak; you provide your own definition. **Callbacks execute in interrupt context**, so all ISR rules apply to them. Forgetting to call the HAL handler from the IRQ function, or filtering the wrong htim->Instance inside the callback, are classic reasons a callback never runs.

3.2.4 Priorities, preemption and nesting

Cortex-M priorities are **inverted**: a *lower number is more urgent* (0 highest; 15 lowest with 4 implemented bits). The priority byte is split by the **priority grouping** into *preemption priority* and *sub-priority*. HAL sets NVIC_PRIORITYGROUP_4 (4 bits preemption, 0 bits sub), which is also what FreeRTOS requires.

- A pending interrupt **preempts** the running ISR only if its *preemption* priority is numerically lower.
- Equal preemption priority never nests; the sub-priority only orders interrupts that are pending simultaneously.
- Configure with HAL_NVIC_SetPriority(TIM3_IRQn, 5, 0); HAL_NVIC_EnableIRQ(TIM3_IRQn); or in the CubeMX NVIC tab.
- Reserve the highest urgencies for the shortest, most time-critical handlers (motor fault, encoder), put communication in the middle and let the HAL tick and RTOS-aware interrupts sit lower.

Listing 3.2 — Debounced button using EXTI

```
static volatile uint32_t g_last_press_ms;
static volatile uint8_t  g_button_event;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == BTN_Pin)                      /* label set in CubeMX */
    {
        uint32_t now = HAL_GetTick();
        if ((now - g_last_press_ms) > 30U)       /* 30 ms debounce window */
        {
            g_last_press_ms = now;
            g_button_event  = 1U;                /* main loop consumes this */
        }
    }
}
```

Newer HAL versions (for example STM32U5) split this callback into rising/falling variants; **Verify** the callback names in your package.

3.2.5 Strict ISR rules

COMMON MISTAKE / CAUTION: ISR rules.

1. Never block: no HAL_Delay(), printf(), malloc() or a HAL_I2C_Master_Transmit() with a long timeout. (HAL_Delay() in an ISR can deadlock if SysTick has lower priority than that ISR.)
2. Keep it to microseconds: set a flag, copy one byte into a ring buffer, or post to an RTOS queue/notification using the ...FromISR() variant.
3. Make sure the interrupt source is cleared. A flag left set re-enters the ISR forever.
4. Share data deliberately. volatile forces the compiler to re-read a variable but does **not** make counter++ atomic, protect structures, order accesses against DMA or provide cache coherency on Cortex-M7. For a flag between an ISR and main on Cortex-M3/M4 a volatile uint8_t load/store is sufficient; for 64-bit values or structs, disable interrupts briefly or use a lock-free index scheme.

Latency budget. Worst-case latency = longest higher-or-equal-priority ISR + longest interrupt-disabled section + Flash wait-state effects + entry overhead. A 200 µs ISR elsewhere will break a 10 µs SPI chip-select timing requirement. Measure with a scope by toggling a pin on entry and exit.

3.3 DMA — Direct Memory Access

3.3.1 Definition and purpose

A **DMA controller** is an autonomous second bus master that copies data between **endpoints** (peripheral register ↔ memory, or memory ↔ memory) according to a configuration you set once. It exists to remove the per-item cost of interrupts and polling from the CPU: at high data rates the *copying* would otherwise consume most of the CPU.

DMA moves bytes, half-words or words. It does **not** interpret data, filter noise, parse protocols or know what a sample means; that remains application work.

3.3.2 What actually happens during a transfer

1. **Setup (CPU, once).** The CPU programs source address, destination address, number of data items, item width, increment behavior, direction, mode (normal or circular) and priority, then enables the stream/channel.
2. **Peripheral request.** The peripheral raises a DMA request when it has data or needs data, for example ADC end-of-conversion (EOC), USART RXNE or TXE, SPI RXNE/TXE, timer update. (Memory-to-memory transfers have no peripheral request and run as fast as the bus allows.)
3. **Bus transaction (DMA).** The controller arbitrates for the bus matrix, reads one item from the source and writes it to the destination. Peripheral addresses are normally **fixed** (the data register); memory addresses **increment**.
4. **Bookkeeping.** The remaining-item counter decrements. At the halfway point the **half-transfer (HT)** flag sets; at zero the **transfer-complete (TC)** flag sets; a bus or configuration error sets the **transfer-error (TE)** flag. Each can raise an interrupt.
5. **Mode behavior.** In *normal mode* the channel disables itself at TC. In *circular mode* the counter and addresses reload and it continues forever.
6. **Notification (CPU).** HAL routes the DMA interrupt to a callback (HAL_ADC_ConvHalfCpltCallback, HAL_UART_TxCpltCallback, …) so that software can process or release the buffer.

The CPU therefore still **configures** DMA, **owns the buffer lifetime**, **handles completion and errors**, and **processes** the data. What it no longer does is touch each item.

3.3.3 Controller structure and configuration parameters

Table 3.1 — DMA architecture by family

Family	Structure	Request routing	Practical consequence
F1	Channels (DMA1: 7, DMA2: 5 on larger parts)	Fixed: each peripheral request is wired to specific channels	Check the reference-manual request table; two peripherals sharing a channel cannot run DMA simultaneously
F4 / F7	Streams with channel selection, FIFO and burst options	Stream + channel number pairs	Easy to mis-click in CubeMX; the wrong stream means "callback never fires"

Family	Structure	Request routing	Practical consequence
G0 / G4 / L4 / H7 / U5	Channels or streams plus **DMAMUX**	Any request routed to any channel	Flexible; still must be enabled and mapped

Table 3.2 — Per-channel configuration fields (CubeMX DMA settings)

Field	Meaning	Typical ADC → RAM	Typical UART TX (RAM → peripheral)
Request / channel / stream	Which peripheral event triggers items	ADC1	USARTx_TX
Direction	Peripheral→memory, memory→peripheral, memory→memory	Peripheral to memory	Memory to peripheral
Peripheral increment	Advance peripheral address each item	**Disabled**	**Disabled**
Memory increment	Advance memory address each item	**Enabled**	**Enabled**
Data width (periph / mem)	Byte, half-word, word	Half-word / half-word (12-bit in uint16_t[])	Byte / byte
Mode	Normal or circular	Circular for continuous acquisition	Normal
Priority	Arbitration among DMA channels	Medium/high	Low/medium
FIFO / burst (F4/F7/H7)	Packing and burst bus use	Usually direct mode	Usually direct mode

Reference F103 mappings (typical, **Verify** in RM0008 for your exact part): ADC1 → DMA1 channel 1; SPI1 RX/TX → DMA1 channel 2/3; USART1 TX/RX → DMA1 channel 4/5; USART2 RX/TX → DMA1 channel 6/7; I2C1 TX/RX → DMA1 channel 6/7. Because USART2 and I2C1 share channels 6 and 7, you cannot DMA both simultaneously on that device.

3.3.4 Modes and buffer ownership

- **Normal mode:** transfers a finite block, then stops and signals TC. Ownership of the buffer returns to the CPU at TC.
- **Circular mode:** wraps endlessly. **Half-transfer** signals that the first half is filled while DMA fills the second; **transfer-complete** signals the second half is filled while DMA wraps to the first. The CPU must finish a half before DMA returns to it. A callback does not freeze the buffer.
- **Memory-to-memory:** CPU-free block copies; on F1 it uses any channel with the MEM2MEM bit and cannot be circular.

Listing 3.3 — Memory-to-memory DMA copy (names depend on the channel CubeMX generated)

```
static uint32_t src_block[256];
static uint32_t dst_block[256];
static volatile uint8_t g_copy_done;

void CopyWithDma(void)
{
    g_copy_done = 0U;
    /* The handle hdma_memtomem_dma1_channel1 is created when you add a
       MemToMem stream in CubeMX (DMA Settings -> MemToMem). */
    HAL_DMA_RegisterCallback(&hdma_memtomem_dma1_channel1,
                             HAL_DMA_XFER_CPLT_CB_ID, CopyDoneCb);
    if (HAL_DMA_Start_IT(&hdma_memtomem_dma1_channel1,
                         (uint32_t)src_block, (uint32_t)dst_block,
                         256U) != HAL_OK)         /* count = number of 32-bit items */
    {
        Error_Handler();
    }
}

void CopyDoneCb(DMA_HandleTypeDef *hdma) { (void)hdma; g_copy_done = 1U; }
```

3.3.5 DMA with the common peripherals

ADC (peripheral → memory, circular). The standard high-rate acquisition path; see Section 3.7.

UART transmit (memory → peripheral, normal). Non-blocking logging; the buffer must stay valid until the callback.

```
static char tx_buf[64];                          /* static: must outlive the call */
int n = snprintf(tx_buf, sizeof(tx_buf), "T=%d.%02d\r\n", t_c, t_frac);
if (HAL_UART_Transmit_DMA(&huart2, (uint8_t *)tx_buf, (uint16_t)n) != HAL_OK) { /* BUSY: drop or queue
*/ }
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) { if (huart->Instance == USART2) { g_tx_busy =
0U; } }
```

UART receive with idle-line detection (variable-length packets with no per-byte interrupt):

```
static uint8_t rx_dma_buf[128];
static volatile uint16_t g_rx_len;
static volatile uint8_t  g_rx_ready;

void Uart_StartRx(void)
{
    HAL_UARTEx_ReceiveToIdle_DMA(&huart2, rx_dma_buf, sizeof(rx_dma_buf));
    __HAL_DMA_DISABLE_IT(huart2.hdmarx, DMA_IT_HT);   /* ignore half-transfer events */
}

void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    if (huart->Instance == USART2) { g_rx_len = Size; g_rx_ready = 1U; }
    /* Main loop copies rx_dma_buf[0..Size-1], then re-arms with Uart_StartRx(). */
}
```

The API exists in recent HAL packages; **Verify** availability and re-arm rules in your family's HAL release notes.

SPI (full duplex). Useful for displays and flash pages; chip select must stay low for the whole transfer and be released in the completion callback.

```
HAL_GPIO_WritePin(IMU_CS_GPIO_Port, IMU_CS_Pin, GPIO_PIN_RESET);
HAL_SPI_TransmitReceive_DMA(&hspi1, tx_cmd, rx_data, len);
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
{
    if (hspi->Instance == SPI1) { HAL_GPIO_WritePin(IMU_CS_GPIO_Port, IMU_CS_Pin, GPIO_PIN_SET);
g_spi_done = 1U; }
}
```

I²C. Register-block reads can use HAL_I2C_Mem_Read_DMA() with HAL_I2C_MemRxCpltCallback() so that a 14-byte IMU burst does not block. I²C DMA has family-specific errata and ordering rules; **Verify** in the errata sheet for your part and fall back to interrupt mode if needed.

3.3.6 Common DMA mistakes

- Wrong request/channel/stream, or MX_DMA_Init() ordered after the peripheral init.
- Mismatched data widths (32-bit DMA word into a 16-bit array corrupts neighbors).
- Memory increment disabled when it is needed (every item overwrites one location), or peripheral increment enabled when it should not be.
- Passing a **stack** array to an asynchronous transfer that outlives the function, or modifying a TX buffer while DMA reads it.
- DMA target memory not accessible to the DMA (for example certain RAM regions on H7).
- Cortex-M7 data-cache coherency ignored (Section 5.8).
- Doing slow work in a DMA callback, or ignoring TE/overrun events.
- Circular DMA overwriting a half before the CPU has finished with it.

3.4 Polling vs. Interrupts vs. DMA

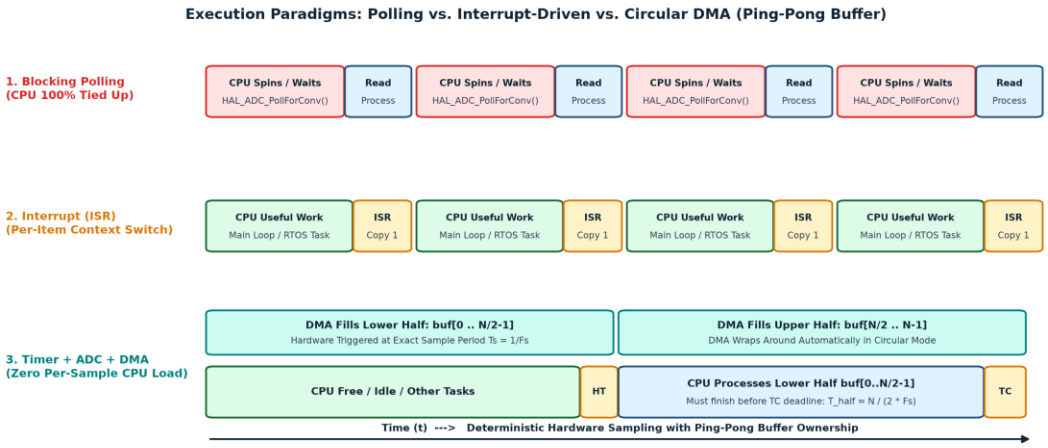


Figure 3.2 — Polling, interrupt-driven and circular DMA timelines

Figure 3.2 — Execution paradigms: (1) blocking polling; (2) interrupt per item; (3) timer + ADC + circular DMA with half-buffer ownership.

Table 3.3 — Trade-offs

Paradigm	CPU involvement	Latency / timing	Prerequisites	Best use	Primary failure
Polling	100 % while waiting	Whenever you next poll	None	Init checks, 1 Hz sensors, labs	Lockup if a flag never sets; jitter

Paradigm	CPU involvement	Latency / timing	Prerequisites	Best use	Primary failure
Interrupt	One ISR (tens to hundreds of cycles) per event	Microseconds, bounded by higher priorities	NVIC enable, handler, flag clearing	Buttons, UART commands, timer ticks, fault lines	ISR too fat; starvation; priority inversion
DMA	Setup plus one notification per block	Hardware-timed transfer; CPU notified at block edges	DMA request, buffer, width/increment settings	ADC streams, bulk UART/SPI, displays, audio	Wrong mapping; overwrite if the CPU is late; cache

Quantifying the CPU cost. If one interrupt per item costs *t_isr* (entry, HAL dispatch, your code, exit; typically 1–3 µs for HAL callbacks at 72 MHz) and items arrive at rate *R*, CPU load ≈ *R* × *t_isr*. UART at 115 200 baud delivers 11 520 bytes/s, so with t_isr = 2 µs the load is 2.3 %, which is fine. An ADC at 100 kS/s with a per-sample interrupt costs 100 000 × 2 µs = **20 %**, and 500 kS/s would be impossible. With DMA at a 256-sample half buffer the same 100 kS/s costs about 2 × 391 interrupts per second, roughly 0.1 %.

Table 3.4 — Starting point by requirement

Requirement	Likely approach
Read a sensor once every few seconds	Polling or simple interrupt
React to a button edge	EXTI interrupt with debounce
Periodic action	Timer interrupt, or timer-triggered peripheral event
Generate PWM	Timer hardware PWM (never bit-bang)
Stable-rate analog sampling	Timer-triggered ADC
Block of ADC samples, efficiently	Timer-triggered ADC + circular DMA
Continuous UART receive	RX interrupt with ring buffer, or DMA with idle-line detection
Tiny low-rate transfer	Polling may be simpler and sufficient
Command console	RX interrupt or DMA-to-idle plus a parser in main
4 KB SPI flash page / display frame	DMA if the CPU has better work to do

> **ENGINEERING NOTE:** Key concept. Polling, interrupts and DMA are not rival technologies. They answer different questions: how is the CPU told? and who moves the data? A real pipeline combines them: a timer schedules the sample, the ADC converts, DMA stores, and an interrupt tells the application a block is ready.

3.5 When Should DMA Be Used?

Use DMA when

- samples must be acquired continuously or at a fixed rate (ADC streams, audio, vibration),

- data rate × per-item interrupt cost would consume a significant share of the CPU,
- you transfer large blocks (display frames, flash pages, log buffers),
- you need predictable data movement while the CPU does other work.

DMA is unnecessary when

- reads are occasional (1 Hz sensor), small and not time critical,
- the transfer is only a few bytes (setup overhead exceeds the copy),
- the project is simple and has no demanding timing requirement,
- you cannot afford the extra complexity of buffer lifetime and ownership.

Performance considerations. DMA and the CPU compete for the bus matrix and for SRAM banks; place high-rate buffers so that CPU code and data fetches do not collide on the same slave where the device allows. Choose block size from sample rate, processing time, RAM and latency: larger blocks reduce interrupt rate and per-block overhead but raise latency and RAM; smaller blocks reduce latency but increase callback frequency. DMA introduces its own complexity: buffer management, completion and error handling, and cache coherency on Cortex-M7 parts.

3.6 Timers

3.6.1 Fundamentals

A **timer** is a hardware counter clocked from a prescaled clock, with compare, capture and trigger logic around it. It is not HAL_Delay(): once started it runs without CPU involvement and can toggle pins, request DMA, trigger an ADC or raise an interrupt at exact times.

Table 3.5 — Timer categories

Category	Typical capabilities	Notes
Basic (TIM6/TIM7)	Time base, update event, trigger output (TRGO)	Not present on every part. **STM32F103C8T6 has none** (it has TIM1–TIM4)
General purpose (TIM2–TIM5, …)	Time base, input capture, output compare, PWM, encoder interface, one-pulse	16-bit; TIM2 and TIM5 are 32-bit on F4
Advanced control (TIM1, TIM8)	Complementary PWM, dead-time insertion, break input, repetition counter	Motor-control oriented
Low-power timer	Timing in stop/low-power modes	Selected devices
SysTick	Core timer; HAL tick and RTOS tick	Not a TIM peripheral
IWDG / WWDG	Watchdogs	Safety timeouts, not general timing (Section 3.6.9)

The core registers

- **PSC (prescaler):** divides the timer kernel clock by PSC + 1; 16-bit.
- **CNT (counter):** the live count (up, down or center-aligned).
- **ARR (auto-reload):** the period; when CNT reaches ARR the counter wraps and an **update event (UEV)** occurs.
- **CCR1–CCR4 (capture/compare):** compared with CNT to create PWM edges or output-compare events, or loaded with CNT on an input edge.
- **Preload (ARPE, OCxPE):** ARR/CCR writes take effect at the next update event, which prevents glitches when changing frequency or duty on the fly.

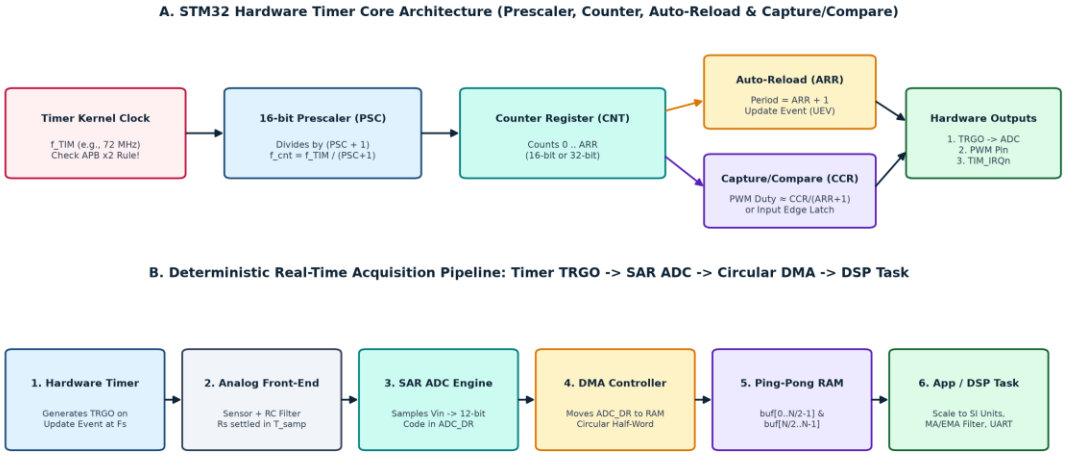


Figure 3.3 — Timer architecture and the timer-triggered ADC/DMA pipeline

Figure 3.3 — (A) Timer core: kernel clock → PSC → CNT with ARR and CCR; (B) deterministic acquisition pipeline: timer TRGO → ADC → circular DMA → processing task.

3.6.2 Timer mathematics

$$f_{\text{cnt}} = \frac{f_{\text{TIM}}}{\text{PSC} + 1}$$

$$f_{\text{update}} = \frac{f_{\text{TIM}}}{(\text{PSC} + 1)\,(\text{ARR} + 1)} \qquad T_{\text{update}} = \frac{(\text{PSC} + 1)(\text{ARR} + 1)}{f_{\text{TIM}}}$$

Variables. *f*TIM is the timer kernel clock in Hz (after the APB ×2 rule), *PSC* and *ARR* are the register values (dimensionless), *f*cnt is the counter tick rate and *f*update the update-event (period) rate. With a repetition counter (advanced timers) divide additionally by (RCR + 1). In **center-aligned** mode the counter runs 0 → ARR → 0, so the period is 2 × ARR counter ticks (not ARR + 1).

Worked examples (F103, f_TIM = 72 MHz)

Table 3.6 — Timer calculations

Goal	PSC	ARR	Check	Result

Goal	PSC	ARR	Check	Result
1 µs tick, 1 ms period (1 kHz)	71	999	72 MHz / 72 = 1 MHz; 1 MHz / 1000	1 kHz update
1 kHz tick from 10 kHz counter	7199	9	72 MHz / 7200 = 10 kHz; / 10	1 kHz update
1 Hz update	7199	9999	10 kHz / 10 000	1 Hz
50 Hz servo frame (20 ms) with 1 µs tick	71	19999	1 MHz / 20 000	50 Hz
20 kHz PWM, max resolution	0	3599	72 MHz / 3600	20 kHz, 3600 steps (~11.8 bits)

WARNING: 16-bit prescaler overflow trap. PSC is a 16-bit register (maximum 65 535). Trying to reach a 1 kHz counter from 72 MHz with PSC = 71 999 writes 71 999 & 0xFFFF = 6 463 and the timer runs more than 11× too fast. Split the division between PSC and ARR: for 1 Hz choose PSC = 7 199 and ARR = 9 999. The same limit applies to ARR on 16-bit timers (max 65 535).

PRACTICAL BENCH TIP: Spurious first interrupt. HAL's timer init generates an update event to latch the prescaler, which leaves the update flag set. Starting with HAL_TIM_Base_Start_IT() can therefore fire one interrupt immediately. Call __HAL_TIM_CLEAR_FLAG(&htim3, TIM_FLAG_UPDATE); before starting if the first early tick matters.

Listing 3.4 — Periodic 1 ms timer interrupt

```
/* CubeMX: TIM4 internal clock, PSC = 7199, ARR = 9, NVIC TIM4 global interrupt enabled */
static volatile uint32_t g_ms_ticks;

void App_StartTick(void)
{
    __HAL_TIM_CLEAR_FLAG(&htim4, TIM_FLAG_UPDATE);
    if (HAL_TIM_Base_Start_IT(&htim4) != HAL_OK) { Error_Handler(); }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM4) { g_ms_ticks++; }    /* keep it this short */
}
```

3.6.3 PWM generation

In edge-aligned PWM mode 1 (up-counting, active-high) the output is active while CNT < CCR and inactive while CNT ≥ CCR:

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1)(ARR + 1)} \qquad D = \frac{CCR}{ARR + 1} \qquad \text{resolution} = ARR + 1 \text{ steps}$$

CCR = 0 gives 0 % and *CCR ≥ ARR + 1* gives 100 %. Polarity (CCER) decides which level is "active". PWM requires the correct **alternate-function pin** for the timer channel, and HAL_TIM_PWM_Start() for that channel; starting only the time base is not enough.

Listing 3.5 — Servo (50 Hz, 1–2 ms pulse) and 20 kHz motor PWM

```
/* TIM4: PSC = 71 (1 us tick), ARR = 19999 (20 ms frame), channel 1 PWM mode 1 */
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, 1500);   /* 1.5 ms = centered servo */

/* TIM1: PSC = 0, ARR = 3599 -> 20 kHz; duty 25 % -> CCR = 0.25 * 3600 = 900 */
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);             /* TIM1 is advanced: also needs main output
enable, handled by HAL */
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 900);
```

PWM drives gate drivers, LEDs and servo inputs; it must never drive a motor directly from the pin. Advanced timers add complementary outputs and **dead time** so that the two transistors of a half-bridge are never on simultaneously, plus a break input for fault shutdown.

3.6.4 Input capture (measuring frequency and pulse width)

On a selected edge, hardware copies CNT into CCRx without CPU involvement, so edge timestamps are exact regardless of interrupt latency. Two consecutive captures give the period; capture on both edges (or the PWM-input configuration) gives pulse width.

Listing 3.6 — Frequency measurement with input capture (1 µs tick, 16-bit counter)

```
/* TIM2 CH1 input capture rising edge, PSC = 71, ARR = 0xFFFF */
static volatile uint32_t g_ic_prev, g_period_us;
static volatile uint8_t  g_ic_first = 1U;

void Capture_Start(void) { HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_1); }

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        uint32_t cur = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
        if (g_ic_first) { g_ic_first = 0U; }
        else            { g_period_us = (cur - g_ic_prev) & 0xFFFFU; }  /* wrap-safe */
        g_ic_prev = cur;
    }
}
/* f = 1e6 / g_period_us. Valid when at most one counter wrap occurs between edges
   (period < 65.536 ms, i.e. f > 15.3 Hz). Extend with an update-interrupt overflow counter if needed.
*/
```

3.6.5 Output compare, one-pulse and encoder modes

- **Output compare** performs an action (toggle, set, clear, interrupt) when CNT == CCRx: a precise scheduler for "do this 350 µs from now". Updating CCRx = CCRx + delta inside the compare interrupt produces a square wave of half-period delta ticks with no drift.
- **One-pulse mode** produces a single delayed pulse after a trigger (for example to start an ADC conversion at a quiet moment after a motor PWM edge).
- **Encoder mode** counts quadrature signals from two input channels in hardware; read the position with __HAL_TIM_GET_COUNTER().

Listing 3.7 — 1 kHz square wave from output compare (toggle), 1 µs tick

```
/* TIM3 CH1 output compare toggle, PSC = 71, ARR = 0xFFFF, NVIC enabled */
#define HALF_PERIOD_TICKS 500U

void OC_Start(void) { HAL_TIM_OC_Start_IT(&htim3, TIM_CHANNEL_1); }

void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim)
{
```

```
    if (htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        uint32_t next = __HAL_TIM_GET_COMPARE(htim, TIM_CHANNEL_1) + HALF_PERIOD_TICKS;
        __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_1, next & 0xFFFFU);
    }
}
```

3.6.6 Timer trigger output (TRGO), interrupts and DMA

- **TRGO.** The master-mode selection (MMS in CR2) chooses what appears on TRGO: reset, enable, **update**, compare pulse, compare channels. An ADC (or DAC) configured with an **external trigger** of that timer's TRGO starts one conversion per event, with no software jitter. The ADC's allowed trigger list is family- and instance-specific; a TRGO that is not on the list will silently never sample.
- **Timer interrupts.** Update, capture/compare, trigger and break events each have their own enable bit and callback.
- **Timer DMA requests.** An update or compare event can request DMA, for example to stream a table into CCRx and generate an arbitrary waveform or drive addressable LEDs with HAL_TIM_PWM_Start_DMA().

3.6.7 Timer pitfalls

- Assuming the timer clock equals SYSCLK or PCLK (Section 2.7).
- Off-by-one: registers divide by *value + 1*.
- Interrupt not enabled in NVIC, or the callback filters the wrong Instance.
- Timer started with the wrong function (Base_Start instead of PWM_Start).
- Pin not configured with the timer's alternate function.
- Timer already used as the HAL time base or by another subsystem (TIM1 for HAL tick, for instance).
- Calling blocking UART output from the timer callback.
- An LED that is active-low giving the impression of inverted duty.

3.6.8 Choosing a timer feature

Requirement	Feature
1 ms periodic interrupt	Basic timer or general timer with update interrupt
20 kHz PWM	Output-compare PWM; mind resolution = ARR + 1
Measure external frequency or pulse width	Input capture (PWM-input mode for duty)
Motor-control PWM with dead time	Advanced-control timer
Accurate timing without blocking	Timer interrupt, output compare, or timer-triggered peripherals
Quadrature encoder	Encoder mode
Fixed-rate ADC sampling	Timer TRGO → ADC

3.6.9 Watchdogs, RTC and SysTick compared with timers

- **IWDG** runs from its own low-speed clock and resets the MCU if not refreshed in time. **WWDG** additionally requires the refresh to fall *inside a window*; refreshing too early is as wrong as too late.
- **Best practice:** never refresh a watchdog unconditionally from a timer interrupt if the main application can deadlock. Refresh only when the checks that define "healthy" have passed, otherwise the watchdog is decoration.
- **RTC** provides calendar time or a low-power timebase (often from LSE). **General timers** give precise events, PWM and capture. **SysTick** supplies the HAL/RTOS tick and is not a substitute for hardware timer features.

3.7 Integrated Example: Timer-Triggered ADC with Circular DMA

This is the most important integrated example because it connects clocks, timers, ADC, DMA, interrupts, buffers and application processing.

Goal. Collect analog measurements at a predictable rate while spending almost no CPU per sample.

```
Timer update (TRGO)  ->  ADC conversion  ->  EOC  ->  DMA request  ->  SRAM buffer
                                                          |
                                                 half / full transfer IRQ
                                                          v
                                                application processes a block
```

3.7.1 Design decisions before touching CubeMX

1. Required sample rate *Fs* and the highest signal frequency of interest (Nyquist: *Fs* > 2 *f*max, plus an analog anti-alias filter).
2. Number of ADC channels (scan) and the sample time each needs.
3. Can one conversion (sample time + 12.5 cycles) finish before the next trigger?
4. Buffer size *N* versus latency and RAM, and whether the CPU can process half a buffer before DMA returns to it.
5. Cache maintenance needs (Cortex-M7).
6. How overruns and missed blocks will be detected and reported.

3.7.2 Configuration (STM32F103: ADC1 + TIM3 TRGO + DMA1 channel 1)

1. Set the analog pin to **Analog** mode; choose the ADC channel and rank.
2. ADC: 12-bit, right alignment, **Continuous Conversion Mode disabled**, **External Trigger = Timer 3 Trigger Out event** (on F103 the regular-group trigger list contains TIM3_TRGO; TIM2 TRGO is *not* a regular ADC1 trigger on this device, **Verify** in RM0008), **DMA Continuous Requests enabled**, sample time chosen from source impedance (Section 5.2).
3. DMA settings: ADC1 request, peripheral→memory, **circular**, peripheral increment off, memory increment on, half-word/half-word, medium or higher priority. Enable DMA1 channel1 interrupt in NVIC.
4. TIM3: internal clock, PSC and ARR from the sample rate, **Trigger Event Selection = Update Event**.
5. Generate code. In firmware: **calibrate the ADC** (HAL_ADCEx_Calibration_Start(); required on F1 and several other families), start HAL_ADC_Start_DMA(), then start the timer.

Sample-rate calculation. For *F*s = 10 kHz from TIM3 at 72 MHz: PSC = 71 (1 MHz), ARR = 99 → 1 MHz / 100 = 10 kHz. Check the conversion time: ADCCLK = 12 MHz, sample time 55.5 cycles → (55.5 + 12.5) / 12 MHz = **5.67 µs**, well below the 100 µs trigger period. For a multi-channel scan, distinguish the trigger rate, the per-channel sample rate (= trigger rate when one trigger starts the whole scan), the time to complete the whole scan and the DMA item rate.

Listing 3.8 — Timer-triggered ADC with circular DMA, ping-pong processing and overrun counting

```
#define ADC_BUF_LEN  200U                /* 200 samples: 100 per half */

static uint16_t          adc_dma_buffer[ADC_BUF_LEN];
static volatile uint8_t  g_half_ready, g_full_ready;
static volatile uint32_t g_adc_overruns;

void Acquisition_Start(void)
{
    if (HAL_ADCEx_Calibration_Start(&hadc1) != HAL_OK)              { Error_Handler(); }
    if (HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_dma_buffer, ADC_BUF_LEN) != HAL_OK)
                                                                      { Error_Handler(); }
    if (HAL_TIM_Base_Start(&htim3) != HAL_OK)                       { Error_Handler(); }
}

void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1) {
        if (g_half_ready) { g_adc_overruns++; }      /* previous half was never processed */
        g_half_ready = 1U;
    }
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1) {
        if (g_full_ready) { g_adc_overruns++; }
        g_full_ready = 1U;
    }
}

void Acquisition_Process(void)                    /* call from the main loop or an RTOS task */
{
    if (g_half_ready) { g_half_ready = 0U; DSP_ProcessBuffer(&adc_dma_buffer[0],
ADC_BUF_LEN / 2U); }
    if (g_full_ready) { g_full_ready = 0U; DSP_ProcessBuffer(&adc_dma_buffer[ADC_BUF_LEN / 2U],
ADC_BUF_LEN / 2U); }
}
```

Order matters. Start the ADC+DMA before the timer so that the first trigger finds an armed converter; some families' application notes specify the opposite for particular modes, **Verify** against your reference manual.

3.7.3 The half-buffer deadline

$$T_{\text{full}} = \frac{N}{F_s} \qquad T_{\text{half}} = \frac{N}{2\,F_s}$$

Variables. *N* total samples in the circular buffer (all channels counted if scanning), *F*s the item rate in samples per second, *T*half the time the CPU has to finish one half before DMA overwrites it.

Example. N = 200 at 10 kS/s → *T*full = 20 ms, *T*half = **10 ms**. If processing 100 samples takes 12 ms the CPU silently mixes old and new samples. Count an overrun whenever a half-ready flag is still set when the next half

arrives, as the listing does. Ramping the sample rate until g_adc_overruns increments tells you the real CPU budget.

3.7.4 Verification

Ground the pin (codes near 0, offset error remains) and feed 1.65 V from a divider (codes near mid-scale). Toggle a GPIO in the half callback and confirm the period equals $N/(2F_s)$ on a scope. Start at a low rate and increase it gradually while watching overrun counts, ADC overrun and DMA error flags. Never print every sample over a 115 200-baud UART from a 10 kS/s stream (20 kB/s needed against about 11.5 kB/s available).

> **ENGINEERING NOTE:** Key takeaway (Part 2). Interrupts tell the CPU when, DMA moves what, timers decide when exactly. Choose each by CPU budget and deadline, make buffer ownership explicit, count overruns, and measure the real timing.

4. Part 3 — Serial Communication Protocols: UART, SPI, and I²C

Embedded systems rarely operate in isolation. Microcontrollers must exchange telemetry, sensor data, actuator setpoints, and diagnostic messages with host computers, companion processors, displays, and peripheral ICs. Rather than running parallel 8-bit or 16-bit buses across circuit boards—which consume excessive PCB area, cause electromagnetic interference, and waste GPIO pins—modern microcontrollers rely on **serial communication protocols**.

This chapter provides a rigorous, hardware-level and HAL-level analysis of the three foundational serial protocols used in STM32 microcontrollers: **UART/USART**, **SPI**, and **I²C**. We analyze their physical layer physics, frame architectures, clock synchronization, driver implementations (Polling, Interrupt, and DMA), error management, and hardware recovery procedures, culminating in an objective protocol selection matrix.

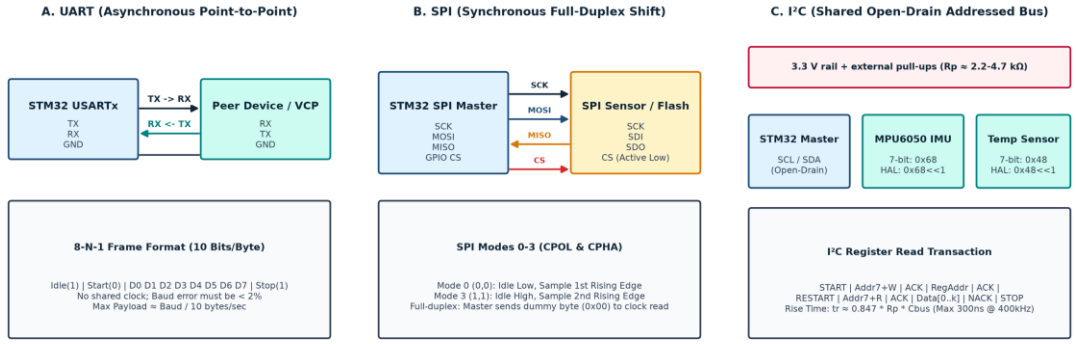


Figure 4.1 — Serial communication architectures: (A) Asynchronous UART with framing and start/stop bits; (B) Synchronous SPI four-wire full-duplex bus with CS gating; (C) Synchronous I²C open-drain two-wire bus with pull-up resistors, START/STOP conditions, and 9-bit clock packets.

4.1 UART / USART (Universal Asynchronous / Synchronous Receiver Transmitter)

4.1.1 Operating Principle and Physics

UART (Universal Asynchronous Receiver-Transmitter) is a point-to-point, full-duplex communication interface that transmits data sequentially bit-by-bit without sharing a dedicated clock signal. Because there is no common clock line between the transmitter (TX) and receiver (RX), both devices must agree beforehand on four core parameters: 1. **Baud Rate**: The transmission speed in symbols (bits) per second (e.g., 9600, 115200, 921600 bps). 2. **Word Length**: The number of payload data bits (typically 8 bits, occasionally 7 or 9 bits). 3. **Parity**: Error-detection bit appended to the data (None, Even, or Odd). 4. **Stop Bits**: The duration of the line idle state concluding the frame (typically 1 or 2 bits).

The standard idle state of a UART transmission line is logic **HIGH** (V_{DD}, typically 3.3 V on STM32).

The UART Frame Structure

Every transmitted byte is wrapped inside a deterministic packet envelope: 1. **Start Bit (Logic LOW)**: The transmitter drives the line from HIGH to LOW for exactly 1 bit period. This high-to-low transition alerts the

receiver's internal shift register that a byte is incoming and triggers its baud rate synchronization timer. 2. **Data Bits (LSB First)**: The payload bits (usually 8 bits, D_0 through D_7) are clocked out sequentially at the nominal bit duration $T_{\text{bit}} = \frac{1}{\text{Baud}}$. 3. **Parity Bit (Optional)**: If enabled, an extra bit is inserted to make the total count of '1's even (Even Parity) or odd (Odd Parity). 4. **Stop Bit (Logic HIGH)**: The transmitter drives the line back to logic HIGH for 1, 1.5, or 2 bit periods. This guarantees a high state before the next frame's start bit can generate a falling edge.



USART vs. UART

A standard STM32 **UART** supports asynchronous serial communication. A **USART** (Universal Synchronous/Asynchronous Receiver Transmitter) includes additional hardware logic to generate and accept a synchronous master/slave clock signal (CK pin), support Smartcard interfaces (ISO 7816), IrDA SIR ENDEC, LIN (Local Interconnect Network), and Modbus protocols. When synchronous mode is not enabled in CubeMX, USART peripherals operate identically to UARTs.

4.1.2 Baud Rate Generation and Clock Tolerances

Inside the STM32 UART peripheral, incoming and outgoing bits are sampled using an oversampling clock generated from the peripheral bus clock (f_{PCLK1} or f_{PCLK2}) divided by the **Baud Rate Register (USART_BRR)**.

Oversampling Architecture (16× vs. 8×)

Most STM32 UART peripherals default to **16× oversampling** (USART_CR1_OVER8 = 0). In this mode: - The receiver divides each bit period into 16 discrete time slices. - When a falling edge (Start bit) is detected, the receiver verifies the line state at slices 3, 5, and 7 to reject high-frequency noise spikes. - Subsequent data bits are sampled at slices **8, 9, and 10**. A majority vote (at least 2 out of 3 samples identical) determines the registered bit value. - On higher-performance families (STM32F4/F7/G4/H7), **8× oversampling** can be selected to achieve double the maximum baud rate, at the expense of reduced tolerance to clock jitter.

Baud Rate Calculation (STM32F1 / Classic USART)

The baud rate formula in 16× oversampling mode is:

$$\text{Baud} = \frac{f_{\text{PCLK}}}{16 \times \text{USART_BRR}}$$

Where USART_BRR is an unsigned fixed-point number: - Bits [15:4] contain the 12-bit integer **Mantissa**. - Bits [3:0] contain the 4-bit **Fraction** (scaled by $\frac{1}{16}$).

$$\text{USART_BRR} = \text{Mantissa} + \frac{\text{Fraction}}{16}$$

Worked Calculation: 115,200 Baud on STM32F103 (USART2 on APB1)

Assume an STM32F103 running at 72 MHz SYSCLK with APB1 prescaler = 2 ($f_{\text{PCLK1}} = 36\ \text{MHz}$). 1. Calculate theoretical divisor:

$$\text{Divisor} = \frac{36{,}000{,}000}{16 \times 115{,}200} = \frac{36{,}000{,}000}{1{,}843{,}200} \approx 19.53125$$

2. Extract integer mantissa:

$$\text{Mantissa} = \lfloor 19.53125 \rfloor = 19 = 0\text{x}13$$

3. Extract fractional part:

$$\text{Fraction} = \text{round}(0.53125 \times 16) = \text{round}(8.5) = 9 = 0\text{x}9$$

4. Register value:

$$\text{USART_BRR} = (19 \ll 4) | 9 = 0\text{x}0139$$

5. Calculate actual baud rate and error:

$$\text{Actual Baud} = \frac{36{,}000{,}000}{16 \times \left(19 + \frac{9}{16}\right)} = \frac{36{,}000{,}000}{16 \times 19.5625} = \frac{36{,}000{,}000}{313} \approx 115{,}015.97 \text{ bps}$$

$$\text{Error} = \frac{115{,}015.97 - 115{,}200}{115{,}200} \times 100\% \approx -0.16\%$$

> **ENGINEERING NOTE:** Clock Tolerance Limit: Asynchronous serial communication requires cumulative transmitter and receiver clock frequency error to remain strictly below (and ideally below) across the entire 10-bit frame. If error exceeds , the 10th bit (Stop bit) will be sampled during the adjacent bit transition, causing catastrophic framing errors (FE). Because internal RC oscillators (HSI) drift by to over temperature, always use an external crystal oscillator (HSE) for reliable UART communication above 115,200 baud.

4.1.3 HAL Implementation: Polling, Interrupt, and DMA Modes

STM32 HAL provides three distinct execution paradigms for UART transmission and reception.

Mode 1: Polling (Blocking)

The CPU loops continuously polling the Transmit Data Register Empty (TXE) or Read Data Register Not Empty (RXNE) status flags until transfer completes or a timeout expires.

```
/* Blocking transmit: CPU halted for ~87 µs per character at 115200 baud */
uint8_t msg[] = "SENSOR_OK\r\n";
HAL_StatusTypeDef status = HAL_UART_Transmit(&huart2, msg, sizeof(msg) - 1, 100);
if (status != HAL_OK) {
    /* Handle timeout or hardware bus error */
}
```

Application: Power-on self-test (POST), panic logging inside HardFault_Handler, single-byte command consoles during initialization.

Mode 2: Interrupt-Driven (Non-Blocking Byte-by-Byte)

The CPU initiates the transfer and immediately returns to application tasks. The UART hardware triggers an interrupt upon every received byte (RXNE) or transmitted byte (TXE/TC). The HAL driver manages internal buffer pointers in the ISR and invokes a high-level callback upon completion.

```
/* Start non-blocking reception of a 1-byte command */
uint8_t rx_char;
HAL_UART_Receive_IT(&huart2, &rx_char, 1);

/* HAL weak callback override */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2) {
```

```
        ProcessCommandByte(rx_char);
        /* Re-arm receiver for the next byte */
        HAL_UART_Receive_IT(&huart2, &rx_char, 1);
    }
}
```

Application: Interactive human command shells, low-bandwidth ASCII streams (< 1000 bytes/sec). *Drawback*: At 921,600 baud, an interrupt fires every $10.85\ \mu s$. If context switching and HAL ISR overhead consume $3\ \mu s$, the CPU suffers **28% utilization** purely serving UART interrupts.

Mode 3: DMA Mode (Zero-CPU High-Throughput Streaming)

The DMA controller routes bytes directly between the peripheral data register (USART_DR or USART_RDR) and an SRAM buffer without CPU involvement.

```
/* Transmit a 512-byte telemetry packet via DMA */
uint8_t telemetry_buf[512];
FillTelemetry(telemetry_buf, sizeof(telemetry_buf));

HAL_UART_Transmit_DMA(&huart2, telemetry_buf, sizeof(telemetry_buf));
/* CPU immediately continues executing sensor control algorithms */
```

4.1.4 The UART Stream Problem: Framing and Packet Delineation

UART transmits an unformatted stream of bytes. Unlike SPI or I^2C, there is no hardware packet boundary or Slave Select edge. The application receiver must reconstruct discrete messages from continuous bytes.

Table 4.1 — UART message framing strategies

Framing Technique	Operating Principle	Advantages	Disadvantages	Best Used For
Delimiter / Newline	Stream ends with \r\n or \0	Human readable, simple string parsing	Cannot transmit raw binary (data may contain delimiter)	ASCII CLI, AT commands, NMEA GPS
Fixed-Length Packet	Every message is exactly N bytes	Trivial buffer management, zero parser state	Inflexible; padding wastes bandwidth	Periodic sensor packets (e.g., 14-byte IMU)
Length-Prefix Header	Header contains [SYNC][LEN][PAYLOAD][CRC]	Flexible binary payloads; high efficiency	Requires state-machine parser; sync loss recovery needed	Binary telemetry, inter-MCU links
COBS / SLIP Encoding	Byte stuffing replaces zero bytes; 0x00 marks boundary	Arbitrary binary packets; unambiguous packet delimitation	Encoding/decoding CPU overhead (~1-2%)	Robust robotic communication, ROSserial
UART IDLE Line Detection	Hardware triggers interrupt when RX line idles for 1 frame	Automatic variable-length reception with DMA	Sensitive to inter-byte transmission pauses	High-speed telemetry, unknown length packets

The Modern Solution: Circular DMA with IDLE Line Detection (`ReceiveToIdle`)

Starting with STM32Cube HAL v1.8+, ST introduced the HAL_UARTEx_ReceiveToIdle_DMA() API. It utilizes the hardware **IDLE flag** (USART_SR_IDLE or USART_ISR_IDLE), which asserts when the RX pin remains HIGH for at least one full character frame duration after receiving data.

Listing 4.1 — Robust variable-length packet reception via DMA and IDLE detection

```
#define RX_BUFFER_SIZE  256
uint8_t dma_rx_buf[RX_BUFFER_SIZE];

void App_UART_Init(void)
{
    /* Start DMA listening in normal or circular mode */
    HAL_UARTEx_ReceiveToIdle_DMA(&huart2, dma_rx_buf, RX_BUFFER_SIZE);
    /* Disable half-transfer interrupt if only complete packets are needed */
    __HAL_DMA_DISABLE_IT(huart2.hdmarx, DMA_IT_HT);
}

/* Invoked when either:
   1. The DMA buffer fills completely (Size == RX_BUFFER_SIZE)
   2. The transmitter stops sending bytes (IDLE line event; Size < RX_BUFFER_SIZE) */
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
    if (huart->Instance == USART2) {
        /* Process received packet of exact length 'Size' */
        App_ProcessPacket(dma_rx_buf, Size);

        /* Re-arm the DMA reception */
        HAL_UARTEx_ReceiveToIdle_DMA(&huart2, dma_rx_buf, RX_BUFFER_SIZE);
        __HAL_DMA_DISABLE_IT(huart2.hdmarx, DMA_IT_HT);
    }
}
```

4.1.5 UART Hardware Errors and Failure Modes

The UART peripheral status register monitors transmission integrity through four dedicated error flags: 1. **Overrun Error (ORE)**: A new byte arrived in the shift register while the previous byte was still unread in the Data Register (RDR/DR). - *Cause*: Interrupts disabled too long, or DMA stream stopped while data was flowing. - *Impact*: In many STM32 families (such as STM32F1), an unhandled ORE flag **freezes the receiver** and completely halts all subsequent reception! 2. **Framing Error (FE)**: The expected Stop bit was sampled as logic LOW instead of HIGH. - *Cause*: Baud rate mismatch (> 2.5%), noisy ground line, or mismatched word length/parity settings. 3. **Noise Error (NE)**: Slices 8, 9, and 10 of a bit period produced conflicting logic levels. - *Cause*: Electrical coupling from motor PWM lines or unshielded cables. 4. **Parity Error (PE)**: The calculated parity of data bits failed to match the received parity bit.

Listing 4.2 — Comprehensive UART error handler callback

```
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    uint32_t err = HAL_UART_GetError(huart);

    if (err & HAL_UART_ERROR_ORE) {
        /* Clear Overrun flag by reading SR followed by DR (F1/F4) or ICR (G0/G4/H7) */
        __HAL_UART_CLEAR_OREFLAG(huart);
    }
    if (err & HAL_UART_ERROR_FE) {
        __HAL_UART_CLEAR_FEFLAG(huart);
    }
```

```
    if (err & HAL_UART_ERROR_NE) {
        __HAL_UART_CLEAR_NEFLAG(huart);
    }
    if (err & HAL_UART_ERROR_PE) {
        __HAL_UART_CLEAR_PEFLAG(huart);
    }

    /* Abort jammed DMA transfer and re-arm reception */
    HAL_UART_AbortReceive(huart);
    HAL_UARTEx_ReceiveToIdle_DMA(huart, dma_rx_buf, RX_BUFFER_SIZE);
}
```

4.1.6 Retargeting printf via UART

Standard C library printf output can be routed directly to an STM32 UART console by implementing the low-level character output hook.

Listing 4.3 — Retargeting standard I/O to USART2 with GCC / Newlib-nano

```
#include <stdio.h>

/* GCC / STM32CubeIDE newlib-nano retarget prototype */
int __io_putchar(int ch)
{
    uint8_t c = (uint8_t)ch;
    /* Transmit single character with 10 ms timeout */
    HAL_UART_Transmit(&huart2, &c, 1, 10);
    return ch;
}

/* Optional: Support _write for POSIX compliance */
int _write(int file, char *ptr, int len)
{
    HAL_UART_Transmit(&huart2, (uint8_t *)ptr, (uint16_t)len, 100);
    return len;
}
```

PRACTICAL BENCH TIP: Float Formatting Support: In STM32CubeIDE, printf("%f") is disabled by default to save flash memory. To enable floating-point printing, navigate to:
Project Properties → C/C++ Build → Settings → Tool Settings → MCU Settings → Check "Use float with printf from newlib-nano (-u _printf_float)".

4.2 SPI (Serial Peripheral Interface)

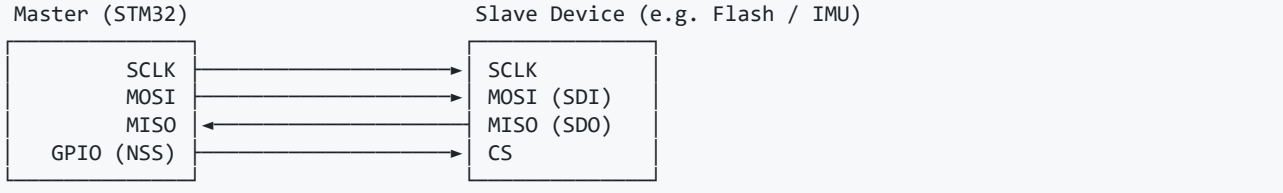
4.2.1 Operating Principle and Architecture

SPI is a synchronous, four-wire, master-slave, full-duplex serial interface engineered for high-bandwidth data transfer (up to 50+ MHz on advanced STM32 families). Communication is orchestrated entirely by a single **Master** device, which generates the serial clock ($SCLK$) and selects peripheral **Slave** devices using individual Chip Select (CS or NSS) lines.

Bus Lines

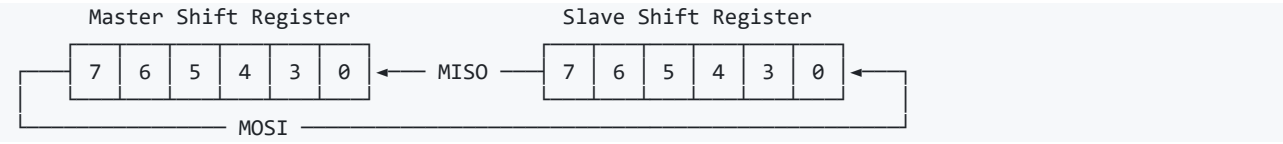
1. **SCLK (Serial Clock)**: Driven exclusively by the Master. Synchronizes bit shifting.
2. **MOSI (Master Out Slave In)**: Data line driven by Master; carries commands and write data to Slaves.

3. **MISO (Master In Slave Out)**: Data line driven by the selected Slave; carries response and read data to Master.
4. **NSS / CS (Slave Select / Chip Select)**: Active-low line driven by the Master to enable a specific slave device. Unselected slaves place their MISO pin in high-impedance (tri-state) mode.



4.2.2 The Shift Register Circular Ring

At the silicon level, SPI communication operates as a **circular dual shift-register ring**. During every clock pulse on SCLK: 1. The Master shifts one bit out of its internal shift register onto **MOSI**, while simultaneously shifting one bit in from **MISO**. 2. The Slave shifts one bit out of its internal shift register onto **MISO**, while simultaneously shifting one bit in from **MOSI**.



> **KEY CONCEPT:** Full-Duplex Implication and Dummy Clocks: In SPI, transmission and reception occur simultaneously. You cannot read a byte from a slave without transmitting a byte, and you cannot transmit a byte without receiving a byte. When an STM32 Master desires to read sensor data, it must actively clock the bus by transmitting dummy bytes (typically 0x00 or 0xFF).

4.2.3 Clock Polarity (CPOL) and Clock Phase (CPHA): The Four SPI Modes

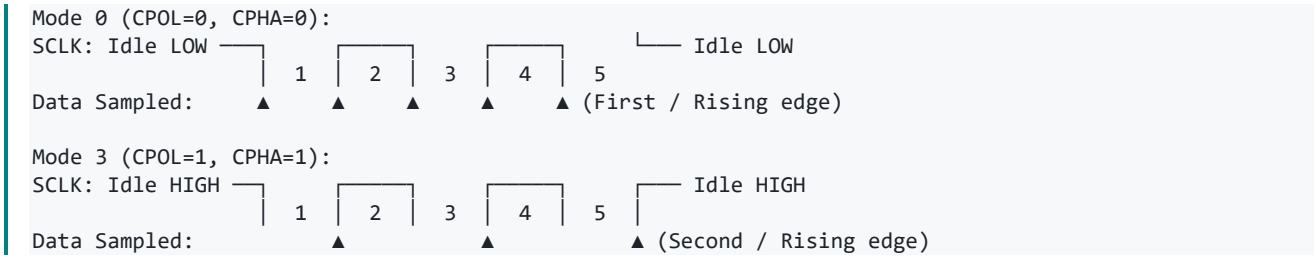
SPI devices specify their timing requirements using two configuration bits: **CPOL** and **CPHA**.

- **CPOL (Clock Polarity)**: Defines the idle logic level of SCLK when no transfer is active.
  - CPOL = 0: Clock line idles LOW.
  - CPOL = 1: Clock line idles HIGH.
- **CPHA (Clock Phase)**: Defines which clock edge captures (samples) data and which edge shifts (toggles) data.
  - CPHA = 0: Data is captured on the **first** clock transition and toggled on the second.
  - CPHA = 1: Data is toggled on the first clock transition and captured on the **second**.

Table 4.2 — The four standard SPI modes

SPI Mode	CPOL (Idle Level)	CPHA (Capture Edge)	Clock Transition for Capture	Waveform Character

SPI Mode	CPOL (Idle Level)	CPHA (Capture Edge)	Clock Transition for Capture	Waveform Character
Mode 0 (0,0)	LOW (0)	1st Edge (0)	Rising Edge	Most common for microcontrollers, sensors, and flash
Mode 1 (0,1)	LOW (0)	2nd Edge (1)	Falling Edge	Common in industrial ADCs and motor drivers
Mode 2 (1,0)	HIGH (1)	1st Edge (0)	Falling Edge	Infrequently used
Mode 3 (1,1)	HIGH (1)	2nd Edge (1)	Rising Edge	Common alternative for high-speed Flash/FRAM



4.2.4 Hardware NSS vs. Software Chip Select (CS)

CubeMX allows configuring the SPI slave select pin as **Hardware NSS** or **Software CS**.

Why Software CS is the Universal Engineering Choice

- **Hardware NSS Pitfall**: In STM32 SPI master hardware mode, the Hardware NSS pin automatically drives LOW at peripheral enable, but **pulses high or stays low unpredictably between bytes** depending on family errata and FIFO configurations. Most external SPI sensors require CS to remain continuously LOW across a multi-byte command/data burst, and toggle HIGH only when the entire transaction completes.
- **Multi-Slave Limitation**: Hardware NSS supports exactly **one** slave per SPI peripheral. A system with an IMU, an SD card, and an external flash would require three separate SPI peripherals.
- **The Solution**: In CubeMX, set **NSS = Disable (Software)**. Configure standard GPIO output pins (e.g., PB12, PA4, PC4) as Chip Select lines, pulling them LOW before a transmission and returning them HIGH after completion.

Listing 4.4 — Reusable SPI register read/write transaction with Software CS

```
#define CS_GPIO_PORT    GPIOB
#define CS_PIN          GPIO_PIN_12

#define CS_SELECT()     HAL_GPIO_WritePin(CS_GPIO_PORT, CS_PIN, GPIO_PIN_RESET)
#define CS_DESELECT()   HAL_GPIO_WritePin(CS_GPIO_PORT, CS_PIN, GPIO_PIN_SET)

/* Read an 8-bit register from an SPI sensor */
uint8_t SPI_ReadRegister(SPI_HandleTypeDef *hspi, uint8_t reg_addr)
{
    uint8_t tx_buf[2];
```

```
    uint8_t rx_buf[2];

    /* Most SPI sensors specify MSB=1 in the register address byte for READ */
    tx_buf[0] = reg_addr | 0x80;
    tx_buf[1] = 0xFF; /* Dummy byte to clock in the response */

    CS_SELECT();
    HAL_SPI_TransmitReceive(hspi, tx_buf, rx_buf, 2, 50);
    CS_DESELECT();

    return rx_buf[1]; /* Received sensor payload */
}

/* Burst read multiple bytes (e.g., 6-axis IMU raw data) */
HAL_StatusTypeDef SPI_ReadBurst(SPI_HandleTypeDef *hspi, uint8_t start_reg,
                                uint8_t *dest, uint16_t length)
{
    uint8_t cmd = start_reg | 0x80;
    HAL_StatusTypeDef status;

    CS_SELECT();
    /* Send register command */
    status = HAL_SPI_Transmit(hspi, &cmd, 1, 10);
    if (status == HAL_OK) {
        /* Clock out data bytes using dummy transfers */
        status = HAL_SPI_Receive(hspi, dest, length, 50);
    }
    CS_DESELECT();

    return status;
}
```

4.2.5 SPI Timing Calculations and Maximum Baud Rates

SPI clock frequency is derived directly from the host bus clock (f_{PCLK1} or f_{PCLK2}) divided by a power-of-two prescaler:

$$f_{SCK} = \frac{f_{PCLK}}{\text{Prescaler}}, \quad \text{Prescaler} \in \{2,4,8,16,32,64,128,256\}$$

Example: STM32F401 SPI1 ($f_{PCLK2} = 84$ MHz)

- Prescaler $2 \Longrightarrow f_{SCK} = 42$ MHz (High-speed displays, external Flash)
- Prescaler $8 \Longrightarrow f_{SCK} = 10.5$ MHz (Most IMUs and precision ADCs)
- Prescaler $64 \Longrightarrow f_{SCK} = 1.3125$ MHz (Low-speed optical mice, long ribbon cables)

COMMON MISTAKE / CAUTION: Signal Integrity on High-Speed SPI: At frequencies above 10 MHz, SPI edges exhibit sub-nanosecond rise times. Without series termination resistors () placed near the transmitter, signal reflections and ringing on SCLK will create false clock edges, corrupting the transfer. Keep PCB traces under 10 cm and provide a continuous ground plane directly beneath SCLK and MOSI/MISO.

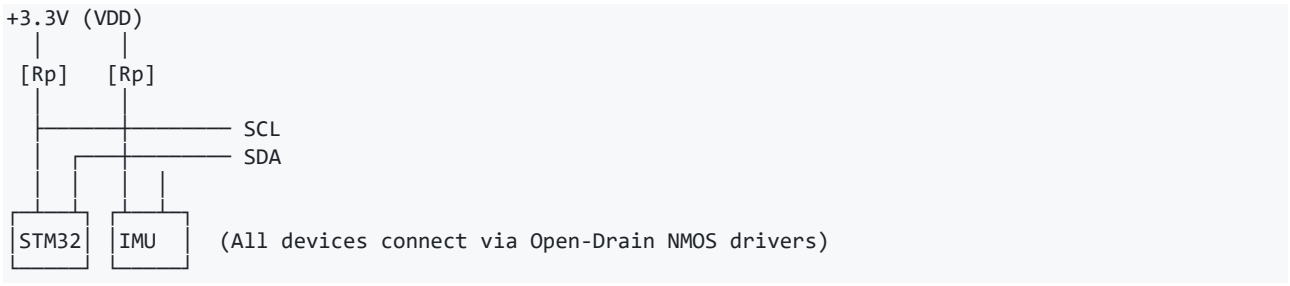
4.3 I²C (Inter-Integrated Circuit)

4.3.1 Operating Principle: The Open-Drain Wired-AND Bus

I²C is a synchronous, multi-master, multi-slave, half-duplex serial bus developed by Philips (NXP) that connects multiple integrated circuits using only **two bidirectional lines**: 1. **SDA (Serial Data)**: Carries data, addresses, and acknowledgment bits. 2. **SCL (Serial Clock)**: Synchronizes bit transfer; driven by the active Master (with optional clock stretching by Slaves).

Open-Drain Architecture

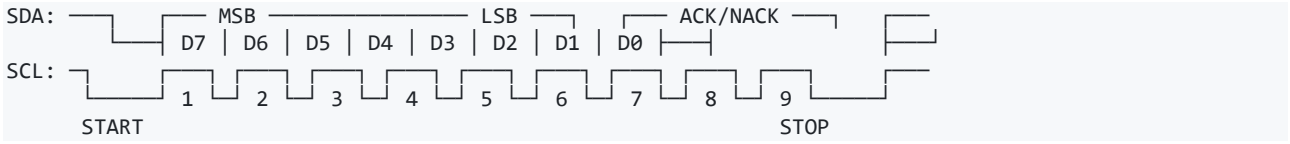
Unlike UART and SPI, which use push-pull output drivers (where transistors actively drive the line HIGH to V_{DD} or LOW to GND), I²C outputs operate strictly in **open-drain** mode: - To transmit a logic **LOW (0)**: The internal MOSFET turns ON, pulling the line to GND. - To transmit a logic **HIGH (1)**: The internal MOSFET turns OFF, leaving the line floating. External **pull-up resistors (R_p)** pull the line voltage up to V_{DD}.



This creates a **Wired-AND** circuit: if even one device pulls the line LOW, the entire bus remains LOW. This physical property enables: 1. **Clock Stretching**: A slow slave can hold SCL LOW to halt the Master until its internal ADC finishes converting. 2. **Multi-Master Arbitration**: If two masters transmit simultaneously, the master asserting a '0' wins the bus without causing a short circuit, while the master attempting to send a '1' senses the mismatch and immediately yields.

4.3.2 Packet Structure: START, STOP, and ACK/NACK

Every transaction on an I²C bus follows a rigid packet structure clocked in 9-bit groups (8 bits data + 1 bit acknowledge).



1. **START Condition (S)**: A transition from HIGH to LOW on the SDA line while SCL remains HIGH. Signals all slaves to listen for an address.
2. **Address Byte (7-bit Address + R/W bit)**:
    - Bits [7:1]: 7-bit slave device address.
    - Bit [0]: Transfer direction (0 = Master Write, 1 = Master Read).
3. **Acknowledge (ACK) / Negative Acknowledge (NACK)**:
    - During the 9th SCL clock pulse, the transmitter releases SDA (floating HIGH).
    - If a slave recognizes its address or successfully receives a byte, it pulls SDA **LOW** (**ACK**).

- If the slave fails to pull SDA LOW, the line floats HIGH (**NACK**), indicating an error, an unpopulated device, or end of read.

4. **Data Bytes**: 8-bit payloads transferred MSB first, each followed by an ACK/NACK bit from the receiver.
5. **Repeated START Condition (Sr)**: A new START condition issued without first issuing a STOP condition. Used to switch from write to read mode atomically without releasing bus control.
6. **STOP Condition (P)**: A transition from LOW to HIGH on the SDA line while SCL remains HIGH. Returns the bus to the IDLE state.

4.3.3 The 7-Bit Address Left-Shift Requirement in STM32 HAL

One of the most frequent traps encountered by embedded engineers involves I²C device addressing in STM32 HAL.

Sensor datasheets specify the device address as a **7-bit hexadecimal value**. For example: - **MPU6050 (AD0 pin LOW)**: Address = 0x68 (0b1101000) - **SSD1306 OLED**: Address = 0x3C (0b0111100) - **BMP280 (SDO pin to GND)**: Address = 0x76 (0b1110110)

However, the hardware I²C shift register places the 7 address bits into bits [7:1] of the first transmitted byte, leaving bit [0] for the Read/Write flag:

```
Transmitted Address Byte:
┌────┬────┬────┬────┬────┬────┬────┬─────┐
│ A6 │ A5 │ A4 │ A3 │ A2 │ A1 │ A0 │ R/W │
└────┴────┴────┴────┴────┴────┴────┴─────┘
 Bit 7                          Bit 0
```

> **WARNING:** The HAL 1-Bit Shift Rule: The STM32 HAL functions (HAL_I2C_Master_Transmit, HAL_I2C_Mem_Read, etc.) require the DevAddress argument to be pre-shifted left by 1 bit:

```
#define MPU6050_7BIT_ADDR   0x68
#define MPU6050_HAL_ADDR    (MPU6050_7BIT_ADDR << 1) /* 0xD0 */
```

If you pass unshifted `0x68` to HAL, the hardware transmits `0x34` on the bus, and the sensor will never respond with an ACK!

4.3.4 Physical Layer Mathematics: Sizing Pull-Up Resistors (R_p)

Because open-drain pins cannot drive a line HIGH, the rising edge of both SCL and SDA is governed entirely by an RC charging curve formed by the pull-up resistor R_p and the total parasitic bus capacitance C_b (pin input capacitance + PCB trace capacitance + connector capacitance):

$$V(t) = V_{DD}\left(1 - e^{-\frac{t}{R_p C_b}}\right)$$

Under the NXP I²C specification (UM10204), the rise time t_r is measured between the 30% threshold ($0.3V_{DD}$) and the 70% threshold ($0.7V_{DD}$). The mathematical time elapsed between these two voltages is:

$$t_r = R_p C_b \left(\ln\frac{1}{1-0.7} - \ln\frac{1}{1-0.3}\right) = R_p C_b(\ln(3.333) - \ln(1.428)) \approx 0.8473 \times R_p C_b$$

1. Maximum Resistor Value ($R_{p,\max}$)

To guarantee that the signal rises fast enough to meet the maximum allowable rise time $t_{r,\max}$ at the chosen speed:

$$R_{p,\max} = \frac{t_{r,\max}}{0.8473 \times C_b}$$

- **Standard Mode (100 kHz)**: $t_{r,\max} = 1000$ ns. For $C_b = 100$ pF:

$$R_{p,\max} = \frac{1000 \times 10^{-9}}{0.8473 \times 100 \times 10^{-12}} \approx 11.8\,\text{k}\Omega$$

- **Fast Mode (400 kHz)**: $t_{r,\max} = 300$ ns. For $C_b = 100$ pF:

$$R_{p,\max} = \frac{300 \times 10^{-9}}{0.8473 \times 100 \times 10^{-12}} \approx 3.54\,\text{k}\Omega$$

2. Minimum Resistor Value ($R_{p,\min}$)

When an STM32 pin drives LOW, the pull-up resistor conducts current into the output transistor. To prevent damaging the pin and ensure the low-level output voltage stays below $V_{OL,\max} = 0.4$ V at the rated sink current $I_{OL} = 3$ mA:

$$R_{p,\min} = \frac{V_{DD} - V_{OL,\max}}{I_{OL}} = \frac{3.3\,\text{V} - 0.4\,\text{V}}{3\,\text{mA}} \approx 967\ \Omega$$

Summary Rule of Thumb for 3.3V Systems: - 4.7 kΩ: Universal standard for 100 kHz Standard Mode. - 2.2 kΩ: Required for 400 kHz Fast Mode and buses with 3+ sensors. - **Internal Weak Pull-ups (40 kΩ)**: Completely inadequate! They yield rise times exceeding 3.5 μs, causing severe I²C clock distortion.

4.3.5 HAL Sensor Communication: HAL_I2C_Mem_Read vs. HAL_I2C_Master_Receive

Most digital sensors organize their internal configuration and data in 8-bit registers. Reading a sensor register requires a **write-then-read** sequence: 1. Master transmits START + Device Address (Write). 2. Master transmits the Target Register Pointer (e.g., 0x3B for MPU6050 Accelerometer). 3. Master issues a **Repeated START** + Device Address (Read). 4. Slave clocks out register contents while Master acknowledges. 5. Master terminates with NACK + STOP.

STM32 HAL wraps this entire sequence into a single atomic function: HAL_I2C_Mem_Read().

Listing 4.5 — Efficient sensor register read and write operations

```
#define MPU6050_ADDR    (0x68 << 1)
#define REG_PWR_MGMT_1  0x6B
#define REG_ACCEL_XOUT  0x3B

/* Write a single 8-bit register */
HAL_StatusTypeDef Sensor_WriteReg(I2C_HandleTypeDef *hi2c, uint8_t reg, uint8_t val)
{
    return HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, reg,
                             I2C_MEMADD_SIZE_8BIT, &val, 1, 100);
}

/* Read a continuous burst of 14 bytes (Accel, Temp, Gyro) in one I2C transaction */
HAL_StatusTypeDef Sensor_ReadBurst(I2C_HandleTypeDef *hi2c, uint8_t start_reg,
                                   uint8_t *buffer, uint16_t len)
```

```
{
    return HAL_I2C_Mem_Read(hi2c, MPU6050_ADDR, start_reg,
                            I2C_MEMADD_SIZE_8BIT, buffer, len, 100);
}
```

4.3.6 The Famous I²C Bus Lockup and 9-Clock Hardware Recovery

One of the most frustrating failure modes in embedded development occurs when an STM32 resets (due to a watchdog timeout, brownout, or debugger connection) in the middle of reading an I²C slave byte.

Why the Bus Locks Up

If the MCU resets while the slave is transmitting a data bit of value **0**, the slave's open-drain driver remains clamped to GND, holding the **SDA line permanently LOW**. When the STM32 reboots, its I²C peripheral initializes and checks the bus state. Seeing SDA LOW, the STM32 hardware controller reports **HAL_I2C_STATE_BUSY** and refuses to transmit START conditions! The entire firmware halts permanently.

The 9-Clock Recovery Sequence

To free a jammed slave without power-cycling the board, the MCU must manually toggle the SCL clock up to 9 times to complete the slave's internal shift cycle, followed by a STOP condition.

Listing 4.6 — I²C GPIO unjam / bus recovery procedure

```
void I2C_Bus_Clear(GPIO_TypeDef *scl_port, uint16_t scl_pin,
                   GPIO_TypeDef *sda_port, uint16_t sda_pin)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* 1. Configure SCL and SDA as GPIO open-drain outputs */
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;

    GPIO_InitStruct.Pin = scl_pin;
    HAL_GPIO_Init(scl_port, &GPIO_InitStruct);
    GPIO_InitStruct.Pin = sda_pin;
    HAL_GPIO_Init(sda_port, &GPIO_InitStruct);

    /* 2. Release SDA and check if line is clamped LOW */
    HAL_GPIO_WritePin(sda_port, sda_pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(scl_port, scl_pin, GPIO_PIN_SET);
    HAL_Delay(1);

    /* If SDA is clamped LOW by a confused slave, clock SCL up to 9 times */
    if (HAL_GPIO_ReadPin(sda_port, sda_pin) == GPIO_PIN_RESET) {
        for (int i = 0; i < 9; i++) {
            HAL_GPIO_WritePin(scl_port, scl_pin, GPIO_PIN_RESET);
            for (volatile int d = 0; d < 200; d++); /* ~5 µs delay */
            HAL_GPIO_WritePin(scl_port, scl_pin, GPIO_PIN_SET);
            for (volatile int d = 0; d < 200; d++);

            /* Check if slave released SDA */
            if (HAL_GPIO_ReadPin(sda_port, sda_pin) == GPIO_PIN_SET) {
                break;
            }
        }
    }

    /* 3. Generate a STOP condition: SDA transitions LOW to HIGH while SCL is HIGH */
    HAL_GPIO_WritePin(sda_port, sda_pin, GPIO_PIN_RESET);
    for (volatile int d = 0; d < 200; d++);
```

```
    HAL_GPIO_WritePin(scl_port, scl_pin, GPIO_PIN_SET);
    for (volatile int d = 0; d < 200; d++);
    HAL_GPIO_WritePin(sda_port, sda_pin, GPIO_PIN_SET);
    for (volatile int d = 0; d < 200; d++);

    /* 4. Pins will be reconfigured as Alternate Function by MX_I2C1_Init() */
}
```

Table 4.3 — I²C debugging and symptom diagnostic matrix

Symptom	Primary Root Cause	Corrective Engineering Action
HAL_BUS_ERROR / NACK on first byte	Incorrect slave address; forgotten 1-bit left shift	Shift 7-bit address left: (ADDR << 1)
HAL_BUSY immediately upon boot	SDA clamped LOW by a hung slave from previous reset	Execute 9-clock bit-bang unjam sequence before MX_I2C_Init()
Intermittent NACKs at 400 kHz	Pull-up resistors too weak ($10\ \text{k}\Omega$); rounded edges	Replace pull-ups with $2.2\ \text{k}\Omega$; inspect SCL on oscilloscope
Driver hangs inside HAL_I2C_Mem_Read	Slave asserting continuous Clock Stretching	Pass strict timeout parameter (e.g., 100 ms) instead of HAL_MAX_DELAY
WHO_AM_I reads 0x00 or 0xFF	Wrong register address, sensor in reset, or V_{DD} unpowered	Verify power rails with DMM; verify reset pin logic state

4.4 Comprehensive Protocol Comparison and Selection Matrix

Choosing the correct serial protocol is an architectural decision that directly influences circuit board complexity, electromagnetic emissions, MCU pin utilization, software complexity, and system throughput.

Table 4.4 — Comprehensive technical comparison of serial protocols

Technical Parameter	UART / USART	SPI	I²C
Topology	Point-to-point (2 devices)	Single Master, Multi-Slave	Multi-Master, Multi-Slave
Number of Wires	**2** (TX, RX) + GND	**3 + N** (SCLK, MOSI, MISO + CS per slave)	**2** (SDA, SCL) + GND
Duplex Capability	Full-Duplex	Full-Duplex	Half-Duplex
Clock Synchronization	Asynchronous (Agreed Baud)	Synchronous (Dedicated SCLK)	Synchronous (Dedicated SCL)
Maximum Data Rate	Typical: 115.2 kbps – 921.6 kbpsMax: 10–27 Mbps (High-end)	Typical: 10 Mbps – 40 MbpsMax: 80+ Mbps	Standard: 100 kbpsFast: 400 kbpsFast+: 1 Mbps
Maximum Distance	Single board: 2–3 m(RS-232/485: up to 1200 m)	Very Short: < 30 cm on PCB	Short: < 1 m (governed by 400 pF bus capacitance)

Technical Parameter	UART / USART	SPI	I²C
Hardware Addressing	None (Software framing required)	None (Hardware CS pin asserts slave)	Built-in 7-bit or 10-bit software addressing
Flow Control	Optional RTS/CTS lines	None	Built-in Clock Stretching and ACK/NACK
Bus Contention / Safety	Driver collision if outputs shorted	Driver collision if multiple CS asserted	Safe (Open-drain Wired-AND logic)
Typical Transmit Overhead	20% (1 Start + 1 Stop bit per 8 bits)	**0%** (Pure raw streaming bytes)	~20–30% (START, Address, ACK per byte)
Primary Use Cases	Debug console, GPS, Cellular modems	Flash memory, Displays, High-rate IMUs	Temp sensors, RTC, EEPROM, Configuration

---

4.4.1 Industrial Protocols Overview: CAN, RS-485, USB, and Ethernet

For applications demanding longer physical transmission lines, high electrical noise immunity, or multi-megabyte throughput, standard UART/SPI/I²C protocols are superseded by industrial standards:

1. **CAN Bus (Controller Area Network)**:
   - *Physical Layer*: Differential balanced two-wire bus (CAN_H and CAN_L) terminated with $120\ \Omega$ at both ends.
   - *Key Advantage*: Built-in non-destructive bitwise arbitration, automatic CRC error detection, hardware message filtering, and exceptional immunity to ground offsets and motor noise.
   - *Use*: Robotics drivetrain, automotive ECUs, UAV avionics.
2. **RS-485**:
   - *Physical Layer*: Differential UART signaling over twisted-pair wiring (A and B).
   - *Key Advantage*: Extends standard UART streams up to 1200 meters and supports up to 32 transceivers in multi-drop configurations.
   - *Use*: Industrial Modbus networks, PTZ surveillance cameras, long-range solar sensors.
3. **USB (Universal Serial Bus)**:
   - *Physical Layer*: Half-duplex differential pair (D+, D-) operating at Full Speed (12 Mbps) or High Speed (480 Mbps).
   - *Key Advantage*: Native plug-and-play PC connectivity, packetized enumeration, virtual COM port (VCP).
   - *Use*: Firmware flashing, host computer high-speed data acquisition.
4. **Ethernet (10/100/1000 Mbps)**:
   - *Physical Layer*: RMII/MII interface to external PHY chip; transformer-isolated twisted pairs.
   - *Key Advantage*: Standard TCP/IP networking, global routability, extremely high bandwidth.

- *Use*: Industrial IoT gateways, vision processing nodes.

4.4.2 Practical Engineering Selection Scenarios

To decide which protocol to use when designing an STM32-based robotic system:

Scenario A: High-Rate 6-Axis IMU (MPU6050 / ICM-20689)

- *Bandwidth Requirement*: Reading 14 bytes (Accel, Gyro, Temp) at 1000 Hz ($14\ \text{kB/s}$).
- *Evaluation*:
  - **I²C at 100 kHz**: Each 14-byte read requires 155 clock cycles $\approx 1.55\ \text{ms}$. Cannot achieve 1000 Hz!
  - **I²C at 400 kHz**: 155 clock cycles $\approx 388\ \mu\text{s}$. Consumes 39% of the CPU bus bandwidth.
  - **SPI at 10 MHz**: 15 bytes clocked out in $12\ \mu\text{s}$. Negligible bus utilization (< 1.2%).
- *Decision*: **Use SPI** if available on the sensor breakout. Use I²C only if pin constraints force it.

Scenario B: Debugging Telemetry Console to PC

- *Requirement*: Simple, streaming textual diagnostics to terminal software (PuTTY / TeraTerm).
- *Decision*: **Use UART (115,200 or 921,600 baud)** connected to the ST-LINK Virtual COM Port (VCP). Universal, non-intrusive, requires no dedicated PC drivers.

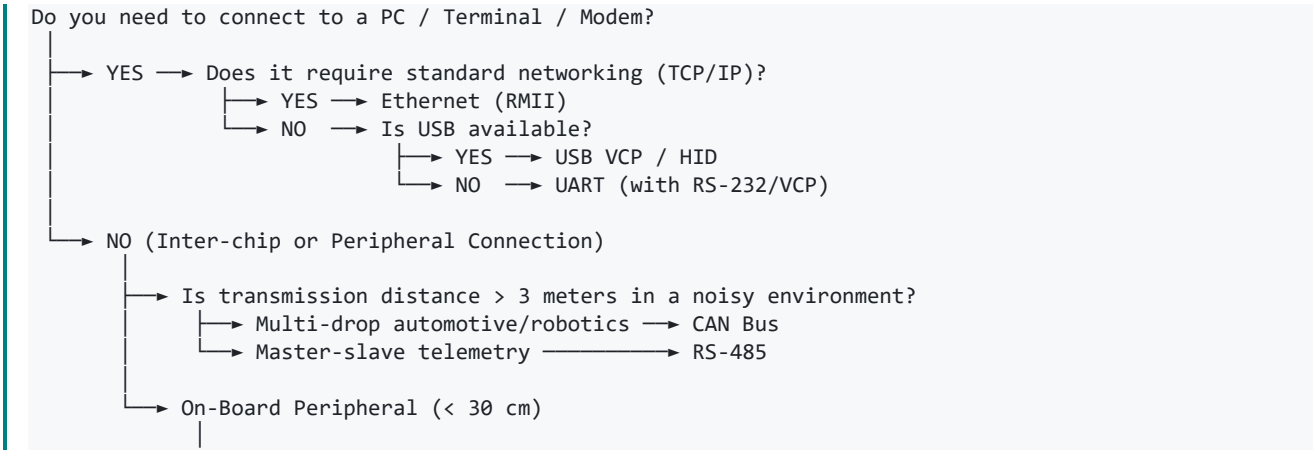
Scenario C: External 128 Mbit Flash Memory (W25Q128)

- *Requirement*: Storing graphics assets and logging large flight telemetry arrays at high speed.
- *Decision*: **Use SPI (or Quad-SPI)**. Quad-SPI allows 4 data bits per clock, achieving up to 40 MB/sec throughput.

Scenario D: Multiple Environmental Sensors (Temperature, Humidity, Pressure, RTC)

- *Requirement*: Sampling 4 different low-bandwidth sensors every 1 second across a small robot chassis.
- *Decision*: **Use I²C**. Connects all 4 devices on the exact same two pins (SDA and SCL), conserving vital microcontroller pins for motor PWM and encoder timers.

4.4.3 Architectural Decision Flowchart



```
                ├──► High Speed (> 1 Mbps), Displays, Flash, or Fast IMUs?
                │      └──► Use SPI (Dedicated CS per slave)
                │
                └──► Multiple low-speed sensors (< 400 kbps), low pin count?
                       └──► Use I²C (Shared SDA/SCL, 2.2kΩ pull-ups)
```

5. Part 4 — Sensors, ADC, and Real-Time Data Acquisition

Physical engineering systems do not deal in abstract software states; they interact with the physical world through voltages, currents, temperatures, pressures, accelerations, and electromagnetic fields. For an embedded microcontroller to perceive its environment, it must interface with two primary classes of sensors: 1. **Analog Sensors**: Transduce physical phenomena into continuous electrical potentials ($0\ \text{V}$ to V_{DD}), which the microcontroller quantizes using an **Analog-to-Digital Converter (ADC)**. 2. **Digital Sensors**: Contain integrated front-end instrumentation amplifiers, ADCs, and digital logic, exposing internal registers accessed over serial buses (**I²C**, **SPI**, or **UART**).

This chapter examines the physical, mathematical, and algorithmic mechanisms of embedded data acquisition. We analyze ADC quantization mechanics, input impedance and sampling capacitor dynamics, internal STM32 calibration channels, register-level digital sensor interrogation, deterministic real-time data flows, digital filtering (moving average and fixed-point exponential smoothing), and high-performance DMA cache coherency on ARM Cortex-M7 architectures.

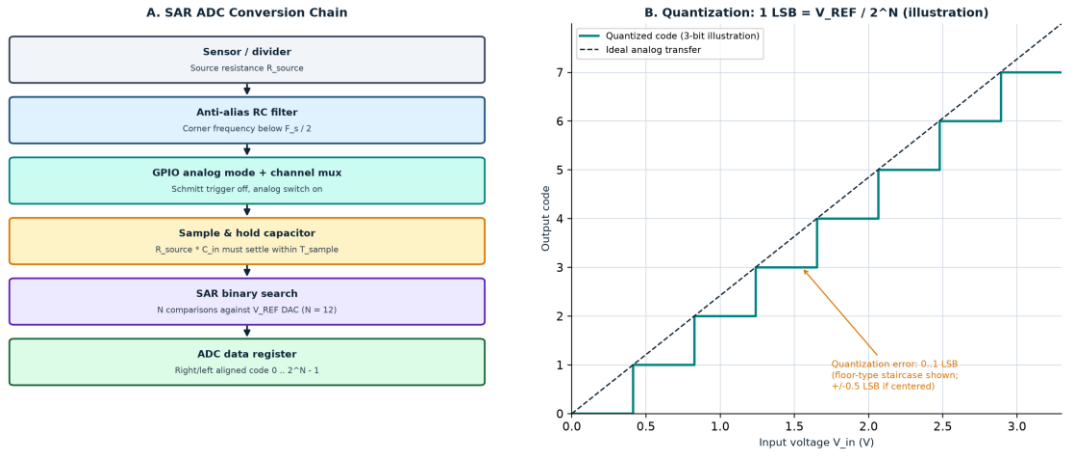


Figure 5.1 — Analog data acquisition: (A) Complete front-end signal chain from physical transducer to DMA buffer; (B) Internal Successive Approximation Register (SAR) ADC architecture with sampling capacitor C_{ADC} and comparator; (C) Quantization staircase showing finite resolution steps, quantization error (± 0.5 LSB), and full-scale saturation.

5.1 The Sensor Taxonomy

Before selecting peripheral hardware and writing driver code, an engineer must classify the sensor according to its electrical interface and acquisition timing requirements.

Table 5.1 — Embedded sensor interface taxonomy

Category	Transducer Examples	Physical Signal	STM32 Interfacing Hardware	Acquisition Strategy

Category	Transducer Examples	Physical Signal	STM32 Interfacing Hardware	Acquisition Strategy
Analog Voltage	Potentiometer, LM35, Force-Sensing Resistor (FSR), Sharp IR	Continuous $0\ \text{V} - V_{\text{REF}}$	ADC (Internal SAR)	Timer-triggered DMA or Polled ADC
Variable Resistance	NTC/PTC Thermistor, Photoresistor (LDR), Strain Gauge	Variable resistance (R_x)	Voltage divider / Wheatstone Bridge → ADC	Periodic ADC conversion + math conversion
Digital Register Bus	MPU6050 (IMU), BMP280 (Pressure), BNO055 (AHRS)	Byte packets over synchronous bus	I^2C or SPI Peripheral	Periodic polled/DMA burst read + DRDY interrupt
Serial ASCII / Stream	NMEA GPS (NEO-6M), LIDAR (TF-Luna), Ultrasonic	Asynchronous serial byte stream	UART / USART	Circular DMA with IDLE Line detection
Pulse / Frequency	Optical Wheel Encoder, Flow meter, Hall Effect	Digital pulses, square wave frequency	Timer Input Capture / Encoder Interface	Hardware counter gating (Zero CPU load)
PWM Output	Ultrasonic (HC-SR04), MEMS Inclinometers	High-time pulse width (T_{high})	Timer Input Capture (Dual channel)	Timer capture on rising and falling edges

5.2 Analog-to-Digital Converter (ADC) Fundamentals

The STM32 internal ADC is a **Successive Approximation Register (SAR)** converter. A SAR ADC converts an unknown continuous analog input voltage V_{IN} into an N-bit digital integer code by performing a binary search across N comparison cycles using an internal capacitive Digital-to-Analog Converter (DAC) and high-speed analog comparator.

5.2.1 Resolution, Reference Voltage (V_{REF}), and Quantization

- **Resolution (N)**: The number of binary bits produced per conversion (typically 12 bits on STM32F1/F4, up to 16 bits on STM32H7). An N-bit converter produces 2^N discrete digital output codes (e.g., $2^{12} = 4096$ levels, spanning codes 0 to 4095).
- **Reference Voltage ($V_{\text{REF+}}$ / $V_{\text{REF−}}$)**: The upper and lower electrical boundary voltages defining the converter's full-scale input span. On smaller packages (e.g., LQFP48 on STM32F103 "Blue Pill"), $V_{\text{REF+}}$ is internally tied directly to the analog supply rail V_{DDA} ($3.3\ \text{V}$), and $V_{\text{REF−}}$ is tied to V_{SSA} ($0\ \text{V}$). On larger packages (LQFP100+), dedicated $V_{\text{REF+}}$ pins allow connecting precision low-noise voltage references.

Least Significant Bit (LSB) Voltage

The voltage weight of one Least Significant Bit (LSB) represents the fundamental quantization step size:

$$\text{LSB} = \frac{V_{\text{REF+}} - V_{\text{REF-}}}{2^N} = \frac{3.3\ \text{V}}{4096} \approx 0.80566\ \text{mV}$$

Ideal Conversion Transfer Functions

For an ideal SAR ADC with floor-type integer truncation:

$$\text{Code} = \left\lfloor \frac{V_{\text{IN}}}{V_{\text{REF+}}} \times 2^N \right\rfloor$$

To reconstruct the equivalent analog input voltage from a digital reading in application code:

$$V_{\text{IN}} = \text{Code} \times \frac{V_{\text{REF+}}}{2^N - 1} = \text{Code} \times \frac{3.3\ \text{V}}{4095}$$

> **ENGINEERING NOTE:** Divisor Convention (vs.): Pure theoretical quantization models divide by (). However, because the maximum digital code that can fit in a 12-bit register is strictly (0x0FFF), scaling by ensures that an input voltage exactly equal to produces exactly in software. For high-precision metrology, use the calibrated internal reference voltage described in Section 5.3.

Resolution vs. Accuracy vs. ENOB

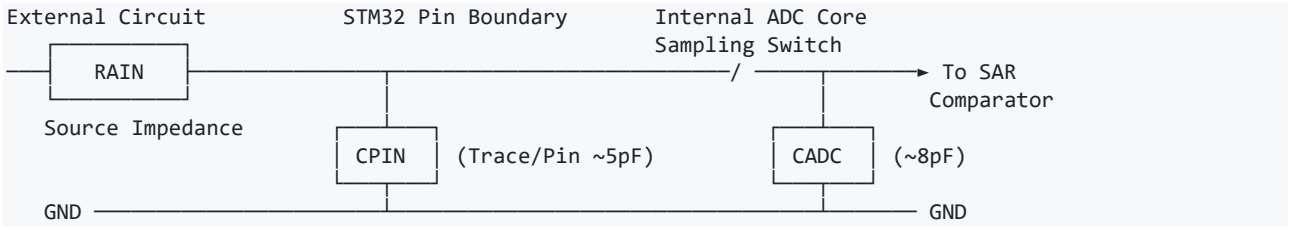
A common junior engineering mistake is confusing **resolution** with **accuracy**: - **Resolution**: The finest step size the converter can distinguish ($12\ \text{bits} \implies 0.8\ \text{mV}$). - **Accuracy**: The real-world deviation between the converted digital code and the true absolute voltage, dictated by **Offset Error**, **Gain Error**, **Integral Non-Linearity (INL)**, and **Differential Non-Linearity (DNL)**. - **Effective Number of Bits (ENOB)**: Reflects total dynamic performance including thermal noise and harmonic distortion (THD):

$$\text{ENOB} = \frac{\text{SINAD} - 1.76\ \text{dB}}{6.02}$$

A typical STM32 12-bit ADC has an ENOB between 10.2 **and** 10.8 **bits** when sampled over a noisy breadboard.

5.2.2 The Sampling Switch, R_{AIN}, and Settling Dynamics

To perform a conversion, the SAR ADC must first capture an accurate electrical charge from the external signal onto its internal sampling capacitor C_{ADC} (typically 5 pF to 8 pF on STM32) through the internal analog switch resistance R_{ADC} (typically $1\ \text{k}\Omega$).



When the sampling switch closes for duration T_{sample}, an RC low-pass circuit is formed with time constant:

$$\tau = (R_{\text{AIN}} + R_{\text{ADC}}) \times C_{\text{ADC}}$$

For a 12-bit ADC, the voltage across C_{ADC} must settle to within $\frac{1}{4}$ LSB (99.988% of the true input voltage) before the switch opens and the conversion begins. Achieving this requires a minimum settling time of at least 8.3 **time constants**:

$$T_{\text{sample}} \geq 8.3 \times (R_{\text{AIN}} + R_{\text{ADC}}) \times C_{\text{ADC}}$$

Practical Engineering Impact

If the external source impedance R_{AIN} is high (for example, a high-resistance $100\ \text{k}\Omega$ voltage divider or unbuffered potentiometer) and T_{sample} is set too short (e.g., 1.5 ADC clock cycles in CubeMX): 1. The sampling capacitor does not finish charging. 2. The ADC converts an incorrect, lower voltage. 3. Charge injection from switching between channels bleeds over, causing **cross-channel crosstalk** (reading channel 1 corrupts channel 2).

> **WARNING:** Maximum Source Impedance Rule: According to STM32 datasheets, for the fastest sampling time (cycles at), must not exceed . If your sensor or resistive divider has an output impedance of to , you must either: 1. Increase the sampling time in CubeMX to or cycles, or 2. Insert an external operational amplifier configured as an analog voltage follower (buffer).

5.2.3 Conversion Time and ADC Clock Limits

Total ADC conversion time per channel is the sum of sampling time and successive approximation cycles:

$$T_{\text{conv}} = T_{\text{sample}} + 12.5 \text{ ADC clock cycles (for 12-bit resolution)}$$

On the STM32F103 family: - **Maximum ADC Clock Frequency (f_{ADC})**: Strictly 14 MHz. Exceeding 14 MHz results in non-monotonicity and missing codes. - Since SYSCLK is typically 72 MHz and APB2 (f_{PCLK2}) is 72 MHz, the ADC Prescaler in CubeMX must be set to at least PCLK2 divided by 6 ($72\ \text{MHz}/6 = 12\ \text{MHz}$) or divided by 8 ($9$ MHz).

Worked Example: Maximum Sampling Rate on STM32F103

With $f_{\text{ADC}} = 12$ MHz ($T_{\text{ADCCLK}} = \frac{1}{12\ \text{MHz}} \approx 83.33$ ns) and setting $T_{\text{sample}} = 1.5$ cycles:

$$T_{\text{conv}} = 1.5 + 12.5 = 14 \text{ clock cycles}$$

$$T_{\text{conv}} = 14 \times 83.33\ \text{ns} \approx 1.17\ \mu\text{s}$$

$$F_{s,\max} = \frac{1}{1.17\ \mu\text{s}} \approx 857{,}142 \text{ samples/sec (857 kSPS)}$$

With a high-impedance sensor requiring $T_{\text{sample}} = 239.5$ cycles:

$$T_{\text{conv}} = 239.5 + 12.5 = 252 \text{ clock cycles} \Longrightarrow 21.0\ \mu\text{s} \Longrightarrow F_{s,\max} \approx 47.6 \text{ kSPS}$$

5.3 Internal ADC Channels: V_{REFINT}, Temperature Sensor, and V_{BAT}

Every STM32 contains internal analog channels multiplexed into the primary ADC (typically ADC1).

5.3.1 Internal Voltage Reference (V_{REFINT})

V_{REFINT} is an internal, temperature-stable bandgap voltage reference ($V_{\text{REFINT_TYP}} \approx 1.20$ V on STM32F1/F4).

Compensating for Supply Voltage (V_{DDA}) Drift

When powering an STM32 from a battery or unregulated USB rail, V_{DDA} may sag from 3.3 V down to 2.9 V. Since the ADC uses V_{DDA} as its reference, all digital conversion readings will drift upwards, reporting false

physical sensor values! To eliminate this error, sample the internal channel V_{REFINT}. The true operating supply voltage can be computed dynamically:

$$V_{DDA,\text{actual}} = V_{\text{REFINT_TYP}} \times \frac{4095}{\text{Code}_{V_{\text{REFINT}}}}$$

On modern families (STM32F4/G4/L4/H7), ST factory-calibrates V_{REFINT} at $3.3\ \text{V}$ and stores the raw conversion code in system ROM (VREFINT_CAL_ADDR, typically 0x1FFF7A2A):

$$V_{DDA,\text{actual}} = 3.3\ \text{V} \times \frac{\text{*VREFINT_CAL_ADDR}}{\text{Code}_{V_{\text{REFINT}}}}$$

5.3.2 Internal Temperature Sensor

STM32 microcontrollers feature an internal silicon temperature sensor connected to an ADC channel (Channel 16 on STM32F1). It measures the internal junction temperature (T_J) of the microcontroller die.

Temperature Calculation Formula (STM32F103)

From the STM32F103 datasheet: - $V_{25} = 1.43\ \text{V}$ (Nominal voltage at $25°\text{C}$) - $\text{Avg_Slope} = 4.3\ \text{mV/°C} = 0.0043\ \text{V/°C}$

$$\text{Temperature (°C)} = \frac{V_{25} - V_{\text{SENSE}}}{\text{Avg_Slope}} + 25$$

Where $V_{\text{SENSE}} = \text{Code} \times \frac{V_{DDA}}{4095}$.

5.4 ADC Acquisition Modes in STM32 HAL

Table 5.2 — Comparison of ADC acquisition modes

Mode	Trigger Source	CPU Involvement	Latency & Jitter	Best Used For
Polled Software	HAL_ADC_Start()	100% blocked inside HAL_ADC_PollForConversion()	High jitter; unpredictable	Power-on sanity checks; DC battery monitor
Interrupt-Driven	Software or Timer	Low (ISR fires on End of Conversion EOC)	Low jitter; high ISR overhead at $>$ $10\ \text{kSPS}$	Low-rate threshold detection (Analog Watchdog)
Timer-Triggered DMA	Timer TRGO Event	**0% during sampling** (Wakes only at Half/Full buffer)	**Zero software jitter** (Hardware routed)	Continuous multi-channel data acquisition, DSP, audio

5.4.1 Mandatory Calibration Step

All STM32 SAR ADCs exhibit internal capacitor mismatch and voltage offsets resulting from silicon manufacturing tolerances. **Before enabling the ADC, you must run internal calibration**.

```
/* Mandatory ADC calibration in main.c during initialization (STM32F1) */
HAL_ADCEx_Calibration_Start(&hadc1);

/* Start Timer and Circular DMA */
HAL_TIM_Base_Start(&htim3);
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_dma_buffer, BUFFER_SIZE);
```

5.5 Digital Sensor Interrogation: Registers, Burst Reads, and Conversions

Modern robotics systems rely heavily on digital inertial measurement units (IMUs, such as the InvenSense MPU6050 / ICM-20689), digital pressure sensors (Bosch BMP280), and magnetometers (QMC5883L).

5.5.1 The Sensor Initialization Lifecycle

Every robust digital sensor driver must execute a deterministic 5-step bring-up sequence: 1. **Power-On Reset Settling (t_{POR})**: Wait for the sensor's internal power supply regulator and charge pump to stabilize (30 ms to 100 ms). Attempting to send I²C commands immediately upon power-up results in bus NACKs. 2. **Device Identification (WHO_AM_I)**: Read the read-only hardware identity register (e.g., register 0x75 on MPU6050, expected value 0x68). If the returned byte does not match, immediately abort initialization and return DEV_NOT_FOUND. 3. **Power Management and Clock Source**: Take the device out of sleep mode and select a stable oscillator (e.g., PLL locked to gyro X-axis rather than the internal 8 MHz RC oscillator). 4. **Range and Filter Configuration**: Configure Full-Scale Range (e.g., Accelerometer $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$; Gyroscope $\pm 250^\circ/\text{s}$ to $\pm 2000^\circ/\text{s}$) and digital low-pass filter (DLPF) bandwidth. 5. **Interrupt Line Configuration (Optional)**: Configure the Data Ready (DRDY) pin to assert an active-high pulse whenever a new conversion completes, routing this to an STM32 EXTI interrupt pin.

5.5.2 Two's Complement Decoding and Unit Conversion

Digital sensors output physical quantities as **signed 16-bit two's complement integers** split across two 8-bit registers: REG_HIGH (MSB) and REG_LOW (LSB).

Bit Assembly and Sign Extension

A common bug occurs when assembling these bytes using 32-bit integers without proper casting, leading to corrupted negative values:

```
/* CORRECT: Assemble into signed 16-bit type (handles two's complement sign extension) */
int16_t raw_accel_x = (int16_t)(((uint16_t)buf[0] << 8) | buf[1]);

/* WRONG: Buggy assembly on 32-bit platforms */
int32_t wrong_val = (buf[0] << 8) | buf[1]; /* Positive numbers only; negative numbers corrupted */
```

Engineering Unit Conversion (MPU6050 Example)

Once assembled, the raw integer is divided by the manufacturer's **Sensitivity Scale Factor**: - **Accelerometer ($\pm 2g$ Range)**: Scale factor = $16{,}384$ LSB/g.

$$a_x\ [g] = \frac{\text{raw_accel_x}}{16384.0f}$$

$$a_x\ [\text{m/s}^2] = a_x\ [g] \times 9.80665\ \text{m/s}^2$$

- **Gyroscope ($\pm 250^\circ/\text{s}$ Range)**: Scale factor = 131.0 LSB/($^\circ$/s).

$$\omega_z\ [^\circ/\text{s}] = \frac{\text{raw_gyro_z}}{131.0f}$$

- **Temperature Sensor**:

$$T\ [^\circ\text{C}] = \frac{\text{raw_temp}}{340.0f} + 36.53f$$

5.6 Real-Time Acquisition Dynamics: Latency, Jitter, and Throughput

In embedded engineering, **"Real-Time" does NOT mean "real-fast"**. It means **deterministic and bounded**. A system that acquires data at 10 Hz with zero microsecond jitter is a true hard real-time system, whereas a system that samples at 100 kHz but occasionally pauses for 20 ms while serving an interrupt is non-real-time and will destabilize a control loop.

5.6.1 Throughput and Bandwidth Mathematics

When architecting high-rate acquisition systems, engineers must calculate the data generation bandwidth:

$$\text{Throughput (Bytes/sec)} = F_s \times \text{Bytes per Sample} \times \text{Number of Channels}$$

Example: Quad-Channel 12-bit ADC at 100 kSPS

- $F_s = 100{,}000$ Hz
- 4 channels, each storing 1 sample as a 16-bit half-word (2 bytes).
- Throughput $= 100{,}000 \times 2 \times 4 = 800{,}000$ bytes/sec $= 800$ kB/s.
- If transmitting this raw data over UART at $115{,}200$ baud (≈ 11.5 kB/s), **the system will instantly overflow**. The data must either be filtered and down-sampled on-chip or streamed over high-speed USB / SPI.

5.6.2 Memory Buffering Strategies

Table 5.3 — Embedded data buffering architectures

Buffer Strategy	Memory Topology	Concurrency Mechanism	Risk / Failure Mode	Best Used For
Single Static Buffer	One array	Mutex or flag	Producer overwrites while consumer reads (Data tearing)	Slow, polled DC sensors
Double / Ping-Pong Buffer	Two separate buffers (A and B)	Producer writes A while consumer reads B; swap upon completion	Buffer overrun if processing time exceeds buffer filling time	High-throughput block processing, DSP, FFT
Circular Ring Buffer	Fixed-size array with head and tail pointers	Atomic pointer arithmetic (modulo)	Queue overflow if consumer stalls	Byte streams, UART RX/TX, event logs

---

5.7 Digital Signal Processing: Anti-Aliasing and Filtering

5.7.1 The Nyquist-Shannon Sampling Theorem and Anti-Aliasing

The Nyquist-Shannon theorem states that to reconstruct a continuous signal of maximum bandwidth $f_{\max}$ without distortion, the sampling rate F_s must satisfy:

$$F_s \geq 2 \times f_{\max}$$

If the analog signal contains high-frequency noise components above the **Nyquist frequency** ($f_{\text{Nyquist}} = \frac{F_s}{2}$)—such as 20 kHz motor PWM switching noise—those frequencies will **fold back (alias)** into the low-frequency spectrum, appearing as fake low-frequency oscillations that cannot be filtered in software! - **Engineering Rule**: Always place a physical **analog RC low-pass filter** on the ADC pin before sampling. Select the cutoff frequency $f_c = \frac{1}{2\pi RC}$ to attenuate noise above $\frac{F_s}{2}$.

5.7.2 Moving Average vs. Exponential Moving Average (EMA)

Once samples are safely digitized, software filters eliminate residual high-frequency sensor noise.

1. Moving Average Filter (Window Size M)

Averages the last M samples. Acts as a linear finite impulse response (FIR) filter with a sinc-like frequency response:

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x\,[n-k]$$

Drawback: Requires maintaining a history ring-buffer of M elements and adds significant phase lag ($T_{\text{lag}} \approx \frac{M-1}{2F_s}$).

2. Exponential Moving Average (EMA / IIR Filter)

An infinite impulse response (IIR) single-pole filter that requires **zero array storage**—only one state variable:

$$y[n] = \alpha \cdot x[n] + (1-\alpha) \cdot y[n-1]$$

Where $\alpha \in (0,1]$ is the smoothing factor. Smaller α provides heavier filtering at the expense of step response latency.

Listing 5.1 — High-efficiency fixed-point integer EMA filter (Zero floating-point overhead)

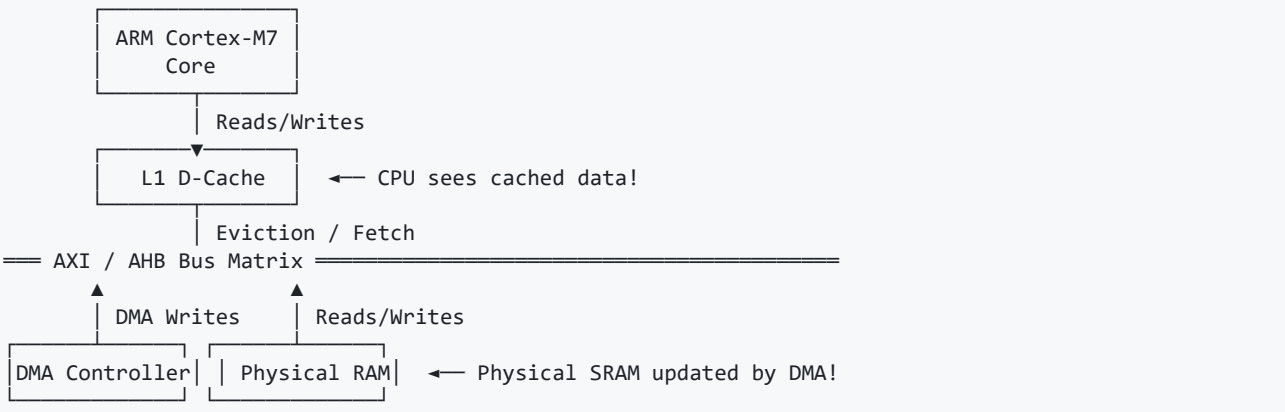
```
/* Fixed-point EMA implementation using Q15 arithmetic (Scale = 32768) */
#define EMA_ALPHA_Q15   3277  /* alpha = 3277 / 32768 ≈ 0.10 */

int32_t EMA_Filter_Q15(int32_t current_sample, int32_t *state_q15)
{
    /* y[n] = y[n-1] + alpha * (x[n] - y[n-1]) */
    int32_t diff = current_sample - (*state_q15 >> 15);
    *state_q15 += (int32_t)(((int64_t)diff * EMA_ALPHA_Q15));
    return *state_q15 >> 15;
}
```

5.8 Cortex-M7 and STM32H7 Data-Cache (D-Cache) Coherency

On high-performance STM32 microcontrollers featuring ARM Cortex-M7 cores (such as the STM32F7 and STM32H7 families running at up to 480–550 MHz), the core includes high-speed **L1 Data Cache (D-Cache)**. The D-Cache transparently accelerates memory reads and writes by keeping local copies of SRAM in 32-byte cache lines.

However, the **DMA controller operates independently of the CPU core** and accesses physical SRAM directly over the internal AXI/AHB bus matrix without passing through the L1 cache. This creates the classic **DMA Cache Incoherency Hazard**.



The Two Incoherency Hazards

1. **DMA Reception Hazard (DMA to Memory)**: The DMA writes fresh sensor samples into physical SRAM. However, the CPU's D-Cache still holds the old data for that address range. When application code reads the buffer, it reads the stale cache line, completely missing the new sensor data!
2. **DMA Transmission Hazard (Memory to Peripheral)**: The CPU writes telemetry data into an SRAM array. Because D-Cache uses a **write-back policy**, the data remains sitting in the CPU's internal cache and has not yet been flushed to physical SRAM. When the DMA starts transmitting, it reads stale garbage from physical RAM.

The Three Architectural Solutions

Solution 1: Cache Maintenance Operations (Clean and Invalidate)

Use CMSIS cache control intrinsics before and after every DMA operation: - **Before DMA Transmit**: Clean the cache (flush dirty cache lines out to physical RAM): c /* Ensure physical RAM holds the latest data written by the CPU */ SCB_CleanDCache_by_Addr((uint32_t *)tx_buf, TX_BUF_SIZE); HAL_UART_Transmit_DMA(&huart1, tx_buf, TX_BUF_SIZE); - **After DMA Receive**: Invalidate the cache (force CPU to discard cache lines and reload from physical RAM): c void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) { /* Invalidate CPU cache so core reads fresh data placed by DMA */ SCB_InvalidateDCache_by_Addr((uint32_t *)rx_buf, RX_BUF_SIZE); App_ProcessPacket(rx_buf, RX_BUF_SIZE); }

COMMON MISTAKE / CAUTION: The 32-Byte Cache Line Alignment Rule: Cache operations operate strictly on 32-byte cache lines. If a DMA buffer is not aligned to a 32-byte boundary, or if its length is not a multiple of 32 bytes, invalidating the buffer will unintentionally destroy adjacent CPU variables allocated in the same cache line! Always declare DMA buffers on Cortex-M7 with 32-byte alignment:

```
uint8_t rx_buf[256] __attribute__((aligned(32)));
```

Solution 2: Non-Cacheable Memory Regions via MPU (Memory Protection Unit)

The most robust solution is to configure the Cortex-M7 MPU to designate a dedicated SRAM section (such as SRAM4 or D2 domain RAM) as **Non-Cacheable (Shareable Device Memory)**. All DMA buffers are placed inside this region using linker attributes:

```
uint8_t dma_buffer[1024] __attribute__((section(".dma_buffer_section")));
```

Because the CPU never caches this memory region, cache coherency hazards are completely eliminated by hardware design.

6. Part 5 — Sensor Drivers and Reusable Embedded Software

In amateur embedded code, peripheral registers, I²C transactions, sensor-specific math, and application logic are frequently mixed inside main.c. While this approach may function for a one-off laboratory demonstration, it is fatal in commercial engineering and robotics. When the microcontroller family changes, when a second identical sensor is added, or when the underlying bus changes from I²C to SPI, tightly coupled code must be completely rewritten.

Professional embedded engineering treats sensor code as a **modular, reusable device driver**. A well-architected sensor driver forms an independent software component that isolates physical bus mechanics from business logic, enforces explicit error handling, and presents a clean, hardware-agnostic API in SI engineering units.

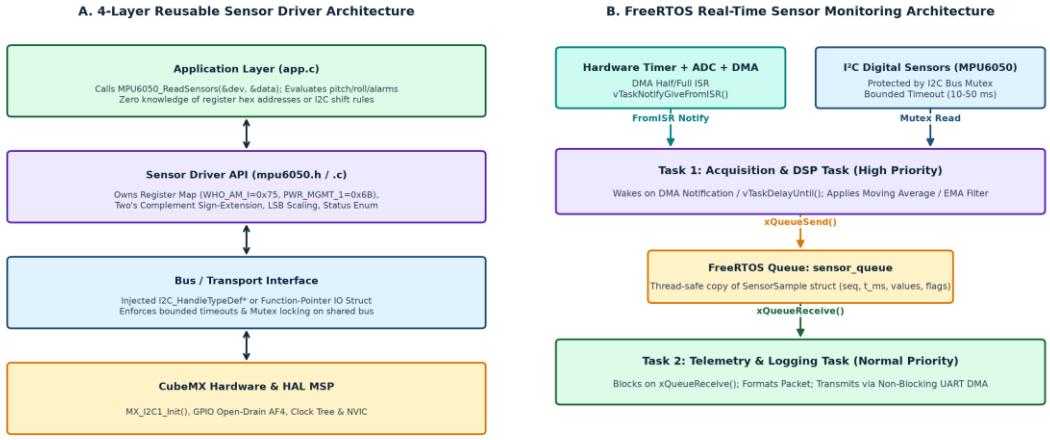


Figure 6.1 — Software architecture: (A) Four-layer reusable driver hierarchy showing complete decoupling of hardware bus from application logic; (B) FreeRTOS dual-task telemetry architecture with thread-safe queue buffering and mutex protection.

6.1 What is a Sensor Driver and Why Decouple It?

A **sensor driver** is a dedicated software abstraction layer responsible for managing the complete lifecycle of a physical sensor: 1. Validating hardware presence via identification registers (WHO_AM_I). 2. Configuring internal operational modes, sampling rates, full-scale ranges, and filters. 3. Executing bus transactions (I²C, SPI, or UART) to retrieve raw data registers. 4. Assembling raw binary words and converting them into calibrated, floating-point physical quantities in SI units (m/s^2, rad/s, °C, Pa). 5. Detecting and reporting hardware bus faults, timeouts, and out-of-range sensor readings.

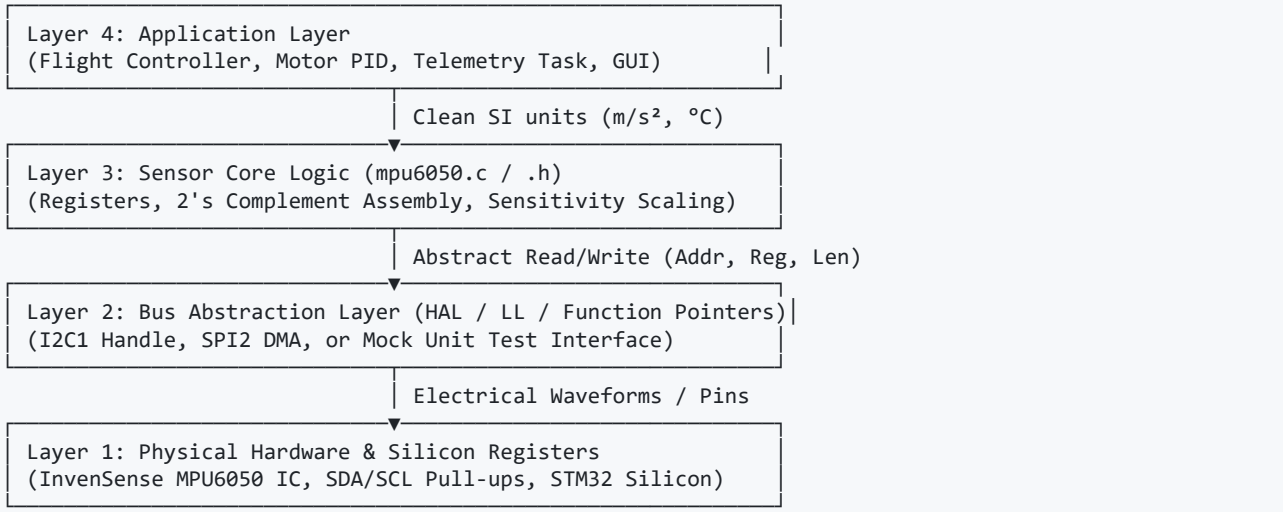
The Dangers of Tightly Coupled Code

When developers write raw HAL_I2C_Mem_Read() calls directly inside their main application loop: - **Zero Portability**: Migrating from an STM32F103 to an STM32G474 or STM32H7 requires rewriting all register accesses across the codebase. - **Hidden Dependencies**: Application logic becomes infected with low-level

details (such as whether an MPU6050 address requires a 1-bit left shift). - **Inability to Test**: The control algorithms (e.g., drone attitude estimation or PID motor loops) cannot be tested or simulated on a PC without physical hardware connected.

6.2 The Four-Layer Driver Architecture

To achieve commercial-grade reusability, embedded software architects structure sensor drivers into four distinct, non-overlapping layers:



1. **Layer 1 (Physical Hardware)**: The silicon sensor IC, printed circuit board traces, external pull-up resistors, and microcontroller pins.
2. **Layer 2 (Bus Abstraction Layer)**: Adapts generic read/write calls to the specific communication hardware. On an STM32, this calls HAL or LL functions; on a desktop unit test runner, this calls simulated memory arrays.
3. **Layer 3 (Sensor Core Logic)**: Encapsulates the sensor's register map, state machine, initialization sequence, sensitivity constants, and calibration offsets. **This layer contains zero references to specific STM32 hardware registers or peripheral handles**.
4. **Layer 4 (Application Layer)**: Consumes clean, validated engineering units to execute control laws, robotic navigation, or user-interface displays.

6.3 The 10 Golden Rules of Embedded Driver Design

Every production sensor driver should adhere to these 10 architectural principles:

Table 6.1 — The 10 golden rules of embedded driver architecture

Rule	Principle	Engineering Justification

Rule	Principle	Engineering Justification
Rule 1	**Never hardcode global peripheral handles**	Passing &hi2c1 directly inside the driver prevents using hi2c2 or instantiating two identical sensors on separate buses.
Rule 2	**Encapsulate all state in a handle struct**	A driver must be completely re-entrant. All configuration, calibration, and bus handles belong in an instance structure (mpu6050_t).
Rule 3	**Return explicit status codes everywhere**	Functions must return an enumerated status (STATUS_OK, STATUS_ERR_TIMEOUT, STATUS_ERR_BUS), never void or raw data directly.
Rule 4	**Enforce finite bus timeouts**	Never execute an unbounded while loop or pass HAL_MAX_DELAY. A disconnected sensor must not hang the operating system.
Rule 5	**Verify hardware identity (WHO_AM_I)**	Always confirm the physical chip responds with its expected ID before attempting to write configuration registers.
Rule 6	**Output calibrated SI engineering units**	Application code should consume accel_mps2 and gyro_dps, never unscaled raw integers like 0x0E2A.
Rule 7	**Assemble bytes portably and safely**	Use explicit fixed-width integers (int16_t, uint8_t) and signed bit-shifts to prevent undefined behavior across compilers.
Rule 8	**Separate init from continuous read**	Never execute sensor mode changes, clock selection, or DLPF configuration inside the periodic high-rate sampling loop.
Rule 9	**Zero UI or console logging inside driver**	Never call printf() or display routines from within a driver. Telemetry formatting belongs strictly in the application layer.
Rule 10	**Decouple via Bus Interface Callbacks**	Abstract physical I^2C/SPI operations behind function pointers to enable automated hardware-in-the-loop (HIL) testing and PC mocking.

6.4 Professional Driver Implementation: InvenSense MPU6050

The following complete, production-grade driver demonstrates these architectural principles. It is family-agnostic, multi-instance capable, unit-testable, and provides robust error handling.

6.4.1 Public Header File: mpu6050.h

Listing 6.1 — Production header file mpu6050.h

```
#ifndef MPU6050_H
#define MPU6050_H

#include <stdint.h>
#include <stdbool.h>

#ifdef __cplusplus
extern "C" {
#endif

/* Typed error codes */
typedef enum {
    MPU6050_OK              = 0,
    MPU6050_ERR_PARAM       = -1,
    MPU6050_ERR_BUS         = -2,
    MPU6050_ERR_TIMEOUT     = -3,
    MPU6050_ERR_NOT_FOUND   = -4
} mpu6050_status_t;

/* Accelerometer Full Scale Range */
typedef enum {
    MPU6050_ACCEL_2G        = 0, /* ±2g  (16384 LSB/g) */
    MPU6050_ACCEL_4G        = 1, /* ±4g  (8192 LSB/g)  */
    MPU6050_ACCEL_8G        = 2, /* ±8g  (4096 LSB/g)  */
    MPU6050_ACCEL_16G       = 3  /* ±16g (2048 LSB/g)  */
} mpu6050_accel_fs_t;

/* Gyroscope Full Scale Range */
typedef enum {
    MPU6050_GYRO_250DPS     = 0, /* ±250 °/s  (131 LSB/°/s)  */
    MPU6050_GYRO_500DPS     = 1, /* ±500 °/s  (65.5 LSB/°/s) */
    MPU6050_GYRO_1000DPS    = 2, /* ±1000 °/s (32.8 LSB/°/s) */
    MPU6050_GYRO_2000DPS    = 3  /* ±2000 °/s (16.4 LSB/°/s) */
} mpu6050_gyro_fs_t;

/* Engineering Units Data Structure */
typedef struct {
    float accel_x_mps2; /* X acceleration in m/s² */
    float accel_y_mps2; /* Y acceleration in m/s² */
    float accel_z_mps2; /* Z acceleration in m/s² */
    float gyro_x_dps;   /* X angular rate in degrees/sec */
    float gyro_y_dps;   /* Y angular rate in degrees/sec */
    float gyro_z_dps;   /* Z angular rate in degrees/sec */
    float temp_c;       /* Die temperature in °C */
} mpu6050_data_t;

/* Abstract Bus Interface Function Pointers */
typedef int8_t (*mpu6050_write_fn)(void *intf_ptr, uint8_t dev_addr,
                                   uint8_t reg_addr, const uint8_t *data, uint16_t len);
typedef int8_t (*mpu6050_read_fn)(void *intf_ptr, uint8_t dev_addr,
                                  uint8_t reg_addr, uint8_t *data, uint16_t len);
typedef void (*mpu6050_delay_fn)(uint32_t ms);

/* Driver Handle Structure (Encapsulates all instance state) */
```

```
typedef struct {
    void               *intf_ptr;        /* Opaque pointer to HAL handle (e.g. &hi2c1) */
    uint8_t             dev_addr;        /* 7-bit I2C address (0x68 or 0x69) */
    mpu6050_write_fn    write_reg;       /* Low-level bus write pointer */
    mpu6050_read_fn     read_reg;        /* Low-level bus read pointer */
    mpu6050_delay_fn    delay_ms;        /* Delay hook */
    float               accel_scale;     /* Scale factor to convert LSB to m/s² */
    float               gyro_scale;      /* Scale factor to convert LSB to deg/s */
    float               accel_bias[3];   /* X, Y, Z zero-g offset calibration */
    float               gyro_bias[3];    /* X, Y, Z zero-rate offset calibration */
    bool                is_initialized;  /* State guard */
} mpu6050_t;

/* Public API */
mpu6050_status_t MPU6050_Init(mpu6050_t *dev, void *intf_ptr, uint8_t dev_addr,
                              mpu6050_write_fn write_fn, mpu6050_read_fn read_fn,
                              mpu6050_delay_fn delay_fn);
mpu6050_status_t MPU6050_ConfigRanges(mpu6050_t *dev, mpu6050_accel_fs_t accel_fs,
                                      mpu6050_gyro_fs_t gyro_fs);
mpu6050_status_t MPU6050_CalibrateOffsets(mpu6050_t *dev, uint16_t sample_count);
mpu6050_status_t MPU6050_ReadSensors(mpu6050_t *dev, mpu6050_data_t *data);

#ifdef __cplusplus
}
#endif

#endif /* MPU6050_H */
```

6.4.2 Driver Implementation: mpu6050.c

Listing 6.2 — Production source file mpu6050.c

```
#include "mpu6050.h"

/* Internal MPU6050 Register Map */
#define REG_SMPLRT_DIV      0x19
#define REG_CONFIG          0x1A
#define REG_GYRO_CONFIG     0x1B
#define REG_ACCEL_CONFIG    0x1C
#define REG_ACCEL_XOUT_H    0x3B
#define REG_TEMP_OUT_H      0x41
#define REG_GYRO_XOUT_H     0x43
#define REG_PWR_MGMT_1      0x6B
#define REG_WHO_AM_I        0x75

#define EXPECTED_WHO_AM_I   0x68
#define GRAVITY_MPS2        9.80665f

mpu6050_status_t MPU6050_Init(mpu6050_t *dev, void *intf_ptr, uint8_t dev_addr,
                              mpu6050_write_fn write_fn, mpu6050_read_fn read_fn,
                              mpu6050_delay_fn delay_fn)
{
    uint8_t who_am_i = 0;

    if (!dev || !write_fn || !read_fn || !delay_fn) {
        return MPU6050_ERR_PARAM;
    }

    dev->intf_ptr       = intf_ptr;
    dev->dev_addr       = dev_addr;
    dev->write_reg      = write_fn;
    dev->read_reg       = read_fn;
    dev->delay_ms       = delay_fn;
```

```
    dev->accel_scale    = GRAVITY_MPS2 / 16384.0f; /* Default ±2g */
    dev->gyro_scale     = 1.0f / 131.0f;           /* Default ±250 °/s */
    dev->is_initialized = false;

    for (int i = 0; i < 3; i++) {
        dev->accel_bias[i] = 0.0f;
        dev->gyro_bias[i]  = 0.0f;
    }

    /* 1. Wait for sensor internal power-on reset */
    dev->delay_ms(50);

    /* 2. Validate device presence */
    if (dev->read_reg(dev->intf_ptr, dev->dev_addr, REG_WHO_AM_I, &who_am_i, 1) != 0) {
        return MPU6050_ERR_BUS;
    }
    if (who_am_i != EXPECTED_WHO_AM_I) {
        return MPU6050_ERR_NOT_FOUND;
    }

    /* 3. Wake up device: Reset sleep bit, select Auto-PLL clock */
    uint8_t pwr_mgmt = 0x01; /* Clock Source = PLL with X-axis Gyroscope */
    if (dev->write_reg(dev->intf_ptr, dev->dev_addr, REG_PWR_MGMT_1, &pwr_mgmt, 1) != 0) {
        return MPU6050_ERR_BUS;
    }
    dev->delay_ms(10);

    /* 4. Configure default ranges (±2g, ±250 deg/s) */
    mpu6050_status_t status = MPU6050_ConfigRanges(dev, MPU6050_ACCEL_2G, MPU6050_GYRO_250DPS);
    if (status != MPU6050_OK) {
        return status;
    }

    dev->is_initialized = true;
    return MPU6050_OK;
}

mpu6050_status_t MPU6050_ConfigRanges(mpu6050_t *dev, mpu6050_accel_fs_t accel_fs,
                                      mpu6050_gyro_fs_t gyro_fs)
{
    uint8_t reg_val;

    if (!dev) return MPU6050_ERR_PARAM;

    /* Configure Accel Full-Scale */
    reg_val = (uint8_t)(accel_fs << 3);
    if (dev->write_reg(dev->intf_ptr, dev->dev_addr, REG_ACCEL_CONFIG, &reg_val, 1) != 0) {
        return MPU6050_ERR_BUS;
    }

    switch (accel_fs) {
        case MPU6050_ACCEL_2G:  dev->accel_scale = GRAVITY_MPS2 / 16384.0f; break;
        case MPU6050_ACCEL_4G:  dev->accel_scale = GRAVITY_MPS2 / 8192.0f;  break;
        case MPU6050_ACCEL_8G:  dev->accel_scale = GRAVITY_MPS2 / 4096.0f;  break;
        case MPU6050_ACCEL_16G: dev->accel_scale = GRAVITY_MPS2 / 2048.0f;  break;
    }

    /* Configure Gyro Full-Scale */
    reg_val = (uint8_t)(gyro_fs << 3);
    if (dev->write_reg(dev->intf_ptr, dev->dev_addr, REG_GYRO_CONFIG, &reg_val, 1) != 0) {
        return MPU6050_ERR_BUS;
    }

    switch (gyro_fs) {
```

```
        case MPU6050_GYRO_250DPS:  dev->gyro_scale = 1.0f / 131.0f; break;
        case MPU6050_GYRO_500DPS:  dev->gyro_scale = 1.0f / 65.5f;  break;
        case MPU6050_GYRO_1000DPS: dev->gyro_scale = 1.0f / 32.8f;  break;
        case MPU6050_GYRO_2000DPS: dev->gyro_scale = 1.0f / 16.4f;  break;
    }

    return MPU6050_OK;
}

mpu6050_status_t MPU6050_ReadSensors(mpu6050_t *dev, mpu6050_data_t *data)
{
    uint8_t raw_buf[14];

    if (!dev || !data || !dev->is_initialized) {
        return MPU6050_ERR_PARAM;
    }

    /* Single continuous 14-byte burst read across all sensor registers */
    if (dev->read_reg(dev->intf_ptr, dev->dev_addr, REG_ACCEL_XOUT_H, raw_buf, 14) != 0) {
        return MPU6050_ERR_BUS;
    }

    /* Assemble signed 16-bit integers */
    int16_t raw_ax = (int16_t)(((uint16_t)raw_buf[0]  << 8) | raw_buf[1]);
    int16_t raw_ay = (int16_t)(((uint16_t)raw_buf[2]  << 8) | raw_buf[3]);
    int16_t raw_az = (int16_t)(((uint16_t)raw_buf[4]  << 8) | raw_buf[5]);
    int16_t raw_t  = (int16_t)(((uint16_t)raw_buf[6]  << 8) | raw_buf[7]);
    int16_t raw_gx = (int16_t)(((uint16_t)raw_buf[8]  << 8) | raw_buf[9]);
    int16_t raw_gy = (int16_t)(((uint16_t)raw_buf[10] << 8) | raw_buf[11]);
    int16_t raw_gz = (int16_t)(((uint16_t)raw_buf[12] << 8) | raw_buf[13]);

    /* Convert to physical SI units and subtract calibrated zero-bias */
    data->accel_x_mps2 = ((float)raw_ax * dev->accel_scale) - dev->accel_bias[0];
    data->accel_y_mps2 = ((float)raw_ay * dev->accel_scale) - dev->accel_bias[1];
    data->accel_z_mps2 = ((float)raw_az * dev->accel_scale) - dev->accel_bias[2];

    data->temp_c       = ((float)raw_t / 340.0f) + 36.53f;

    data->gyro_x_dps   = ((float)raw_gx * dev->gyro_scale) - dev->gyro_bias[0];
    data->gyro_y_dps   = ((float)raw_gy * dev->gyro_scale) - dev->gyro_bias[1];
    data->gyro_z_dps   = ((float)raw_gz * dev->gyro_scale) - dev->gyro_bias[2];

    return MPU6050_OK;
}
```

6.4.3 Application Usage and HAL Binding

Notice how cleanly the driver binds to STM32 HAL inside main.c without contaminating the driver source with STM32 headers.

Listing 6.3 — Application integration in main.c

```
#include "main.h"
#include "mpu6050.h"

extern I2C_HandleTypeDef hi2c1;
mpu6050_t imu;

/* HAL I2C Adapter Callbacks */
static int8_t STM32_I2C_Write(void *intf_ptr, uint8_t dev_addr, uint8_t reg_addr,
                              const uint8_t *data, uint16_t len)
{
```

```
    I2C_HandleTypeDef *hi2c = (I2C_HandleTypeDef *)intf_ptr;
    /* Pass pre-shifted HAL address: (dev_addr << 1) */
    HAL_StatusTypeDef st = HAL_I2C_Mem_Write(hi2c, (uint16_t)(dev_addr << 1), reg_addr,
                                             I2C_MEMADD_SIZE_8BIT, (uint8_t *)data, len, 100);
    return (st == HAL_OK) ? 0 : -1;
}

static int8_t STM32_I2C_Read(void *intf_ptr, uint8_t dev_addr, uint8_t reg_addr,
                             uint8_t *data, uint16_t len)
{
    I2C_HandleTypeDef *hi2c = (I2C_HandleTypeDef *)intf_ptr;
    HAL_StatusTypeDef st = HAL_I2C_Mem_Read(hi2c, (uint16_t)(dev_addr << 1), reg_addr,
                                            I2C_MEMADD_SIZE_8BIT, data, len, 100);
    return (st == HAL_OK) ? 0 : -1;
}

void App_Init(void)
{
    mpu6050_status_t status = MPU6050_Init(&imu, &hi2c1, 0x68,
                                           STM32_I2C_Write, STM32_I2C_Read, HAL_Delay);
    if (status != MPU6050_OK) {
        /* Error handling: Illuminate red fault LED and log error code */
        Error_Handler();
    }
}

void App_Loop(void)
{
    mpu6050_data_t sample;
    if (MPU6050_ReadSensors(&imu, &sample) == MPU6050_OK) {
        /* Consume clean SI units */
        ProcessAttitude(sample.accel_x_mps2, sample.accel_y_mps2, sample.accel_z_mps2,
                        sample.gyro_x_dps, sample.gyro_y_dps, sample.gyro_z_dps);
    }
    HAL_Delay(10); /* 100 Hz sampling */
}
```

6.5 Automated Unit Testing and Desktop Mocking

Because mpu6050.c relies exclusively on standard C types and function pointers, **it can be compiled and verified on a desktop computer (Linux/macOS/Windows) with GCC without an STM32 board**.

Listing 6.4 — Desktop PC unit test mock runner

```
#include <stdio.h>
#include <assert.h>
#include <string.h>
#include "mpu6050.h"

/* Mock sensor register memory bank */
static uint8_t mock_registers[256];

static int8_t Mock_I2C_Write(void *intf_ptr, uint8_t dev_addr, uint8_t reg_addr,
                             const uint8_t *data, uint16_t len)
{
    memcpy(&mock_registers[reg_addr], data, len);
    return 0;
}

static int8_t Mock_I2C_Read(void *intf_ptr, uint8_t dev_addr, uint8_t reg_addr,
                            uint8_t *data, uint16_t len)
```

```
{
    memcpy(data, &mock_registers[reg_addr], len);
    return 0;
}

static void Mock_Delay(uint32_t ms) {}

int main(void)
{
    mpu6050_t dev;
    mpu6050_data_t data;

    /* Test Case 1: Unpopulated device should report NOT_FOUND */
    mock_registers[0x75] = 0x00; /* Wrong WHO_AM_I */
    assert(MPU6050_Init(&dev, NULL, 0x68, Mock_I2C_Write, Mock_I2C_Read, Mock_Delay)
           == MPU6050_ERR_NOT_FOUND);

    /* Test Case 2: Valid device initialization */
    mock_registers[0x75] = 0x68; /* Correct WHO_AM_I */
    assert(MPU6050_Init(&dev, NULL, 0x68, Mock_I2C_Write, Mock_I2C_Read, Mock_Delay)
           == MPU6050_OK);

    /* Test Case 3: Verify 1.0g on Z-axis converts accurately */
    /* 1.0g in ±2g mode = +16384 LSB = 0x4000 */
    mock_registers[0x3F] = 0x40; /* ACCEL_ZOUT_H */
    mock_registers[0x40] = 0x00; /* ACCEL_ZOUT_L */

    assert(MPU6050_ReadSensors(&dev, &data) == MPU6050_OK);
    /* Expect 9.80665 m/s² within floating point tolerance */
    assert(data.accel_z_mps2 > 9.80f && data.accel_z_mps2 < 9.82f);

    printf("ALL DRIVER UNIT TESTS PASSED SUCCESSFULLY.\n");
    return 0;
}
```

7. Part 6 — Real-Time Operating Systems (RTOS) and FreeRTOS

As embedded systems expand from simple single-purpose controllers to complex multi-rate devices—integrating motor PID loops, sensor fusion algorithms, wireless communication stacks, file systems, and user interfaces—traditional bare-metal superloops begin to collapse under their own complexity. A blocking HAL_Delay() executed during a sensor read halts the entire processor, delaying critical motor adjustments or dropping incoming UART packets.

A **Real-Time Operating System (RTOS)** introduces a preemptive, priority-based multitasking scheduler that allows developers to decompose complex monolithic firmware into independent, concurrent execution threads (**Tasks**).

This chapter examines the internal mechanics of embedded operating systems, focusing on **FreeRTOS** running on ARM Cortex-M microcontrollers. We analyze context-switching physics, scheduling algorithms, thread synchronization primitives, interrupt integration boundaries, and critical HAL timebase configuration rules.

7.1 Bare-Metal Superloops vs. RTOS: When is an RTOS Justified?

An RTOS is not a status symbol or an automatic badge of engineering quality. It is a tool for managing complex execution timing.

7.1.1 The Anatomy of a Bare-Metal Superloop

In a traditional bare-metal architecture, firmware executes within an infinite loop:

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    App_Init();

    while (1) {
        App_ProcessCommands();     /* Parses CLI packets */
        App_ReadSensors();         /* Samples I2C/SPI sensors */
        App_UpdateControlLoop();   /* Computes motor PID */
        App_SendTelemetry();       /* Streams UART data */
    }
}
```

The fundamental limitation of the superloop is that **all tasks share a single call stack and run sequentially**: - If App_ReadSensors() takes $5\ \text{ms}$ to complete an I^2C read, App_UpdateControlLoop() is delayed by at least $5\ \text{ms}$. - If someone inserts a blocking HAL_Delay(100) inside a state machine, the entire system freezes for $100\ \text{ms}$. - Meeting strict deadlines requires restructuring the code into cooperative non-blocking state machines using HAL_GetTick() timestamps. Once a project exceeds 4 or 5 independent timing requirements, state-machine code becomes unmaintainable.

7.1.2 The RTOS Justification Matrix

Table 7.1 — Superloop vs. RTOS selection criteria

Architecture	Fits When	Breaks Down When
Bare-Metal Superloop	• Simple, coupled activities with harmonic rates (e.g. 100 Hz and 10 Hz).• Memory is severely constrained (< 4 kB RAM).• Execution is purely event-driven using hardware interrupts.• Functions are non-blocking and have bounded Worst-Case Execution Time (WCET).	• Multiple asynchronous, independent deadlines coexist.• Third-party stacks (USB, TCP/IP, FATFS) require dedicated blocking loops.• Developers insert blocking delays inside main logic.• Adding a new feature breaks the timing of existing features.
Real-Time OS (FreeRTOS)	• Independent activities require strict priority-based preemption.• Background logging or telemetry must not starve a 1 kHz motor control loop.• Reusable software components communicate via structured queues.• System uses complex middleware (USB host, cellular modem, filesystem).	• Memory is too small to afford private stacks per task.• Developers believe an RTOS "makes slow code run faster".• Engineers create dozens of tasks without budgeting RAM.• Developers use HAL_Delay() inside high-priority tasks.

Table 7.2 — Good vs. poor reasons for adopting an RTOS

Valid Engineering Justifications	Flawed / Dangerous Misconceptions
Independent deadlines where a high-priority task must preempt long background transfers	*"Professionals always use an RTOS, so we should too."*
Integrating complex blocking middleware (e.g., USB device stack or LwIP Ethernet)	*"We will use an RTOS so we don't have to write non-blocking state machines."*
Clean separation of concerns with thread-safe inter-task queues	*"An RTOS will make our slow 100 kHz I^2C read execute faster."*
Deterministic rate enforcement using vTaskDelayUntil()	*"We will create 25 tasks for every single function in the project."*

7.2 Core RTOS Architecture and Mechanics

7.2.1 Tasks, Stacks, and the Task Control Block (TCB)

In FreeRTOS, a **Task** is an independent thread of execution represented as an infinite loop that never returns. Each task possesses: 1. **A Function Pointer**: The entry point of the task code. 2. **A Priority**: An integer from 0 (Idle task priority) up to configMAX_PRIORITIES - 1. 3. **A Private Stack**: A dedicated block of SRAM allocated from the FreeRTOS heap. The task stores its local variables, function call frames, and saved CPU registers here. 4. **A Task Control Block (TCB)**: A system data structure that records the task's state, current top of stack pointer (pxTopOfStack), priority, and list links.

```
      SRAM Allocation per Task
┌──────────────────────────────────────┐
│ Task Control Block (TCB)             │
│ • pxTopOfStack                       │
│ • uxPriority                         │
│ • pcTaskName                         │
├──────────────────────────────────────┤
│ Task Private Stack                   │
│ (Local variables, nested calls,      │
│  and saved CPU registers R0-R15)     │
│                   ▲                  │
│                   | Grows downwards  │
└──────────────────────────────────────┘
```

7.2.2 Preemptive Scheduling and Rate Monotonic Intuition

FreeRTOS implements a **Preemptive Priority-Based Scheduler with Round-Robin Time-Slicing**: - **Strict Preemption**: The scheduler guarantees that the **highest-priority task in the READY state is ALWAYS running on the CPU**. - If a low-priority telemetry task is executing and a high-priority motor task transitions from BLOCKED to READY (e.g., because a timer expired or an interrupt posted data), the scheduler **immediately preempts** the low-priority task. - Tasks with equal priority share the CPU equally via Round-Robin time-slicing at every **SysTick tick interrupt** (typically configured to $1000\ \text{Hz} \Longrightarrow 1\ \text{ms}$ tick).

> **COMMON MISTAKE / CAUTION:** Priority Starvation Hazard: In a preemptive system, if a high-priority task enters an infinite computation loop without calling a blocking FreeRTOS API (vTaskDelay, xQueueReceive, xSemaphoreTake), no lower-priority task will ever execute. Even the FreeRTOS Idle Task (which frees dynamic memory and services power-saving modes) will starve!

Rate Monotonic Scheduling (RMS) Guideline

When assigning task priorities: **Tasks with shorter execution periods / tighter deadlines should be assigned higher numerical priorities**. - Priority 4: Fast Motor PID / Inverter Control ($1000\ \text{Hz}$, $1\ \text{ms}$ period) - Priority 3: IMU Sensor Reading & Attitude Filter ($200\ \text{Hz}$, $5\ \text{ms}$ period) - Priority 2: Battery & Temperature Monitor ($1\ \text{Hz}$, $1000\ \text{ms}$ period) - Priority 1: UART / Wireless Telemetry Stream ($10\ \text{Hz}$, $100\ \text{ms}$ period)

7.2.3 Context Switching on ARM Cortex-M: PendSV and SysTick

The transition from one task to another is called a **Context Switch**. On ARM Cortex-M processors (Cortex-M0/M3/M4/M7), this is executed using the **PendSV (Pended Servicing Call)** exception.

```
1. SysTick Interrupt fires (1 ms)
2. Scheduler detects Task A (Priority 2) must yield to Task B (Priority 3)
3. Scheduler sets PendSV interrupt pending bit in ICSR
4. Active ISR concludes; CPU immediately enters PendSV_Handler (lowest priority)
5. PendSV saves Task A's remaining registers (R4-R11) to Task A's stack
6. PendSV switches Process Stack Pointer (PSP) to Task B's stack
7. PendSV restores Task B's registers (R4-R11)
8. PendSV executes Exception Return (0xFFFFFFFD); Hardware pops R0-R3, R12, LR, PC
9. Task B resumes execution exactly where it was preempted!
```

Hardware Auto-Stacking vs. Software Stacking

The ARM Cortex-M architecture splits context saving into two phases: 1. **Hardware Automatic Stacking**: Upon entering any exception, the hardware automatically pushes 8 registers onto the current stack (**R0, R1, R2, R3, R12, LR, PC, and xPSR**). 2. **Software Stacking inside PendSV**: The FreeRTOS assembly handler pushes the

remaining 8 core registers (**R4 through R11**) onto the task stack, saves the stack pointer into the task's TCB, loads the new task's stack pointer, pops its R4–R11 registers, and returns.

7.3 Inter-Task Communication and Synchronization Primitives

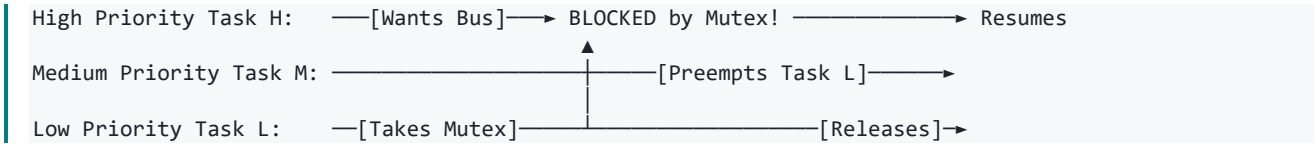
Tasks must exchange data and synchronize actions safely without race conditions.

Table 7.3 — FreeRTOS synchronization and communication primitives

Primitive	Operating Mechanism	Typical Use Case	ISR Safe?
Queue	FIFO buffer; copies data **by value** into queue storage	Passing sensor samples from producer to consumer	Yes (FromISR variant)
Binary Semaphore	Flag with state 0 or 1; unblocks a waiter	Task-to-task or ISR-to-task event signaling	Yes (FromISR variant)
Counting Semaphore	Integer token counter (0 to N)	Managing a pool of identical resources; event counting	Yes (FromISR variant)
Mutex	Mutual exclusion lock with **Priority Inheritance**	Protecting a shared hardware bus (e.g. single I²C bus)	**NO! Never call from an ISR**
Task Notification	Direct 32-bit value updated inside target task's TCB	Lightweight, zero-RAM event signaling (45% faster)	Yes (FromISR variant)
Event Group	Bitmask of flags; tasks wait for pattern (AND/OR)	Synchronizing complex multi-stage boot sequences	Yes (FromISR variant)

7.3.1 Priority Inversion and Priority Inheritance in Mutexes

The **Priority Inversion Problem** is a classic real-time failure mode that famously affected the NASA Mars Pathfinder spacecraft:



1. Low-priority Task L acquires a mutex protecting a shared I²C bus.
2. High-priority Task H wakes up and attempts to take the mutex. Seeing it locked, Task H blocks.
3. Medium-priority Task M (which does not even use the I²C bus) wakes up. Because Task M has higher priority than Task L, **Task M preempts Task L**!
4. Task L cannot finish its transaction to release the mutex.
5. As a result, **the highest-priority Task H is indirectly blocked by the medium-priority Task M for an unbounded duration**!

The Solution: Priority Inheritance

FreeRTOS **Mutexes** (unlike basic binary semaphores) feature automatic **Priority Inheritance**: - When Task H blocks waiting for a mutex held by Task L, the scheduler **temporarily elevates Task L's priority to match Task H**. - Medium-priority Task M can no longer preempt Task L. - Task L quickly finishes its bus transaction, releases the mutex, and is immediately demoted back to its low priority, allowing Task H to proceed without delay.

7.4 Interrupts and FreeRTOS: The Critical Priority Rule

Interfacing hardware interrupts with FreeRTOS is the most common source of HardFaults and system freezes among STM32 developers.

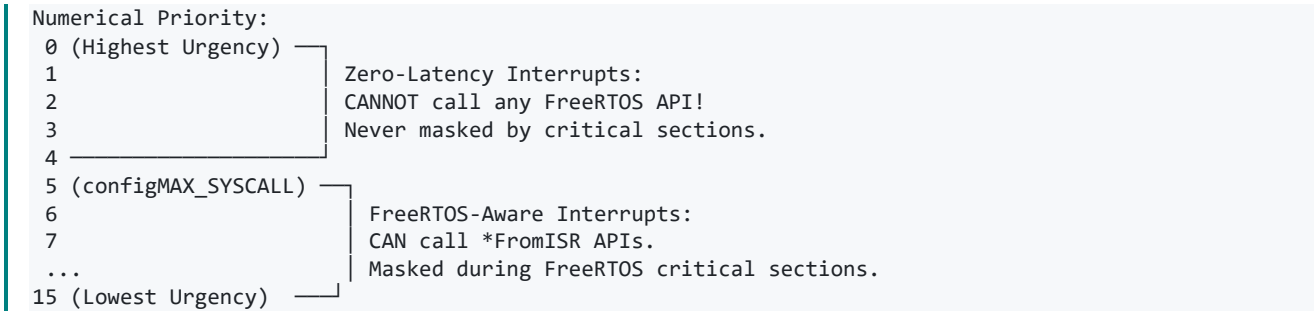
7.4.1 Strict ISR API Rules

1. **Never call standard FreeRTOS APIs inside an ISR**. Always use the FromISR variants (e.g., xQueueSendFromISR(), xSemaphoreGiveFromISR()).
2. **Never attempt to take a Mutex inside an ISR**. Mutexes can block; an interrupt service routine cannot block.
3. **Never call vTaskDelay() inside an ISR**.

7.4.2 The configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY Rule

ARM Cortex-M NVIC allows assigning numerical priorities to interrupts, where 0 **is the highest physical urgency and** 15 **is the lowest** (on 4-bit priority implementations).

In FreeRTOSConfig.h, two vital priority thresholds are configured: - configLIBRARY_LOWEST_INTERRUPT_PRIORITY: Typically set to 15 (lowest urgency; assigned to SysTick and PendSV). - configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY: Typically set to 5.



COMMON MISTAKE / CAUTION: The FreeRTOS NVIC Priority Law: Any interrupt service routine that calls a FreeRTOS FromISR API MUST have its NVIC priority set numerically equal to or higher than configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY (i.e., a priority value between and).

If an ISR with NVIC priority $0,1,2,3$, or 4 calls `xQueueSendFromISR()`, it corrupts the internal FreeRTOS scheduler lists, triggering an immediate assertion failure in `vPortValidateInterruptPriority()` or an unrecoverable HardFault!

Furthermore, in CubeMX, you **MUST configure NVIC Priority Grouping to `NVIC_PRIORITYGROUP_4`** (4 bits preemption, 0 bits subpriority). Any other grouping violates FreeRTOS priority calculation assumptions.

7.5 Production Example: Thread-Safe Multi-Task Architecture

The following complete, verified FreeRTOS implementation demonstrates a dual-task architecture: 1. **Sensor Task (50 Hz)**: Samples the MPU6050 with fixed period, packs readings into a SensorSample struct, and pushes to a queue. 2. **Telemetry Task (Event-Driven)**: Blocks waiting on the queue and streams ASCII packets over UART. 3. **Shared I²C Bus Protection**: Guarded by a dedicated Mutex.

Listing 7.1 — Production FreeRTOS dual-task telemetry architecture

```
#include "main.h"
#include "cmsis_os.h"
#include "mpu6050.h"
#include <stdio.h>
#include <string.h>

/* Data sample packet passed through the queue */
typedef struct {
    uint32_t timestamp_ms;
    int16_t  accel_x_mg;      /* Fixed-point integer (1000 = 1.0g) */
    int16_t  accel_y_mg;
    int16_t  accel_z_mg;
    int16_t  gyro_z_deg_x10;  /* 125 = 12.5 deg/sec */
} SensorPacket_t;

/* FreeRTOS Handles */
QueueHandle_t     xSensorQueue;
SemaphoreHandle_t xI2CMutex;

/* Task Handles */
TaskHandle_t xSensorTaskHandle;
TaskHandle_t xTelemetryTaskHandle;

extern I2C_HandleTypeDef hi2c1;
extern UART_HandleTypeDef huart2;
extern mpu6050_t imu;

/* ============================================================================
   TASK 1: Sensor Acquisition Task (High Priority: 3, Period: 20 ms / 50 Hz)
   ============================================================================ */
void SensorTask(void *argument)
{
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(20); /* Strict 20 ms period */
    mpu6050_data_t imu_data;
    SensorPacket_t packet;

    for (;;) {
        /* Deterministic periodic execution */
        vTaskDelayUntil(&xLastWakeTime, xPeriod);

        /* Acquire I2C Bus Mutex with 10 ms timeout */
        if (xSemaphoreTake(xI2CMutex, pdMS_TO_TICKS(10)) == pdTRUE) {
            mpu6050_status_t status = MPU6050_ReadSensors(&imu, &imu_data);
            xSemaphoreGive(xI2CMutex); /* Release bus immediately */
```

```
            if (status == MPU6050_OK) {
                packet.timestamp_ms    = HAL_GetTick();
                packet.accel_x_mg      = (int16_t)(imu_data.accel_x_mps2 * 101.97f);
                packet.accel_y_mg      = (int16_t)(imu_data.accel_y_mps2 * 101.97f);
                packet.accel_z_mg      = (int16_t)(imu_data.accel_z_mps2 * 101.97f);
                packet.gyro_z_deg_x10  = (int16_t)(imu_data.gyro_z_dps * 10.0f);

                /* Post to queue with 5 ms timeout; drop sample if queue is full */
                if (xQueueSend(xSensorQueue, &packet, pdMS_TO_TICKS(5)) != pdPASS) {
                    /* Queue overflow policy: drop sample and record diagnostic counter */
                }
            }
        }
    }
}

/* ============================================================================
   TASK 2: Telemetry Stream Task (Medium Priority: 2, Event-Driven)
   ============================================================================ */
void TelemetryTask(void *argument)
{
    SensorPacket_t packet;
    char uart_msg[96];

    for (;;) {
        /* Block indefinitely until a sensor packet arrives in the queue */
        if (xQueueReceive(xSensorQueue, &packet, portMAX_DELAY) == pdTRUE) {
            int len = snprintf(uart_msg, sizeof(uart_msg),
                               "$IMU,%lu,%d,%d,%d,%d\r\n",
                               packet.timestamp_ms,
                               packet.accel_x_mg, packet.accel_y_mg, packet.accel_z_mg,
                               packet.gyro_z_deg_x10);

            if (len > 0) {
                HAL_UART_Transmit(&huart2, (uint8_t *)uart_msg, (uint16_t)len, 50);
            }
        }
    }
}

/* ============================================================================
   Application Initialization Hook
   ============================================================================ */
void App_RTOS_Init(void)
{
    /* Create Queue for 16 packets (fixed allocation) */
    xSensorQueue = xQueueCreate(16, sizeof(SensorPacket_t));
    xI2CMutex    = xSemaphoreCreateMutex();

    /* Create Tasks with explicit stack budgets (words, where 1 word = 4 bytes) */
    xTaskCreate(SensorTask, "SensorTask", 256, NULL, 3, &xSensorTaskHandle);
    xTaskCreate(TelemetryTask, "TelemTask", 384, NULL, 2, &xTelemetryTaskHandle);

    /* Start FreeRTOS Scheduler (never returns) */
    vTaskStartScheduler();
}
```

7.6 Critical CubeMX Setup: The HAL Timebase Source Conflict

When FreeRTOS is enabled in an STM32CubeMX project, **both FreeRTOS and the STM32 HAL library require a 1 ms periodic tick interrupt**: - FreeRTOS uses the Cortex-M **SysTick** timer to manage task time-slicing and

unblock delays. - HAL functions (HAL_Delay(), HAL_GetTick(), internal driver timeouts) also depend on a 1 ms tick.

If both systems share SysTick, a severe priority conflict arises: FreeRTOS configures SysTick at the lowest interrupt priority (15) so that hardware ISRs can preempt task switching. If a hardware interrupt handler calls a HAL function with a timeout (such as HAL_I2C_Mem_Read()), the SysTick interrupt cannot preempt the ISR. As a result, HAL_GetTick() stops incrementing, **and the HAL function hangs indefinitely inside an infinite loop!**

The Required CubeMX Configuration

STMicroelectronics officially requires separating the HAL timebase from FreeRTOS: 1. In CubeMX, open the **System Core → SYS** category. 2. Locate the **Timebase Source** dropdown (which defaults to *SysTick*). 3. **Change Timebase Source to an unused general-purpose timer** (e.g., **TIM4** on STM32F103, or **TIM6/TIM7** on STM32F4/G4). 4. CubeMX will automatically generate TIM4_IRQHandler() to drive HAL_IncTick(), leaving SysTick exclusively dedicated to FreeRTOS.

```
CubeMX SYS Configuration:
┌──────────────────────────────────────────────────┐
│ System Core → SYS                                │
│ • Debug: Serial Wire                             │
│ • Timebase Source: TIM4  ◄── DO NOT LEAVE AS SysTick! │
└──────────────────────────────────────────────────┘
```

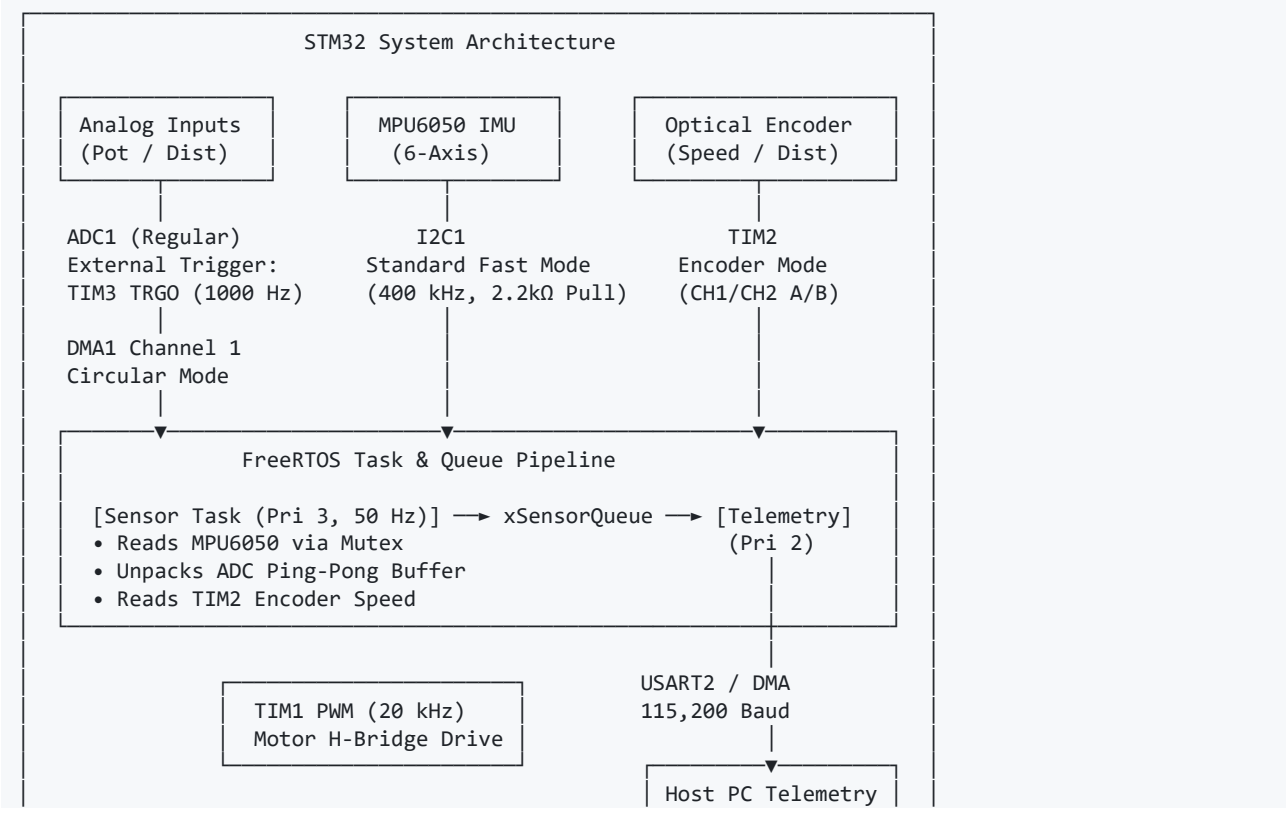
8. Capstone Architecture, Debugging Playbook, and Bench Practice

Real-world embedded systems do not exercise peripherals in isolation. A modern robotic platform concurrently runs timer-triggered analog sampling, digital sensor bus polling, closed-loop PWM actuator control, and serial communications while handling asynchronous external interrupts. When a fault occurs in such an integrated environment, simplistic "trial-and-error" debugging fails.

This chapter synthesizes all previous modules into a cohesive engineering reference: 1. **The Multi-Sensor Robotic Capstone Architecture**: A complete system integrating ADC, DMA, Timers, I^2C, SPI, UART, and FreeRTOS. 2. **The 10-Layer Root-Cause Debugging Playbook**: A structured, physics-to-firmware diagnostic methodology. 3. **HardFault Forensics on ARM Cortex-M**: Assembly-level exception decoding and register analysis. 4. **Per-Subsystem Failure Diagnostic Checklists**: Rapid bench reference tables for every peripheral. 5. **Seven Practical Hands-On Labs and Capstone**: Rigorous lab walkthroughs with quantifiable acceptance criteria. 6. **Master Engineering Formula Sheet**: Bench calculations for quick lookup. 7. **20-Question Technical Knowledge Check**: Diagnostic questions with comprehensive engineering answers.

8.1 The Multi-Sensor Robotic Capstone Architecture

The capstone system represents a high-reliability autonomous robotics node. The microcontroller coordinates five concurrent subsystems:





Architectural Guarantees

1. **Zero-Jitter Analog Conversion**: Analog sampling is triggered exclusively by hardware timer TIM3 TRGO. Software latency or RTOS context switches cannot cause sample jitter.
2. **Deterministic Bus Access**: The I^2C bus is guarded by a recursive Mutex with a finite $10\ \text{ms}$ timeout, preventing priority inversion.
3. **Lossless Telemetry Stream**: Telemetry packets flow through an RTOS queue into an interrupt- or DMA-driven UART transmitter. If the host PC stalls, an explicit queue overflow policy discards the oldest non-critical packet rather than locking the CPU.
4. **Hardware Fail-Safe**: A hardware Independent Watchdog (IWDG) enforces a $500\ \text{ms}$ maximum execution window.

8.2 HardFault Forensics on ARM Cortex-M

A **HardFault** is an ARM Cortex-M exception triggered when the processor encounters an unrecoverable error during instruction execution, memory access, or interrupt servicing.

8.2.1 The Exception Stack Frame

When a HardFault occurs, the Cortex-M processor pushes 8 core registers onto the active stack pointer (**Process Stack Pointer PSP** if faulted in an RTOS task, or **Main Stack Pointer MSP** if faulted in an ISR or bare-metal):

```
Stack Pointer (SP) ──► [ R0   ]  (Offset 0x00)
                       [ R1   ]  (Offset 0x04)
                       [ R2   ]  (Offset 0x08)
                       [ R3   ]  (Offset 0x0C)
                       [ R12  ]  (Offset 0x10)
                       [ LR   ]  (Offset 0x14) - Link Register (Caller function)
                       [ PC   ]  (Offset 0x18) - Program Counter (Crashed instruction!)
                       [ xPSR ]  (Offset 0x1C) - Program Status Register
```

- **Program Counter (PC)**: The exact address of the assembly instruction that caused the crash.
- **Link Register (LR)**: The address of the function that called the failing routine.

8.2.2 Decoding the Fault Status Registers

The System Control Block (SCB) provides hardware status registers that pinpoint the exact cause of the crash.

Table 8.1 — Cortex-M fault status registers and bit definitions

Register	Key Bit Flags	Meaning and Root Cause
CFSR (Configurable Fault Status)	Bits [7:0]: MMFSRBits [15:8]: BFSRBits [31:16]: UFSR	Master 32-bit register containing Memory Management, Bus, and Usage fault flags.

Register	Key Bit Flags	Meaning and Root Cause
MMFSR (Memory Management)	DACCVIOL (Bit 1)IACCVIOL (Bit 0)	Data or instruction access violation. Attempted to read/write a memory address prohibited by the MPU.
BFSR (Bus Fault)	PRECISERR (Bit 1)IMPRECISERR (Bit 2)BFARVALID (Bit 7)	**PRECISERR**: Attempted to access an unmapped or unclocked peripheral register (PC points to offending instruction).**IMPRECISERR**: Asynchronous write buffer error (crashed instruction occurred earlier).**BFARVALID**: The address stored in BFAR is valid.
UFSR (Usage Fault)	UNDEFINSTR (Bit 0)INVSTATE (Bit 1)UNALIGNED (Bit 8)DIVBYZERO (Bit 9)	**UNDEFINSTR**: Execution of corrupted flash/RAM memory.**INVSTATE**: Attempted to execute ARM state instructions (Cortex-M only supports Thumb-2).**UNALIGNED**: Unaligned 32-bit memory access (when trap enabled).**DIVBYZERO**: Integer division by zero (when trap enabled).
BFAR (Bus Fault Address)	32-bit Address	The exact memory address that caused the bus fault (e.g., accessing unpowered peripheral 0x40013800).
MMFAR (MemManage Address)	32-bit Address	The memory address that violated MPU permissions.

8.2.3 Production HardFault Handler Implementation

Listing 8.1 — Assembly stub and C diagnostic handler for HardFault forensics

```
/* ============================================================================
   Assembly Stub: Extracts correct stack pointer (MSP vs PSP)
   ============================================================================ */
__attribute__((naked)) void HardFault_Handler(void)
{
    __asm volatile (
        "tst lr, #4            \n" /* Check bit 2 of EXC_RETURN */
        "ite eq                \n"
        "mrseq r0, msp         \n" /* If 0, exception stack frame is on MSP */
        "mrsne r0, psp         \n" /* If 1, exception stack frame is on PSP */
        "ldr r1, =HardFault_Decoder \n"
        "bx r1                 \n"
    );
}

/* ============================================================================
   C Decoder: Traps register state and halts for debugger inspection
   ============================================================================ */
void HardFault_Decoder(uint32_t *fault_stack)
{
```

```
    volatile uint32_t stacked_r0  = fault_stack[0];
    volatile uint32_t stacked_r1  = fault_stack[1];
    volatile uint32_t stacked_r2  = fault_stack[2];
    volatile uint32_t stacked_r3  = fault_stack[3];
    volatile uint32_t stacked_r12 = fault_stack[4];
    volatile uint32_t stacked_lr  = fault_stack[5];
    volatile uint32_t stacked_pc  = fault_stack[6]; /* CRASHED INSTRUCTION ADDRESS */
    volatile uint32_t stacked_psr = fault_stack[7];

    volatile uint32_t cfsr = SCB->CFSR;
    volatile uint32_t hfsr = SCB->HFSR;
    volatile uint32_t bfar = SCB->BFAR;
    volatile uint32_t mmfar = SCB->MMFAR;

    /* Prevent compiler from optimizing away diagnostic variables */
    (void)stacked_r0; (void)stacked_r1; (void)stacked_r2; (void)stacked_r3;
    (void)stacked_r12; (void)stacked_lr; (void)stacked_pc; (void)stacked_psr;
    (void)cfsr; (void)hfsr; (void)bfar; (void)mmfar;

    /* Halt processor here for debugger attachment */
    __BKPT(0);
    while (1);
}
```

8.3 The 10-Layer Root-Cause Debugging Playbook

When embedded firmware fails to operate, apply this systematic bottom-up diagnostic playbook:

```
┌──────────────────────────────────────────────────────┐
│ Layer 10: Application Logic & State Machines         │
├──────────────────────────────────────────────────────┤
│ Layer 9:  Concurrency, RTOS Priorities, Queues, & Race Cond │
├──────────────────────────────────────────────────────┤
│ Layer 8:  DMA Channel Routing, Buffer Ownership, Alignment  │
├──────────────────────────────────────────────────────┤
│ Layer 7:  NVIC Interrupt Enables, Preemption Groups & ISRs  │
├──────────────────────────────────────────────────────┤
│ Layer 6:  Peripheral Configuration Registers (Baud, Presc)  │
├──────────────────────────────────────────────────────┤
│ Layer 5:  Peripheral Clock Gating (RCC Enable Registers)    │
├──────────────────────────────────────────────────────┤
│ Layer 4:  Pin Multiplexing & Alternate Function (AF) Config │
├──────────────────────────────────────────────────────┤
│ Layer 3:  Clock Tree, PLL Lock, & Bus Prescalers     │
├──────────────────────────────────────────────────────┤
│ Layer 2:  Reset Circuit, NRST Line, BOOT0/BOOT1 Pins │
├──────────────────────────────────────────────────────┤
│ Layer 1:  Power Rails (VDD, VDDA, VREF+), Decoupling, GND   │
└──────────────────────────────────────────────────────┘
```

1. **Layer 1: Power & Ground**: Measure V_{DD} and V_{DDA} with a DMM directly at the MCU pins. Verify $3.30\,\text{V} \pm 5\%$. Check that all $100\,\text{nF}$ decoupling capacitors are populated. Verify continuity between sensor GND and MCU GND.
2. **Layer 2: Reset & Boot State**: Verify NRST is logic HIGH ($3.3\,\text{V}$). Confirm BOOT0 is tied securely to GND (a floating BOOT0 causes random boots into the system ROM bootloader).
3. **Layer 3: Clocks & PLL**: Verify the HSE external crystal is oscillating using an oscilloscope with a $10\times$ low-capacitance probe. Check if firmware is stuck in Error_Handler() inside SystemClock_Config().

4. **Layer 4: Pinout & Alternate Functions**: Check the MCU package datasheet alternate function mapping table. Ensure the GPIO mode is set to GPIO_MODE_AF_PP (or GPIO_MODE_AF_OD for I^2C), not standard GPIO input/output.
5. **Layer 5: Peripheral Clock Gating**: Verify that the peripheral clock enable bit is set in RCC (e.g., RCC_APB1ENR_USART2EN). Accessing any peripheral register while its clock is disabled causes an immediate Bus Fault!
6. **Layer 6: Peripheral Registers**: Inspect peripheral SFRs in the STM32CubeIDE debugger:
    - UART: Check USART_BRR matches baud rate math; verify UE and TE/RE bits are asserted.
    - Timers: Verify TIMx_CR1_CEN is set; verify PSC and ARR values.
7. **Layer 7: Interrupts & NVIC**: Verify the peripheral interrupt enable bit is set (e.g., USART_CR1_RXNEIE). Verify the global interrupt is enabled in the NVIC (NVIC_EnableIRQ()).
8. **Layer 8: DMA & Memory**: Confirm DMA channel mapping matches silicon hardware. Verify buffer pointers are located in valid SRAM and meet 32-byte cache alignment rules if on Cortex-M7.
9. **Layer 9: Concurrency & RTOS**: Verify stack high-water marks (uxTaskGetStackHighWaterMark()). Verify ISRs calling FreeRTOS APIs obey the configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY rule.
10. **Layer 10: Application Logic**: Verify data typing, signed two's complement decoding, endianness, and unit conversions.

8.4 Per-Subsystem Failure Diagnostic Checklists

8.4.1 GPIO and Smoke-Test Checklist

- ☐ Is __HAL_RCC_GPIOx_CLK_ENABLE() called before configuring the pin?
- ☐ For LEDs, is the drive mode Push-Pull (GPIO_MODE_OUTPUT_PP)?
- ☐ Is the output speed appropriate (GPIO_SPEED_FREQ_LOW for LEDs to minimize EMI)?
- ☐ If using an external button, is an internal pull-up/pull-down resistor enabled, or is there an external physical resistor?
- ☐ For buttons triggering interrupts, is the pin configured as GPIO_MODE_IT_FALLING / RISING, and is HAL_NVIC_EnableIRQ(EXTIx_IRQn) called?

8.4.2 UART / USART Checklist

- ☐ Are TX and RX swapped? (MCU TX must connect to Peripheral RX; MCU RX to Peripheral TX).
- ☐ Is there a common GND connection between both boards?
- ☐ Does the measured bit width on an oscilloscope match $\frac{1}{\text{Baud}}$ ($8.68\ \mu s$ for 115,200 baud)?
- ☐ Is the baud rate error strictly under 2.0%?
- ☐ If reception locks up permanently, are you clearing the Overrun Error (ORE) flag in HAL_UART_ErrorCallback()?
- ☐ In DMA mode, is HAL_UARTEx_ReceiveToIdle_DMA() re-armed inside the callback?

8.4.3 I²C Bus Checklist

- ☐ Are physical external pull-up resistors ($2.2\ \text{k}\Omega$ to $4.7\ \text{k}\Omega$) populated on both SDA and SCL?
- ☐ Is the 7-bit device address shifted left by 1 bit (ADDR << 1) when calling STM32 HAL functions?
- ☐ Does the sensor's WHO_AM_I register read the expected value?
- ☐ Has the sensor's power-on reset time ($t_{\text{POR}} \approx 50\ \text{ms}$) elapsed before sending commands?
- ☐ If the bus is locked with SDA clamped LOW, does the startup code execute the 9-clock bit-bang unjam sequence?

8.4.4 SPI Bus Checklist

- ☐ Is the Chip Select (CS) pin configured as a standard GPIO output, driven LOW before the transfer and returned HIGH after?
- ☐ Do the Clock Polarity (CPOL) and Clock Phase (CPHA) settings in CubeMX match the sensor datasheet SPI Mode (Mode 0 vs Mode 3)?
- ☐ When reading data from an SPI slave, are you transmitting dummy bytes (0x00 or 0xFF) to generate clock pulses?
- ☐ Is the SPI clock frequency (f_{SCK}) within the maximum rating of the slowest peripheral on the bus?

8.4.5 ADC Analog Front-End Checklist

- ☐ Is the pin configured in Analog Mode (GPIO_MODE_ANALOG) with digital input buffers disabled?
- ☐ Is f_{ADC} strictly below the family maximum ($14\ \text{MHz}$ on STM32F1)?
- ☐ Is the sampling time T_{sample} long enough for the external source impedance R_{AIN}?
- ☐ Has HAL_ADCEx_Calibration_Start(&hadc1) executed before the first conversion?
- ☐ In multi-channel Scan Mode, is DMA enabled? (Without DMA, the Data Register only retains the last converted channel).

8.4.6 Timers and PWM Checklist

- ☐ Did you account for the APB Timer $\times 2$ clock multiplier rule when computing input frequency?
- ☐ Did you remember that PSC and ARR divide by $(N + 1)$?
- ☐ Did you call HAL_TIM_PWM_Start() for the specific channel, not just HAL_TIM_Base_Start()?
- ☐ For advanced timers (TIM1/TIM8), is the main output enabled (TIM1->BDTR |= TIM_BDTR_MOE or HAL_TIM_PWM_Start)?

8.5 Seven Practical Hands-On Labs and Capstone

Every lab defines a specific engineering behavior and quantifiable acceptance test.

Lab 1 — GPIO Heartbeat and Oscilloscope Clock Verification

- **Objective**: Prove power, reset, clock tree stability, and baseline execution loop.
- **Procedure**:

1. In CubeMX, configure HSE 8 MHz crystal with PLL multiplier $\times 9$ to achieve 72 MHz SYSCLK.
2. Route MCO (Microcontroller Clock Output) pin to PA8, selecting SYSCLK.
3. Configure LED pin (PC13 on Blue Pill, PA5 on Nucleo-F401) as Push-Pull Output.
4. In main.c, toggle LED with HAL_Delay(250).

- **Acceptance Criteria**: Scope on PA8 measures exactly $72.00\ \text{MHz} \pm 50\ \text{ppm}$. LED blinks steadily at $2.00\ \text{Hz}$ across cold power-on without debugger attached.

Lab 2 — Interactive UART Telemetry Console with IDLE Line DMA

- **Objective**: Implement robust, variable-length, non-blocking serial communications.
- **Procedure**:
  1. Configure USART2 for 115,200 baud, 8N1. Enable DMA1 Channel 6 for RX.
  2. Implement HAL_UARTEx_ReceiveToIdle_DMA().
  3. Support commands: PING (replies PONG), LED 1 (turns on LED), LED 0 (turns off LED).
  4. Flooding test: Send a continuous stream of 500 random characters without line breaks.
- **Acceptance Criteria**: The system successfully parses commands. Flooded inputs do not trigger an unhandled Overrun Error (ORE) or lock up the console.

Lab 3 — Precision Timer Calculations and PWM Oscilloscope Verification

- **Objective**: Validate timer prescaler/auto-reload math against physical silicon waveforms.
- **Procedure**:
  1. Target: Exactly $1.000\ \text{kHz}$ PWM with 25.0% duty cycle on TIM2 Channel 1 (PA0).
  2. With $f_{\text{TIM}} = 72\ \text{MHz}$, calculate: $\text{PSC} = 71 \implies 1\ \text{MHz tick}$; $\text{ARR} = 999 \implies 1000\ \text{ticks} = 1.000\ \text{ms}$; $\text{CCR1} = 250$.
  3. Start PWM: HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1).
- **Acceptance Criteria**: Oscilloscope cursor measures frequency $= 1000.0\ \text{Hz} \pm 0.5\ \text{Hz}$ and positive pulse width $= 250.0\ \mu\text{s} \pm 0.5\ \mu\text{s}$.

Lab 4 — ADC Polling and Dynamic Supply Compensation

- **Objective**: Understand quantization math and eliminate supply voltage sag errors.
- **Procedure**:
  1. Configure ADC1 Channel 0 (PA0) and Channel 17 (V_{REFINT}).
  2. Connect a potentiometer to PA0. Measure wiper voltage with a calibrated 6.5-digit DMM.
  3. Compute V_{IN} using uncompensated formula, then compute using V_{REFINT} compensation.
  4. Lower board supply voltage from $3.3\ \text{V}$ to $3.0\ \text{V}$ using a lab power supply.
- **Acceptance Criteria**: As supply voltage varies, uncompensated reading drifts by $> 9\%$; V_{REFINT}-compensated calculation remains accurate within $\pm 0.8\%$ of DMM reading.

Lab 5 — Deterministic Timer-Triggered ADC with Circular DMA

- **Objective**: Implement zero-CPU, zero-jitter multi-sample data acquisition.
- **Procedure**:
  1. Configure TIM3 TRGO to trigger ADC1 at exactly 10.0 kSPS.
  2. Configure ADC1 in Circular DMA mode into a 1000-halfword buffer ($N = 1000$).
  3. In HAL_ADC_ConvHalfCpltCallback(), toggle PB0; in HAL_ADC_ConvCpltCallback(), toggle PB1.
- **Acceptance Criteria**: Oscilloscope probe on PB0 shows pulses at exactly 20.0 Hz ($T = \frac{N}{2F_S} = \frac{500}{10{,}000} = 50.0$ ms). CPU utilization remains under 1%.

Lab 6 — Reusable InvenSense MPU6050 Sensor Driver Integration

- **Objective**: Integrate a multi-layer, modular device driver with bus abstraction.
- **Procedure**:
  1. Connect MPU6050 to I2C1 (PB6/PB7) with 2.2 kΩ pull-up resistors.
  2. Integrate mpu6050.h and mpu6050.c. Initialize with MPU6050_Init().
  3. Execute offset calibration while the sensor rests flat on the lab bench.
  4. Stream real-time pitch and roll angles calculated from accelerometer vectors.
- **Acceptance Criteria**: With sensor stationary, $a_z = 9.81 \pm 0.1$ m/s^2, $a_x = 0.0 \pm 0.1$ m/s^2, $a_y = 0.0 \pm 0.1$ m/s^2. Disconnecting the sensor returns MPU6050_ERR_BUS without freezing the MCU.

Lab 7 — FreeRTOS Dual-Task Producer-Consumer Telemetry Pipeline

- **Objective**: Multi-threaded real-time execution with queue buffering and priority enforcement.
- **Procedure**:
  1. In CubeMX, enable FreeRTOS. Switch HAL Timebase Source to TIM4.
  2. Create Sensor Task (50 Hz, Priority 3) and Telemetry Task (Priority 2).
  3. Measure task execution times and verify stack high-water marks.
- **Acceptance Criteria**: Both tasks run continuously. Artificially stalling the UART transmitter causes queue drops to increment safely without starving the sensor task.

8.6 Master Engineering Formula Sheet

Table 8.2 — Master engineering formula reference sheet

Parameter	Governing Formula	Key Variables & Units	Critical Practical Rule
Timer Frequency	$f_{\text{update}} = \frac{f_{\text{TIM}}}{(\text{PSC} + 1) \times (\text{ARR} +$	f_{TIM}: Timer clock (Hz)PSC: 16-bit prescalerARR: Auto-reload register	Account for APB timer $\times 2$ multiplier if APB divider > 1.

Parameter	Governing Formula	Key Variables & Units	Critical Practical Rule
PWM Duty Cycle	$\text{Duty (\%)} = \frac{\text{CCR}}{\text{ARR} + 1} \times 100\%$	CCR: Capture/Compare register	In PWM Mode 1, output is HIGH while $\text{CNT} < \text{CCR}$.
ADC LSB Size	$\text{LSB} = \frac{V_{\text{REF+}} - V_{\text{REF–}}}{2^N}$	V_{REF}: Reference voltage (V)N: Resolution bits (12 bits $\Longrightarrow 4096$)	Typical LSB for 3.3V 12-bit ADC is 0.80566 mV.
Code to Voltage	$V_{\text{IN}} = \text{Code} \times \frac{V_{\text{REF}}}{2^N - 1}$	Code: Raw integer reading	Divide by 4095 so that full-scale code represents exactly 3.300 V.
Total ADC Conversion	$T_{\text{conv}} = T_{\text{sample}} + 12.5 \text{ cycles}$	T_{sample}: Sampling time cycles	Ensure $T_{\text{sample}} \geq 8.3 \times R_{\text{AIN}} \times C_{\text{ADC}}$.
Circular DMA Deadline	$T_{\text{deadline}} = \frac{N}{2 \times F_s}$	N: Total buffer length (samples)F_s: Sampling rate (Hz)	Half-buffer processing time must be strictly less than T_{deadline}.
UART Baud Divisor	$\text{Div} = \frac{f_{\text{PCLK}}}{16 \times \text{Baud}}$	f_{PCLK}: Bus clock (Hz)	Baud error must be strictly under $\pm 2.0\%$.
UART Bandwidth	$\text{Throughput} \approx \frac{\text{Baud}}{10}$ Bytes/sec	Assumes 8N1 frame format	115,200 baud yields maximum 11.52 kB/s.
I²C Max Pull-up	$R_{p,\max} = \frac{t_{r,\max}}{0.8473 \times C_b}$	$t_{r,\max}$: Max rise time (300 ns for 400 kHz)C_b: Bus capacitance (Farads)	Slower rise times violate maximum high-level timing.
I²C Min Pull-up	$R_{p,\min} = \frac{V_{DD} - V_{OL,\max}}{I_{OL}}$	$V_{OL} \leq 0.4$ V, $I_{OL} = 3$ mA	$R_{p,\min} \approx 967\ \Omega$ at 3.3 V.

8.7 20-Question Technical Knowledge Check with Comprehensive Answers

Use these questions for oral technical interviews or written batch examinations.

Question 1: What does an `.ioc` file actually store, and what is its role?

Answer: An .ioc file is a plain-text configuration file maintained by STM32CubeMX. It stores all user-selected hardware parameters: target MCU part number, pin multiplexing (GPIO modes and Alternate Functions), clock tree configurations (PLL multipliers and dividers), peripheral parameter settings, and middleware configurations.

It is used exclusively by the CubeMX code generation engine to synthesize C initialization source files (main.c, stm32f4xx_hal_msp.c, etc.). It is not compiled or flashed to the microcontroller.

Question 2: Why can regenerating code in CubeMX overwrite custom application logic, and how is this avoided?

Answer: When CubeMX regenerates code, its template engine rewrites the project files from scratch. To protect custom firmware, CubeMX parses the existing files and preserves only code enclosed within paired /* USER CODE BEGIN <Name> */ and /* USER CODE END <Name> */ comments. Any application logic placed outside these designated blocks is permanently deleted during code generation. The best practice is to place application logic in separate, dedicated .c and .h files that CubeMX never touches.

Question 3: What is the fundamental difference between STM32CubeMX and STM32CubeIDE?

Answer: STM32CubeMX is a graphical hardware configuration and initialization code generation tool. STM32CubeIDE is a full-featured integrated development environment built on Eclipse and the GNU ARM GCC toolchain that includes code editing, compiling, linking, flashing, and hardware debugging (GDB/OpenOCD/ST-LINK). STM32CubeIDE includes an embedded version of CubeMX internally, allowing .ioc editing directly within the IDE.

Question 4: When is Polling appropriate in professional embedded systems, and when is it forbidden?

Answer: Polling is appropriate during early power-on initialization, single-character debug transmissions inside a HardFault crash handler, and brief bounded delays where the timeout is short and no higher-priority deadlines can be missed. It is strictly forbidden in continuous multi-tasking loops, high-rate sensor acquisition, and battery-powered applications where blocking wastes vital CPU cycles and prevents power-saving sleep modes.

Question 5: Compare Interrupts and DMA in terms of CPU involvement.

Answer: An interrupt requests the CPU to immediately suspend its current thread, execute a hardware context switch, and run an Interrupt Service Routine (ISR) to process data. The CPU is directly involved in executing the transfer. DMA (Direct Memory Access) utilizes an independent hardware bus master to transfer data directly between peripherals and SRAM across the internal bus matrix with zero CPU involvement. In high-performance systems, DMA performs the continuous data transfers, while interrupts are generated only when the transfer reaches halfway (HT) or finishes (TC).

Question 6: Why is the C `volatile` keyword insufficient for synchronizing shared buffers between an ISR and `main()`?

Answer: The volatile keyword only instructs the compiler not to optimize away reads and writes to a variable by caching it in a CPU register; it forces memory access on every read. However, volatile provides **zero atomicity** for multi-byte variables on 32-bit cores, provides **no mutual exclusion** (it cannot prevent an ISR from preempting a read mid-structure), enforces **no memory barriers** against out-of-order CPU pipeline execution, and provides **no cache invalidation** on Cortex-M7 cores with D-Cache. Synchronization requires disabling interrupts, atomic primitives, or RTOS queues.

Question 7: Why can the timer clock frequency (f_{TIM}) be double the APB bus clock frequency?

Answer: On STM32 microcontrollers, internal timer clock multipliers automatically double the clock frequency feeding the timer peripherals whenever the APB prescaler is greater than 1 ($\text{APB Prescaler} \neq 1$). This hardware feature ensures that even when peripheral bus frequencies are divided down to conserve dynamic power, the timers retain maximum high-resolution counting frequency.

Question 8: Why do timer formulas use $(PSC + 1)$ and $(ARR + 1)$ instead of PSC and ARR?

Answer: The hardware prescaler and auto-reload registers are zero-indexed counters. When a register value of 0 is loaded into the prescaler, the input clock is divided by 1 (no division). When a value of N is loaded, the hardware counter must count through $N + 1$ discrete clock edges (from 0 up to and including N) before rolling over and generating an update event.

Question 9: Explain Clock Polarity (CPOL) and Clock Phase (CPHA) in SPI.

Answer: CPOL defines the idle electrical logic level of the Serial Clock line (SCLK) when no communication is active ($0 = \text{LOW}$, $1 = \text{HIGH}$). CPHA defines which clock edge samples (captures) data and which edge toggles (shifts) data ($0 = \text{sample on first clock transition}$; $1 = \text{sample on second clock transition}$). Combined, they define SPI Modes 0 through 3.

Question 10: Why does an I^2C bus require external pull-up resistors, and what governs their values?

Answer: I^2C pins operate in an open-drain (wired-AND) configuration. Devices can only actively pull the bus lines LOW to ground; they cannot drive them HIGH. External pull-up resistors are required to pull the lines up to V_{DD} when all devices release the bus. The maximum allowable resistance is governed by the total bus capacitance C_b and the maximum rise time specification ($R_{p,\max} = \frac{t_{r,\max}}{0.8473 C_b}$). The minimum resistance is governed by the maximum allowable sink current of the output transistors ($I_{OL} = 3\ \text{mA}$) to guarantee that V_{OL} remains below $0.4\ \text{V}$.

Question 11: What are the primary root causes of garbled characters on a UART console?

Answer: 1. Baud rate mismatch exceeding $\pm 2.5\%$, caused by incorrect clock tree configuration or relying on an inaccurate internal RC oscillator (HSI) over temperature. 2. Mismatched frame parameters (e.g., host set to 8N1 while MCU transmits 8E1 or 9-bit data). 3. Missing common ground (GND) wire between the microcontroller and the USB-UART adapter. 4. Swapped TX and RX wiring, or connecting TTL 3.3V UART directly to an RS-232 $\pm 12\ \text{V}$ port.

Question 12: What is the difference between ADC Resolution and ADC Accuracy?

Answer: Resolution is the number of discrete digital intervals the converter can divide the input span into ($12\ \text{bits} \Longrightarrow 4096$ quantization codes, $\text{LSB} \approx 0.8\ \text{mV}$). Accuracy is the real-world degree of conformity between the converted digital code and the true absolute physical voltage. Accuracy is degraded by offset error, gain error, integral non-linearity (INL), thermal noise, and reference voltage sag.

Question 13: What is Aliasing in sampled data systems, and how is it prevented?

Answer: Aliasing occurs when an analog signal contains frequency components higher than half the sampling frequency ($f > \frac{F_s}{2}$, the Nyquist frequency). During sampling, these high-frequency components fold back across the Nyquist threshold, appearing as indistinguishable, false low-frequency signals in the digitized data. Aliasing cannot be removed in software; it must be prevented before sampling using an analog hardware low-pass **anti-aliasing filter**.

Question 14: What causes data loss in circular DMA, and how is it detected?

Answer: In circular DMA, data loss occurs when the CPU consumer fails to finish processing the previous half of the buffer before the DMA hardware producer wraps around and begins overwriting that same memory space. This is detected by implementing a software processing flag or overrun counter inside the Half-Transfer (HT) and Transfer-Complete (TC) interrupt callbacks.

Question 15: What makes an embedded sensor driver truly "reusable"?

Answer: A reusable driver has zero hardcoded peripheral instances (&hi2c1), encapsulates all instance state inside a handle struct, returns typed error codes, enforces finite bus timeouts, outputs clean engineering SI units, isolates physical I/O behind function-pointer bus abstractions, and can be compiled and unit-tested on a host PC without hardware.

Question 16: When should an engineer transition from a bare-metal superloop to an RTOS?

Answer: When an application contains multiple independent execution threads with different, asynchronous deadlines; when integrating heavy third-party middleware stacks (USB, TCP/IP, SD-card file systems) that rely on blocking operations; and when non-blocking state machines become too complex to verify and maintain.

Question 17: What is the fundamental difference between a Queue, a Semaphore, and a Mutex in FreeRTOS?

Answer: - **Queue**: A thread-safe FIFO buffer that passes data packets **by value** between tasks or from ISRs to tasks. - **Semaphore**: A flag (binary) or counter (counting) used for **synchronization and event signaling**. Does not possess ownership. - **Mutex**: A mutual exclusion lock used to protect a **shared resource**. Has ownership semantics and features **Priority Inheritance** to prevent priority inversion. Mutexes must never be accessed from an ISR.

Question 18: Why must Interrupt Service Routines (ISRs) be kept as short as possible?

Answer: While an ISR is executing, all interrupts of equal or lower priority are blocked. Lengthy ISRs increase interrupt latency across the entire system, induce severe timing jitter in periodic control loops, cause UART/SPI receiver overrun errors (ORE), and delay the execution of RTOS tasks.

Question 19: An I^2C sensor fails to acknowledge its address upon boot. List the diagnostic sequence.

Answer: 1. Check V_{DD} and GND at the sensor pins with a DMM. 2. Verify that physical $2.2\ \text{k}\Omega$ to $4.7\ \text{k}\Omega$ pull-up resistors are installed. 3. Confirm that the 7-bit datasheet address is shifted left by 1 bit in software (ADDR << 1). 4. Ensure the sensor's power-on reset time (t_{POR}) has elapsed before the first command. 5. Inspect SDA and SCL with an oscilloscope to verify that clock pulses are toggling and that the signal achieves sharp rise times to $3.3\ \text{V}$.

Question 20: How should an engineer verify that a timer is generating the exact calculated frequency?

Answer: Configure a timer channel in Output Compare (Toggle) or PWM mode routed to an external GPIO pin. Measure the output waveform directly using a calibrated oscilloscope or hardware logic analyzer with a frequency counter. Never attempt to verify timer rates by toggling software flags and reading HAL_GetTick(), as software overhead and SysTick resolution introduce significant measurement error.

9. Master Reference and Comparison Tables

This chapter compiles comprehensive reference tables summarizing STM32 hardware architecture, peripheral trade-offs, and communication buses.

9.1 STM32 Microcontroller Families At a Glance

Table 9.1 — Major STM32 microcontroller series comparison

Family	ARM Core	Max Frequency	L1 Cache	DMA Controller	ADC Max Speed	Target Application
STM32F1 (F103)	Cortex-M3	72 MHz	None	Channel-based (DMA1/2)	1.0 MSPS (12-bit)	Cost-sensitive general purpose, educational robotics
STM32F4 (F401/F407/F446)	Cortex-M4F	84–180 MHz	ART Accelerator	Stream/FIFO-based (DMA1/2)	2.4 MSPS (12-bit)	Flight controllers, audio DSP, advanced robotics
STM32G4 (G474)	Cortex-M4F	170 MHz	ART Accelerator	DMAMUX + Flexible Streams	4.0 MSPS (12-bit) + HRTIM	Digital power supply, drone ESCs, motor FOC
STM32L4 (L476)	Cortex-M4F	80 MHz	ART Accelerator	Channel-based with DMAMUX	5.0 MSPS (12-bit)	Ultra-low-power IoT, wearable medical devices
STM32H7 (H743/H753)	Cortex-M7 (+ M4)	480–550 MHz	32 KB I-Cache + 32 KB D-Cache	MDMA + BDMA + DMA1/2	3.6 MSPS (16-bit)	Computer vision, AI edge processing, industrial gateways

9.2 Comprehensive Communication Protocol Matrix

Table 9.2 — Master comparison of serial, industrial, and networking protocols

Protocol	Physical Topology	Duplex	Max Data Rate	Wires (+ GND)	Max Distance	Addressing	Primary Advantages
UART / USART	Point-to-Point	Full	Up to 10–27 Mbps	2	2–3 m (TTL)	None (Framed)	Ubiquitous, simple, PC Virtual COM port support.
SPI	Single Master / Multi-Slave	Full	Up to 40–80 Mbps	3 + N (CS)	< 30 cm	Hardware CS pin	Extremely high speed, zero protocol overhead.
I^2C	Multi-Master / Multi-Slave	Half	100k / 400k / 1 Mbps	2	< 1 m	7-bit / 10-bit	Only 2 wires for dozens of sensors; built-in flow control.
CAN / CAN-FD	Multi-Drop Differential	Half	1 Mbps (FD: 5 Mbps)	2	Up to 1000 m	11-bit / 29-bit ID	Non-destructive arbitration, hardware CRC, extreme EMI immunity.
RS-485	Multi-Drop Differential	Half	Up to 10–20 Mbps	2	Up to 1200 m	Application level	Robust long-range industrial Modbus networks.
USB 2.0 (FS/HS)	Tiered Star	Half	12 Mbps / 480 Mbps	2 (Diff D+/D-)	5 m	Dynamic Enumeration	High-speed PC connectivity, plug-and-play, power delivery.
Ethernet	Star / Switch	Full	100 Mbps / 1 Gbps	4 / 8	100 m	MAC / IP Address	Global network routability, massive bandwidth.

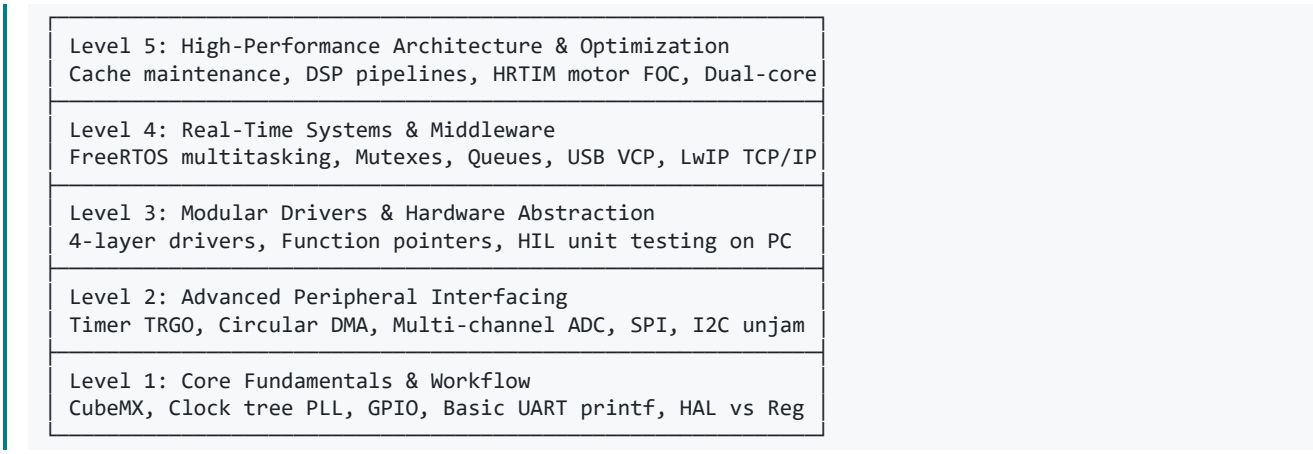
9.3 Execution and Data Transfer Paradigms

Table 9.3 — Trade-offs between Polling, Interrupts, DMA, and RTOS

Paradigm	CPU Utilization	Software Complexity	Maximum Throughput	Latency & Jitter	Best Engineering Application
Polling	100% blocked	Minimal	High (single task)	Extreme jitter	Boot initialization, HardFault crash reporting.
Interrupts	Medium (context switch overhead)	Moderate	Medium (< 100 kSPS)	Low latency, low jitter	Infrequent asynchronous events (Buttons, Fault pins).
DMA	**Near 0%**	Moderate	**Maximum silicon bandwidth**	**Zero jitter**	High-speed ADC, audio streaming, display SPI, UART logs.
RTOS Multitasking	Low (~1–3% scheduler load)	High	Balanced across threads	Deterministic priority-based	Complex systems with multiple asynchronous deadlines.

10. Conclusion and Recommended Learning Path

Mastering STM32 embedded systems requires progressing through five distinct levels of engineering maturity.



The Engineering Mindset: Observe and Measure

A professional embedded systems engineer does not guess or assume. When a peripheral behaves unexpectedly: 1. **Probe the physical silicon**: Connect an oscilloscope to observe rise times, ringing, ground bounce, and clock frequencies. 2. **Read the registers**: Inspect peripheral SFRs in the debugger to verify that configuration bits match the datasheet. 3. **Verify mathematics**: Calculate prescalers, auto-reload values, baud rate dividers, and filter coefficients on paper before writing firmware. 4. **Isolate layers**: Test device drivers independently with mock functions on a host PC before integrating them into a robotic chassis.

---

11. Comprehensive Technical Glossary

This glossary defines essential technical terminology, including all 30 foundational STM32CubeMX concepts.

- **ADC (Analog-to-Digital Converter)**: An internal peripheral that quantizes continuous physical voltages into discrete digital integer codes (e.g., 12-bit SAR ADC yielding 0–4095).
- **AHB (Advanced High-performance Bus)**: The primary high-speed bus connecting the Cortex-M core, DMA controllers, and high-bandwidth memory (Flash, SRAM).
- **APB (Advanced Peripheral Bus)**: Peripheral buses (APB1 and APB2) bridged to the AHB, clocking timers, UART, I^2C, and SPI peripherals.
- **ARR (Auto-Reload Register)**: The 16-bit or 32-bit timer register defining the maximum counter value before rollover and update event generation.
- **BFSR (Bus Fault Status Register)**: Hardware status register recording bus errors, invalid memory accesses, and whether BFAR holds a valid address.
- **BRR (Baud Rate Register)**: Peripheral register in USART containing the mantissa and fraction clock divider used to establish serial baud rates.
- **BSP (Board Support Package)**: A collection of software drivers tailored to a specific development board (e.g., Nucleo-F401RE, Discovery), managing onboard LEDs, buttons, and external sensors.
- **CFSR (Configurable Fault Status Register)**: Cortex-M 32-bit register combining MemManage, Bus Fault, and Usage Fault status flags.
- **Circular DMA**: An operating mode where the DMA controller automatically wraps its memory pointer back to the beginning upon reaching the buffer end, continuing transfers indefinitely.
- **CMSIS (Cortex Microcontroller Software Interface Standard)**: ARM's vendor-independent hardware abstraction layer providing standardized register definitions, core intrinsics, and DSP/RTOS APIs.
- **CPHA (Clock Phase)**: SPI configuration bit specifying whether data is sampled on the first or second clock transition.
- **CPOL (Clock Polarity)**: SPI configuration bit defining the idle electrical logic level (LOW or HIGH) of the SCLK line.
- **D-Cache (Data Cache)**: High-speed L1 cache on ARM Cortex-M7 cores accelerating memory reads and writes; requires explicit invalidation and cleaning during DMA operations.
- **DMA (Direct Memory Access)**: A specialized hardware bus master that transfers data directly between peripherals and memory without CPU intervention.
- **ENOB (Effective Number of Bits)**: A true measure of ADC dynamic range and accuracy accounting for thermal noise, quantization distortion, and non-linearities.
- **EXTI (Extended Interrupt and Event Controller)**: Hardware peripheral managing external pin edge detection and routing interrupt requests to the NVIC.

- **Flash Memory**: Non-volatile, electrically erasable on-chip storage where compiled executable firmware instructions and constant data reside.
- **FreeRTOS**: A market-leading open-source real-time operating system kernel providing preemptive scheduling, tasks, queues, semaphores, and mutexes.
- **GPIO (General-Purpose Input/Output)**: Software-configurable pins capable of operating as digital inputs, outputs, analog pins, or peripheral alternate functions.
- **HAL (Hardware Abstraction Layer)**: STMicroelectronics' high-level, portable C library providing standardized API calls and handle structures across all STM32 families.
- **HSE (High-Speed External)**: An accurate, temperature-stable external clock source provided by a quartz crystal or ceramic resonator (typically 4–26 MHz).
- **HSI (High-Speed Internal)**: An on-chip RC oscillator providing an immediate clock upon power-up (e.g., 8 MHz or 16 MHz); subject to thermal drift ($\pm 1\%$ to $\pm 3\%$).
- **I^2C (Inter-Integrated Circuit)**: A synchronous, multi-master, open-drain two-wire serial bus (SDA, SCL) using pull-up resistors and 7-bit/10-bit software addressing.
- **IDE (Integrated Development Environment)**: Software suite combining text editing, compilation, linking, flashing, and source-level debugging (e.g., STM32CubeIDE).
- **INL / DNL**: Integral Non-Linearity and Differential Non-Linearity; metrics measuring the deviation of an ADC quantization transfer function from an ideal straight line.
- **IOC File**: Plain-text XML-like project configuration file used by STM32CubeMX to store hardware configuration settings and synthesize C initialization code.
- **ISR (Interrupt Service Routine)**: A dedicated function invoked automatically by the hardware NVIC when a peripheral or external interrupt event occurs.
- **IWDG (Independent Watchdog)**: A failsafe hardware countdown timer clocked by an independent low-speed oscillator (LSI) that resets the MCU if firmware locks up.
- **LL (Low-Layer Drivers)**: ST's lightweight, non-blocking, register-oriented C library providing direct, inline hardware access with zero abstraction overhead.
- **LSE (Low-Speed External)**: An accurate external 32.768 kHz crystal oscillator used exclusively to drive the Real-Time Clock (RTC).
- **LSI (Low-Speed Internal)**: An internal low-power ~32–40 kHz RC oscillator used to clock the Independent Watchdog (IWDG) and low-power wake-up timers.
- **MCU (Microcontroller Unit)**: A highly integrated single-chip computer combining a CPU core, volatile RAM, non-volatile Flash, and diverse hardware peripherals.
- **MPU (Memory Protection Unit)**: Hardware unit on ARM cores that monitors memory transactions and enforces access permissions, isolating OS kernels and disabling cache on DMA regions.
- **Mutex (Mutual Exclusion)**: An RTOS synchronization primitive featuring priority inheritance used to serialize access to shared hardware resources across tasks.

- **NVIC (Nested Vectored Interrupt Controller)**: The tightly coupled ARM Cortex-M interrupt controller providing low-latency, nested priority interrupt handling.
- **Open-Drain**: An output pin stage featuring an NMOS transistor that can actively pull a line LOW to ground but floats when driving HIGH, requiring an external pull-up resistor.
- **PendSV (Pended Servicing Call)**: A low-priority exception on Cortex-M processors dedicated to executing RTOS context switches when no other interrupts are active.
- **PLL (Phase-Locked Loop)**: An on-chip high-frequency clock synthesizer that multiplies lower-frequency reference clocks (HSE or HSI) up to maximum core operating limits (e.g. 72–480 MHz).
- **Priority Inheritance**: An algorithmic protocol implemented in RTOS mutexes that temporarily elevates the priority of a low-priority task holding a lock to prevent priority inversion.
- **Push-Pull**: An active output pin stage featuring complementary PMOS and NMOS transistors capable of driving strong logic HIGH and strong logic LOW.
- **PWM (Pulse Width Modulation)**: A digital square-wave signal generated by a timer with a fixed frequency and variable duty cycle, used for motor speed control and LED dimming.
- **Queue**: A thread-safe FIFO data structure in FreeRTOS that passes discrete data packets by value between tasks or from ISRs to tasks.
- **RCC (Reset and Clock Control)**: The foundational peripheral controlling reset generation, oscillator selection, PLL configuration, and peripheral clock gating.
- **RTC (Real-Time Clock)**: An ultra-low-power autonomous hardware calendar counter that tracks seconds, minutes, hours, days, months, and years.
- **RTOS (Real-Time Operating System)**: Software providing deterministic multitasking scheduling, thread synchronization, and resource management.
- **SAR (Successive Approximation Register)**: A high-speed, high-resolution ADC architecture that determines digital codes through binary search comparisons.
- **SPI (Serial Peripheral Interface)**: A synchronous, full-duplex, four-wire master-slave serial bus capable of multi-megahertz data transfers.
- **SysTick (System Tick Timer)**: A 24-bit down-counter timer built into the ARM Cortex-M core, used to generate periodic 1 ms tick interrupts for OS scheduling and HAL_Delay().
- **TCB (Task Control Block)**: A FreeRTOS data structure storing all context, stack boundaries, priority, and state flags for an active task.
- **TIM (Timer)**: A versatile peripheral comprising a prescaler, counter, and capture/compare registers capable of generating PWM, measuring pulse widths, and triggering DMA.
- **TRGO (Trigger Output)**: An internal hardware signal generated by a timer upon counter update, compare match, or reset, used to trigger ADC conversions with zero CPU jitter.
- **UART / USART**: Universal Asynchronous/Synchronous Receiver Transmitter; a serial communication interface supporting framed byte transfers over TX and RX lines.

- **VDD / VREF**: V_{DD} is the primary digital operating voltage supply rail (typically 3.3 V); V_{REF} is the analog reference voltage governing ADC quantization full-scale limits.
- **WWDG (Window Watchdog)**: A safety watchdog timer clocked from the APB bus that resets the system if refreshed too late OR too early, catching runaway execution loops.

---

12. References and Official Documentation

This technical reference is built upon verified silicon specifications and official technical documentation.

12.1 STMicroelectronics Official Documentation

1. **STM32F103x8/B Datasheet**: *Medium-density performance line ARM-based 32-bit MCU with 64 or 128 KB Flash, USB, CAN, 7 timers, 2 ADCs, 9 com. interfaces*. Document ID: DS5319.
2. **RM0008 Reference Manual**: *STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs*. STMicroelectronics.
3. **RM0090 Reference Manual**: *STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced ARM-based 32-bit MCUs*. STMicroelectronics.
4. **UM1718 User Manual**: *STM32CubeMX for STM32 configuration and initialization C code generation*. STMicroelectronics.
5. **UM2609 User Manual**: *STM32CubeIDE user guide*. STMicroelectronics.
6. **UM1850 User Manual**: *Description of STM32F1 HAL and low-layer drivers*. STMicroelectronics.
7. **AN2586 Application Note**: *STM32F10xxx hardware development: getting started*. STMicroelectronics.
8. **AN2834 Application Note**: *How to get the best ADC accuracy in STM32 microcontrollers*. STMicroelectronics.
9. **AN4031 Application Note**: *Using the DMA controller on STM32F2, STM32F4 and STM32F7 series*. STMicroelectronics.

12.2 ARM Architecture Documentation

1. **ARM DDI 0403E.e**: *ARMv7-M Architecture Reference Manual*. ARM Limited.
2. **ARM DUI 0552A**: *Cortex-M3 Devices Generic User Guide*. ARM Limited.
3. **ARM DUI 0553A**: *Cortex-M4 Devices Generic User Guide*. ARM Limited.
4. **Yiu, Joseph**: *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*. 3rd Edition, Newnes, 2013.

12.3 FreeRTOS Documentation

1. **Barry, Richard**: *Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide*. Real Time Engineers Ltd.
2. **FreeRTOS Kernel Reference Manual**: *API Functions and Configuration Options*. Official FreeRTOS Documentation (https://www.freertos.org).

12.4 Industry Standards and Sensor Specifications

1. **NXP Semiconductors UM10204**: *I2C-bus specification and user manual*. Rev. 7.0, 2021.
2. **InvenSense Inc.**: *MPU-6000 and MPU-6050 Product Specification*. Revision 3.4, Document Number: PS-MPU-6000A-00.
3. **InvenSense Inc.**: *MPU-6000 and MPU-6050 Register Map and Descriptions*. Revision 4.2, Document Number: RM-MPU-6000A-00.

Appendix A — Source Coverage Checklist and Reconciliation Log

This appendix documents the systematic analysis, extraction, deduplication, and technical reconciliation performed across all seven source files provided for this session consolidation.

A.1 Source File Coverage Checklist

Table A.1 — Comprehensive source file coverage and extraction audit

Source File	Format & Scope	Unique Technical Content Identified	Where Incorporated in Reference
Overview.md	Markdown; Session overview & curriculum map	High-level 6-part session architecture, core philosophy ("no magic, no black boxes"), student learning outcomes.	Chapter 1 (1.2, 1.3), Chapter 10.
STM32_Session_Instructor_Guide.docx	DOCX; Instructor manual & pedagogical guide	6-question instructor rule, 6-hour evening delivery schedule, 30-term CubeMX glossary, callout boxes for common student traps.	Chapter 1 (1.3.2), Chapter 8 (8.5), Chapter 11 (complete glossary).
STM32_Session_Complete_Guide.md	Markdown; Full training manual	Per-subsystem failure diagnostic checklists (GPIO, UART, I2C, SPI, ADC, Timers, DMA, RTOS), 7 hands-on labs with acceptance criteria, master formula table.	Chapter 8 (8.4 checklists, 8.5 labs, 8.6 formulas).
STM32_Deep_Dive_In_Depth.md	Markdown; Detailed textbook	10-layer physical-to-firmware chain, ownership model, APB timer ×2 rule, 9-clock I^2C unjamming, Cortex-M7 D-Cache coherency, 4-layer driver architecture, 20-question knowledge check.	Chapter 1 (1.4), Chapter 2 (2.1, 2.7), Chapter 4 (4.3.6), Chapter 5 (5.8), Chapter 6, Chapter 8 (8.7).
STM32_Deep_Dive_Complete_Guide.md	Markdown; Reference text	Code generation lifecycle, .ioc internals, USER CODE preservation, HAL vs LL vs Register comparison with PC13 examples, DMA modes, SPI modes table.	Chapter 2 (2.4, 2.8, 2.9), Chapter 3 (3.3, 3.6), Chapter 4 (4.2).

Source File	Format & Scope	Unique Technical Content Identified	Where Incorporated in Reference
STM32_Deep_Dive_Complete_Guide_Compact.md	Markdown; Byte-identical to above	Redundant duplicate (verified byte-identical SHA256); consolidated without information loss.	Merged with Chapter 2–4.
STM32_Deep_Dive_Complete_Guide.docx	DOCX; Reference manual	System architecture diagrams, callout blocks, table formatting, retargeting printf implementation.	Chapter 4 (4.1.6), Chapter 9, diagrams 1–8.

A.2 Reconciled Contradictions and Technical Corrections

During the consolidation, several conflicting statements, outdated formulas, and family-specific inaccuracies were discovered in the source documents. These have been reconciled using official STMicroelectronics reference manuals, ARM architecture specifications, and NXP bus standards:

1. **ADC Trigger Timer on STM32F103**:
    - *Source Conflict*: An earlier script (code.py) hardcoded TIM2_TRGO as the external hardware trigger for ADC1.
    - *Correction*: On STM32F103 devices (Reference Manual RM0008, Section 11.3.6), the regular group external triggers for ADC1 and ADC2 are TIM1_CC1, TIM1_CC2, TIM1_CC3, TIM2_CC2, TIM3_TRGO, TIM4_CC4, and EXTI11. **TIM2_TRGO is NOT a valid trigger on STM32F103** (it is used on STM32F4). The example was corrected to use **TIM3_TRGO**, which is natively supported on all STM32F103 density lines.
2. **HAL Timebase Timer on STM32F103C8T6 with FreeRTOS**:
    - *Source Conflict*: Several tutorials generically recommend selecting TIM6 or TIM7 as the alternate timebase timer when FreeRTOS claims SysTick.
    - *Correction*: The STM32F103C8T6 (Medium-Density) contains **no basic timers** (TIM6 and TIM7 do not exist in silicon on this chip; only TIM1, TIM2, TIM3, and TIM4 are populated). The documentation explicitly instructs students to allocate **TIM4** (or a spare general-purpose timer) on STM32F103 parts.
3. **I²C Rise-Time Mathematics and Constant**:
    - *Source Conflict*: Different files used $0.8 \times R_p C_b$ versus $0.8473 \times R_p C_b$ for rise-time calculations.
    - *Correction*: Under NXP I²C specification UM10204, rise time t_r is measured from 30% to 70% of V_{DD}. The exact analytical solution is:

$$t_r = R_p C_b \left(\ln \frac{1}{1-0.7} - \ln \frac{1}{1-0.3} \right) = R_p C_b (\ln 3.333 - \ln 1.428) \approx 0.8473 \times R_p C_b$$

This rigorous constant was adopted. Furthermore, the minimum pull-up resistor was derived from $I_{OL} = 3$ mA at $V_{OL} = 0.4$ V, yielding $R_{p,\min} \approx 967\ \Omega$ at 3.3 V.

4. **I²C Sensor Burst Read Duration**:
   - *Source Conflict*: Sources claimed a 14-byte IMU burst read takes "~250 µs".
   - *Correction*: At 400 kHz Fast Mode, a 14-byte HAL_I2C_Mem_Read requires: START (1) + Address (8) + ACK (1) + Reg Addr (8) + ACK (1) + Repeated START (1) + Address+Read (8) + ACK (1) + 14 × [Data (8) + ACK (1)] + STOP (1) = 155 clock cycles.

$$T = \frac{155}{400{,}000\ \text{Hz}} = 387.5\ \mu\text{s}$$

   At 100 kHz Standard Mode, this consumes 1.55 ms. This distinction is critical for students budgeting high-rate 1 kHz control loops.

5. **ADC Quantization Formula and Divisor (2^N vs. $2^N - 1$)**:
   - *Source Conflict*: Documents inconsistently divided by 4096 or 4095 when calculating voltage from digital code.
   - *Correction*: The document clarifies both conventions: 4096 (2^N) defines the mathematical width of one LSB step interval ($\Delta V = \frac{V_{REF}}{2^N}$). However, because an unsigned 12-bit integer register saturates at 4095 (0x0FFF), software reconstructing full-scale voltage from digital code divides by 4095 ($2^N - 1$) so that Code == 4095 maps to exactly 3.300 V.

6. **FreeRTOS SysCall Interrupt Priority Validation**:
   - *Source Conflict*: Some files claimed violating the priority rule causes an "immediate HardFault".
   - *Correction*: The ARM Cortex-M architecture itself does not trigger a hardware exception purely based on priority numbers. Rather, FreeRTOS executes an internal assertion configASSERT(ucCurrentPriority >= ucMaxSysCallPriority) inside vPortValidateInterruptPriority(). If assertions are enabled, it traps the core; if disabled, the unmasked ISR corrupts scheduler linked lists, producing memory faults later.

7. **Character Literal Escapes in Generated Code**:
   - *Source Conflict*: An earlier python generator script (code.py) emitted double-escaped '\\n' characters into C source blocks.
   - *Correction*: All C listings have been validated to contain clean, standard escape sequences ('\n', '\r', '\0').

Appendix B — Hardware-Specific Verification Items

The following architectural parameters must be verified against the exact silicon reference manual when porting this reference to other STM32 microcontrollers:

1. **GPIO Alternate Function Multiplexing**:
    - **STM32F1**: Uses legacy AFIO remap registers (AFIO_MAPR), allowing only fixed standard or remapped pin groupings.
    - **STM32F4/G4/H7**: Uses modern per-pin 4-bit multiplexer registers (GPIOx_AFRL and GPIOx_AFRH, supporting AF0 through AF15).
2. **DMA Request Routing**:
    - **STM32F1**: Fixed hardware channels. ADC1 is hardwired to DMA1 Channel 1; USART2_RX is hardwired to DMA1 Channel 6.
    - **STM32F4**: Stream and Channel request matrix (e.g., Stream 0 Channel 0).
    - **STM32G4/L4/H7**: Dynamic request routing managed by the DMAMUX peripheral. Any peripheral can be mapped to any DMA channel.
3. **ADC External Hardware Triggers**:
    - Always verify the trigger table in the family Reference Manual (RM0008, RM0090, RM0440). Never assume trigger names are interchangeable across different chip families.
4. **Toolchain Version Parity**:
    - This document was verified against **STM32CubeIDE Version 2.2.0** and **STM32CubeMX Version 6.18.1**. Menu layouts, code generator options, and package directories may differ in legacy versions (CubeMX v4.x/v5.x).